Java DSA Interview Handbook — FAANG | Big Tech | Global Banks

Java DSA Interview Complete Handbook

Complete Preparation Guide for FAANG | Big Tech | Global Banks

---

Section 1.1 — Java Syntax, Variables, and Data Types

> **For experienced engineers:** Java syntax is C-like. If you know JavaScript (MERN), the concepts map directly — Java is just stricter about types and requires compilation.

---

1. Java Program Structure

```java
// Every Java program needs a class and a main method
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, FAANG!");
    }
}
```

Key differences from JavaScript: - Strongly typed — every variable needs a declared type - Compiled language — `javac Main.java` → `java Main` - No hoisting, no `var` ambiguity - Semicolons are mandatory

---

2. Primitive Data Types

Type	Size	Range	Default	Use Case
`byte`	1 byte	-128 to 127	0	Memory-sensitive arrays
`short`	2 bytes	-32,768 to 32,767	0	Rarely used
`int`	4 bytes	-2^31 to 2^31-1	0	**Most common integer type**
`long`	8 bytes	-2^63 to 2^63-1	0L	Large numbers, timestamps
`float`	4 bytes	~7 decimal digits	0.0f	Rarely used in DSA
`double`	8 bytes	~15 decimal digits	0.0	Floating point calculations
`char`	2 bytes	'000' to ''	'000'	Character manipulation
`boolean`	1 bit	true/false	false	Flags, conditions

```
// DSA-critical declarations
int n = 1_000_000;          // underscore for readability (Java 7+)
long bigNum = 2_000_000_000L;  // L suffix required for long literals
double pi = 3.14159;
char ch = 'A';
boolean found = false;

// Integer limits — memorize these for overflow detection
int MAX = Integer.MAX_VALUE;   // 2,147,483,647 (~2.1 billion)
int MIN = Integer.MIN_VALUE;   // -2,147,483,648
long LMAX = Long.MAX_VALUE;    // ~9.2 × 10^18

// Overflow trap — common interview bug:
int a = Integer.MAX_VALUE;
int b = a + 1;  // OVERFLOW: becomes -2147483648 (not an error!)
// Fix: use long
long safe = (long) a + 1;
```

3. Variable Declaration and Initialization

```
// Declaration
int x;           // declared, not initialized (value is undefined, not 0)
int y = 10;      // declared and initialized

// Multiple declarations
int p = 1, q = 2, r = 3;

// final (equivalent to const in JS)
final int MAX_SIZE = 100;  // cannot be reassigned

// Type inference with var (Java 10+) — use sparingly in interviews
var list = new ArrayList<Integer>();  // compiler infers ArrayList<Integer>

// Naming conventions (follow these in interviews)
int camelCase = 1;
final int UPPER_SNAKE_CASE = 100;  // constants
class PascalCase {}
```

---

4. Type Casting

```java
// Widening (automatic, safe)
int i = 42;
long l = i;       // int → long (automatic)
double d = i;     // int → double (automatic)

// Narrowing (explicit cast, may lose data)
double pi = 3.99;
int truncated = (int) pi;   // = 3 (truncates, doesn't round)

// char ↔ int casting (frequently used in DSA)
char c = 'A';
int ascii = c;              // = 65
char back = (char)(ascii + 1);  // = 'B'

// String ↔ int conversion (common in DSA)
int num = Integer.parseInt("42");
long lnum = Long.parseLong("123456789");
double dnum = Double.parseDouble("3.14");
String s = String.valueOf(42);      // "42"
String s2 = Integer.toString(42);   // "42"

// DSA trick: char digit to int
char digit = '7';
int val = digit - '0';   // = 7 (subtract ASCII of '0' = 48)
int letterIdx = 'e' - 'a';  // = 4 (0-indexed position in alphabet)
```

---

5. Wrapper Classes

> Used with Collections (which require objects, not primitives)

Primitive	Wrapper	Parse Method
`int`	`Integer`	`Integer.parseInt(s)`
`long`	`Long`	`Long.parseLong(s)`
`double`	`Double`	`Double.parseDouble(s)`
`char`	`Character`	`Character.isDigit(c)`
`boolean`	`Boolean`	`Boolean.parseBoolean(s)`

```java
// Autoboxing: primitive → wrapper (automatic)
Integer obj = 42;           // int → Integer

// Unboxing: wrapper → primitive (automatic)
int val = obj;              // Integer → int

// Null trap with unboxing — NullPointerException!
Integer nullObj = null;
int x = nullObj;            // THROWS NullPointerException

// Useful Integer methods for DSA
Integer.MAX_VALUE           // 2147483647
Integer.MIN_VALUE           // -2147483648
Integer.toBinaryString(42)  // "101010"
Integer.bitCount(42)        // number of set bits = 3
Integer.highestOneBit(42)   // 32 (highest power of 2 ≤ 42)
Integer.numberOfLeadingZeros(42)
Integer.reverse(42)         // bit reversal
Math.max(a, b)
Math.min(a, b)
Math.abs(-5)                // 5
Math.pow(2, 10)             // 1024.0 (returns double)
(int) Math.pow(2, 10)       // 1024

// Character utility methods (heavily used in string problems)
Character.isDigit('5')      // true
Character.isLetter('A')     // true
Character.isLetterOrDigit('a')
Character.isUpperCase('A')  // true
Character.isLowerCase('a')  // true
Character.toUpperCase('a')  // 'A'
Character.toLowerCase('A')  // 'a'
Character.isWhitespace(' ') // true
```

---

6. Input/Output for Competitive Coding

```java
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        // Fast Input (use BufferedReader for competitive coding with large inputs)
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim()); // number of test cases

        while (t-- > 0) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            int n = Integer.parseInt(st.nextToken());
            int k = Integer.parseInt(st.nextToken());

            int[] arr = new int[n];
            st = new StringTokenizer(br.readLine());
            for (int i = 0; i < n; i++) {
                arr[i] = Integer.parseInt(st.nextToken());
            }
        }

        // Fast Output (use StringBuilder + single print)
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 10; i++) {
            sb.append(i).append('\n');
        }
        System.out.print(sb);  // single I/O call is much faster
    }
}
```

```java
// Scanner — easier but slower (fine for LeetCode)
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
String s = sc.next();      // reads one word
String line = sc.nextLine(); // reads full line
double d = sc.nextDouble();

// Standard LeetCode — no input needed, just implement the method
class Solution {
    public int someMethod(int[] nums, int k) {
        // your solution
    }
}
```

---

7. Common DSA Interview Patterns Using Primitives

```java
// Counting character frequencies
int[] freq = new int[26];
String s = "hello";
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}

// XOR for finding unique element
int[] nums = {1, 2, 3, 2, 1};
int xor = 0;
for (int n : nums) xor ^= n;  // xor = 3

// Bit manipulation checks
boolean isPowerOfTwo = (n > 0) && (n & (n - 1)) == 0;
boolean isEven = (n & 1) == 0;
int lastBit = n & 1;
int clearLastBit = n & (n - 1);

// Integer overflow safe comparison (avoid (a + b) overflow)
// Bad: if (a + b > Integer.MAX_VALUE)
// Good:
if (a > Integer.MAX_VALUE - b) { /* overflow */ }
// Or cast to long first:
if ((long)a + b > Integer.MAX_VALUE) { /* overflow */ }
```

Summary — Key Takeaways

Concept	DSA Relevance
`int` overflow	Always check when summing large arrays
`char - 'a'`	Index into frequency array
`Integer.MAX_VALUE`	Initialize min tracking variables
`Integer.MIN_VALUE`	Initialize max tracking variables
`(int) Math.pow(2, k)`	Bit masks, knapsack sizing
Autoboxing trap	Never rely on `==` for Integer objects

> **Interview Tip:** When you see "find the sum of all elements", immediately ask: can the sum exceed *`Integer.MAX_VALUE`*? If yes, use *`long`*. Interviewers love this attention to detail.

Section 1.2 — Operators, Conditions, and Loops

---

1. Operators

Arithmetic Operators

```
int a = 17, b = 5;
int sum  = a + b;   // 22
int diff = a - b;   // 12
int prod = a * b;   // 85
int quot = a / b;   // 3  (integer division — truncates toward zero)
int rem  = a % b;   // 2  (modulo)

// DSA trap: negative modulo
int neg = -7 % 3;   // -1 in Java (NOT 2!)
// To always get positive result:
int posMod = ((n % m) + m) % m;  // canonical positive modulo formula

// Integer division rules — critical for binary search
int mid = (left + right) / 2;       // can overflow if left+right > MAX_VALUE
int safeMid = left + (right - left) / 2;  // preferred — overflow safe
```

Comparison Operators

```
a == b   // equal
a != b   // not equal
a <  b   // less than
a >  b   // greater than
a <= b   // less than or equal
a >= b   // greater than or equal

// WARNING: never use == to compare objects (Strings, Integer wrappers, etc.)
String s1 = new String("hello");
String s2 = new String("hello");
s1 == s2          // FALSE (compares references, not content)
s1.equals(s2)     // TRUE (compares content) — always use this

// Integer wrapper caching trap (common interview trick):
Integer x = 127;
Integer y = 127;
x == y      // TRUE (Integer cache: -128 to 127 are cached objects)
Integer p = 128;
Integer q = 128;
p == q      // FALSE (beyond cache range, different objects)
p.equals(q) // TRUE  — always use equals() for objects
```

Logical Operators

```
// Short-circuit evaluation
boolean a = true, b = false;
a && b    // AND: false (short-circuits if a is false)
a || b    // OR:  true  (short-circuits if a is true)
!a        // NOT: false

// Short-circuit safety pattern (avoid NullPointerException)
if (node != null && node.val == target) { ... }
// If node is null, the second condition is never evaluated

// Bitwise logical (operate on bits)
a & b     // bitwise AND
a | b     // bitwise OR
a ^ b     // bitwise XOR (different bits = 1)
~a        // bitwise complement
```

Bitwise Shift Operators

```
int n = 8;  // binary: 1000
n << 1      // = 16  (1000 → 10000): multiply by 2
n >> 1      // = 4   (1000 → 0100):  divide by 2 (arithmetic, preserves sign)
n >>> 1     // = 4   (unsigned right shift: fills with 0, not sign bit)

// Useful patterns
int pow2 = 1 << k;          // 2^k
boolean kthBitSet = (n >> k & 1) == 1;
int setKthBit = n | (1 << k);
int clearKthBit = n & ~(1 << k);
int toggleKthBit = n ^ (1 << k);
```

---

2. Conditional Statements

if / else if / else

```
int score = 85;

if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
} else {
    System.out.println("F");
}
```

Ternary Operator

```
// condition ? valueIfTrue : valueIfFalse
int max = (a > b) ? a : b;
String label = (n % 2 == 0) ? "even" : "odd";

// Nested ternary (use sparingly — reduces readability)
int sign = (n > 0) ? 1 : (n < 0) ? -1 : 0;
```

Switch Statement

```java
// Classic switch (Java 7+)
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    case 3: System.out.println("Wed"); break;
    default: System.out.println("Other");
}

// Switch expression (Java 14+) — cleaner syntax
String result = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    default -> "Other";
};

// String switch (Java 7+)
String direction = "NORTH";
switch (direction) {
    case "NORTH": break;
    case "SOUTH": break;
    // ...
}
```

3. Loops

for Loop

```java
// Standard indexed loop
for (int i = 0; i < n; i++) {
    // forward iteration
}

// Reverse iteration
for (int i = n - 1; i >= 0; i--) {
    // backward iteration
}

// Step iteration
for (int i = 0; i < n; i += 2) {
    // every other element
}

// 2D array traversal
int[][] matrix = new int[rows][cols];
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        // process matrix[i][j]
    }
}
```

Enhanced for Loop (for-each)

```java
int[] arr = {1, 2, 3, 4, 5};

// Array
for (int num : arr) {
    System.out.println(num);
}

// Collection
List<Integer> list = new ArrayList<>();
for (int val : list) { }

// Map
Map<String, Integer> map = new HashMap<>();
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String key = entry.getKey();
    int val = entry.getValue();
}
for (String key : map.keySet()) { }
for (int val : map.values()) { }

// String characters
String s = "hello";
for (char c : s.toCharArray()) { }
```

while Loop

```java
// Standard while
int i = 0;
while (i < n) {
    i++;
}

// do-while (executes at least once)
do {
    // process
} while (condition);

// Pattern: while with two pointers
int left = 0, right = n - 1;
while (left < right) {
    // two pointer logic
    left++;
    right--;
}

// Pattern: process digits
int num = 12345;
while (num > 0) {
    int digit = num % 10;
    num /= 10;
}
```

Loop Control Statements

```java
// break — exit the loop immediately
for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        break;
    }
}

// continue — skip current iteration
for (int i = 0; i < n; i++) {
    if (arr[i] < 0) continue;  // skip negative numbers
    process(arr[i]);
}

// Labeled break (for nested loops)
outer:
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == target) {
            System.out.println("Found at " + i + ", " + j);
            break outer;  // breaks out of BOTH loops
        }
    }
}
```

---

4. DSA-Critical Loop Patterns

Binary Search Loop Template

```
// Iterative binary search (memorize this exactly)
int left = 0, right = n - 1;
while (left <= right) {
    int mid = left + (right - left) / 2;  // overflow-safe
    if (arr[mid] == target) {
        return mid;
    } else if (arr[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
return -1;
```

Sliding Window Loop Pattern

```
int left = 0, maxLen = 0;
for (int right = 0; right < n; right++) {
    // expand window: add arr[right] to window

    while (/* window is invalid */) {
        // shrink window: remove arr[left] from window
        left++;
    }

    maxLen = Math.max(maxLen, right - left + 1);
}
```

Two Pointer Loop Pattern

```
int left = 0, right = n - 1;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) {
        // found pair
        left++; right--;
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
```

Counting Loop Patterns

```
// Count elements satisfying condition
int count = 0;
for (int num : arr) {
    if (num % 2 == 0) count++;
}

// Running sum (prefix sum building block)
int[] prefix = new int[n + 1];
for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}
// Range sum [l, r] = prefix[r+1] - prefix[l]
```

---

5. Performance Considerations

Loop Type	Time	When to Use
Simple for	O(n)	Indexed access needed
Enhanced for	O(n)	Read-only, cleaner syntax
while with two pointers	O(n)	Searching in sorted data
Nested loops	$O(n^2)$	Usually brute force — try to optimize
Binary search loop	O(log n)	Sorted data, answer space search

> **Interview Tip:** When you write a nested loop ($O(n^2)$), explicitly call it out: “This is $O(n^2)$ — let me think if we can do better.” Interviewers award points for identifying complexity, even before you optimize.

Section 1.3 — Functions, Arrays, and Strings

---

1. Functions (Methods) in Java

```java
// Method signature: modifier returnType methodName(params)
public static int add(int a, int b) {
    return a + b;
}

// void method (no return)
public static void printArray(int[] arr) {
    for (int n : arr) System.out.print(n + " ");
    System.out.println();
}

// Overloading — same name, different parameters
public static int max(int a, int b) { return a > b ? a : b; }
public static double max(double a, double b) { return a > b ? a : b; }

// Varargs — variable number of arguments
public static int sum(int... nums) {
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum(1, 2, 3, 4, 5);  // works

// Recursive method
public static int factorial(int n) {
    if (n <= 1) return 1;          // base case
    return n * factorial(n - 1);   // recursive case
}

// Static vs instance methods
// static: belongs to class, called as ClassName.method()
// instance: belongs to object, called as obj.method()
```

Pass by Value vs Pass by Reference

```java
// Java is ALWAYS pass-by-value
// For primitives: the value itself is copied
public static void changeInt(int x) {
    x = 100;  // only changes the local copy
}
int a = 5;
changeInt(a);
System.out.println(a);  // still 5

// For objects/arrays: the reference is copied (but points to same object)
public static void changeArray(int[] arr) {
    arr[0] = 100;  // modifies the actual array (reference was copied)
}
int[] nums = {1, 2, 3};
changeArray(nums);
System.out.println(nums[0]);  // 100 — array was modified

// To avoid mutation, copy the array:
int[] copy = arr.clone();
// or: Arrays.copyOf(arr, arr.length)
```

---

2. Arrays

Declaration and Initialization

```
// 1D arrays
int[] arr = new int[5];          // [0, 0, 0, 0, 0] — default values
int[] arr2 = {1, 2, 3, 4, 5};   // initialize with values
int[] arr3 = new int[]{1, 2, 3}; // explicit constructor

// Access and modify
arr[0] = 10;
int val = arr[2];
int len = arr.length;   // NOTE: .length (not .length() — that's for String)

// 2D arrays
int[][] matrix = new int[3][4];  // 3 rows, 4 cols
int[][] grid = {{1,2,3}, {4,5,6}, {7,8,9}};
int rows = grid.length;          // 3
int cols = grid[0].length;       // 3

// Jagged arrays (different row sizes)
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[1];
```

Arrays Utility Class (import java.util.Arrays)

```
import java.util.Arrays;

int[] arr = {5, 3, 1, 4, 2};

// Sorting — O(n log n)
Arrays.sort(arr);                    // sorts in-place: [1, 2, 3, 4, 5]
Arrays.sort(arr, 1, 4);             // sort subarray [1, 4) only

// Sort in reverse (requires Integer[] not int[])
Integer[] arr2 = {5, 3, 1, 4, 2};
Arrays.sort(arr2, (a, b) -> b - a); // descending: [5, 4, 3, 2, 1]

// Searching (array must be sorted)
int idx = Arrays.binarySearch(arr, 3);  // index of 3 in sorted arr

// Filling
Arrays.fill(arr, 0);           // fill with 0
Arrays.fill(arr, 1, 4, -1);    // fill index [1,4) with -1

// Copying
int[] copy = Arrays.copyOf(arr, arr.length);     // full copy
int[] partial = Arrays.copyOfRange(arr, 1, 4);   // copy [1, 4)

// Comparing
Arrays.equals(arr, copy);      // true if same length and same elements

// Converting to String (for debugging)
System.out.println(Arrays.toString(arr));        // [1, 2, 3, 4, 5]
System.out.println(Arrays.deepToString(matrix)); // for 2D arrays

// 2D sort — sort by first element, then second
int[][] points = {{3,1}, {1,2}, {1,0}};
Arrays.sort(points, (a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);
```

Critical Array Patterns for DSA

```
// Frequency array
int[] freq = new int[26];  // for lowercase letters
for (char c : s.toCharArray()) freq[c - 'a']++;

// Prefix sum array
int[] prefix = new int[n + 1];
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + arr[i];
int rangeSum = prefix[r + 1] - prefix[l];  // sum from index l to r inclusive

// Difference array (for range update in O(1))
int[] diff = new int[n + 1];
// Add val to range [l, r]:
diff[l] += val;
diff[r + 1] -= val;
// Reconstruct:
int[] result = new int[n];
result[0] = diff[0];
for (int i = 1; i < n; i++) result[i] = result[i-1] + diff[i];

// Monotonic stack using array
int[] stack = new int[n];
int top = -1;
stack[++top] = 0;  // push
top--;             // pop
stack[top];        // peek
```

---

3. Strings

> Java Strings are **immutable** — every modification creates a new String. This is critical for performance.

String Basics

```java
String s = "Hello World";
int len = s.length();          // 11 (method, not property — unlike array.length)
char c = s.charAt(3);          // 'l' (0-indexed)
int idx = s.indexOf('o');      // 4 (first occurrence)
int lastIdx = s.lastIndexOf('o'); // 7

// Substring — [startInclusive, endExclusive)
String sub = s.substring(6);      // "World"
String sub2 = s.substring(0, 5);  // "Hello"

// Comparison
s.equals("Hello World");          // true — always use equals() for strings
s.equalsIgnoreCase("hello world"); // true
s.compareTo("Hello");             // positive (lexicographic comparison)

// Search
s.contains("World");   // true
s.startsWith("He");    // true
s.endsWith("ld");      // true

// Case
s.toLowerCase();       // "hello world"
s.toUpperCase();       // "HELLO WORLD"

// Trim/strip
"  hello  ".trim();       // "hello" (removes leading/trailing whitespace)
"  hello  ".strip();      // "hello" (Unicode-aware, Java 11+)

// Replace
s.replace('l', 'r');              // "Herro Worrd"
s.replace("World", "Java");       // "Hello Java"
s.replaceAll("[aeiou]", "*");      // regex replace
s.replaceFirst("l", "L");         // "HeLlo World"

// Split
String[] parts = "a,b,c".split(",");  // ["a", "b", "c"]
String[] words = "hello world".split("\\s+");  // splits on whitespace

// Join
String joined = String.join("-", "a", "b", "c");  // "a-b-c"
String.join(", ", list);  // join collection

// Convert to char array
char[] chars = s.toCharArray();
```

```
// Check empty/blank
s.isEmpty();           // true if length == 0
s.isBlank();           // true if empty or only whitespace (Java 11+)
```

String to/from Integer

```
int n = Integer.parseInt("42");
String s = String.valueOf(42);      // "42"
String s2 = Integer.toString(42);  // "42"
String s3 = "" + 42;               // "42" (avoid — creates garbage)

// Number base conversions
Integer.toBinaryString(10);  // "1010"
Integer.toHexString(255);    // "ff"
Integer.toOctalString(8);    // "10"
Integer.parseInt("1010", 2); // 10 (parse binary string)
Integer.parseInt("ff", 16);  // 255 (parse hex string)
```

StringBuilder — Critical for Performance

```java
// String concatenation in loop — O(n²) due to immutability!
String result = "";
for (int i = 0; i < n; i++) {
    result += chars[i];  // creates new String each time — BAD
}

// StringBuilder — O(n)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) {
    sb.append(chars[i]);
}
String result = sb.toString();

// StringBuilder methods
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");         // "Hello World"
sb.insert(5, ",");           // "Hello, World"
sb.delete(5, 7);             // "Hello World"
sb.deleteCharAt(0);          // "ello World"
sb.replace(0, 4, "Hi");      // "Hi World"
sb.reverse();                // "dlroW iH"
sb.charAt(0);                // 'd'
sb.setCharAt(0, 'D');        // "DlroW iH"
sb.length();
sb.toString();               // convert to String
sb.indexOf("World");
sb.lastIndexOf("l");

// StringBuilder as stack (for bracket matching, etc.)
StringBuilder stack = new StringBuilder();
stack.append(c);                          // push
stack.deleteCharAt(stack.length() - 1);   // pop
stack.charAt(stack.length() - 1);         // peek
stack.length() == 0;                      // isEmpty
```

String Patterns for DSA

```java
// Palindrome check
public boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}

// Anagram check (same characters, different order)
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] freq = new int[26];
    for (char c : s.toCharArray()) freq[c - 'a']++;
    for (char c : t.toCharArray()) {
        freq[c - 'a']--;
        if (freq[c - 'a'] < 0) return false;
    }
    return true;
}

// Reverse words
public String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(" ");
    }
    return sb.toString();
}

// Check all characters unique
public boolean allUnique(String s) {
    boolean[] seen = new boolean[128];  // ASCII
    for (char c : s.toCharArray()) {
        if (seen[c]) return false;
        seen[c] = true;
    }
    return true;
}

// Longest common prefix
public String longestCommonPrefix(String[] strs) {
```

```
        if (strs.length == 0) return "";
        String prefix = strs[0];
        for (String s : strs) {
            while (!s.startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
```

---

4. Complexity Reference

Operation	Array	String
Access by index	O(1)	O(1) via charAt()
Search (unsorted)	O(n)	O(n) via indexOf()
Sort	O(n log n)	O(n log n) via toCharArray+sort
Concatenation	—	O(n) per concat, use StringBuilder
Substring	—	O(n)
StringBuilder append	—	O(1) amortized

> **Interview Tip:** Always use `StringBuilder` for string building in loops. Mention it unprompted — it shows you understand Java internals. A senior engineer who writes `result += s` in a loop is a red flag to interviewers.

Section 1.4 — OOP Concepts (Java)

> For DSA interviews at FAANG/Big Tech, OOP is tested through system design rounds and when implementing custom data structures (LRU Cache, Trie, Graph Node, etc.)

---

1. Classes and Objects

```java
// Class definition
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    // Constructor
    TreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }

    // Overloaded constructor
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

// Creating objects
TreeNode root = new TreeNode(5);
TreeNode node = new TreeNode(3, null, null);

// Standard LeetCode node definitions you must know by heart:
// ListNode (Linked List)
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

// TreeNode (Binary Tree)
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

2. Encapsulation

```
public class BankAccount {
    private double balance;    // private — cannot access from outside
    private String accountId;

    public BankAccount(String id, double initial) {
        this.accountId = id;
        this.balance = initial;
    }

    // Getter
    public double getBalance() { return balance; }

    // Setter with validation
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
}
```

---

3. Inheritance

```java
// Base class
public class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public double area() { return 0; }  // can be overridden

    public String toString() {
        return "Shape[color=" + color + "]";
    }
}

// Derived class
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);  // call parent constructor
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}

public class Rectangle extends Shape {
    private double width, height;

    public Rectangle(String color, double w, double h) {
        super(color);
        this.width = w;
        this.height = h;
    }

    @Override
    public double area() { return width * height; }
}

// Usage
```

```
Shape s = new Circle("red", 5);
s.area();  // calls Circle's area() — runtime polymorphism
```

---

4. Polymorphism

```
// Compile-time polymorphism: Method Overloading
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

// Runtime polymorphism: Method Overriding
Shape[] shapes = {new Circle("red", 3), new Rectangle("blue", 4, 5)};
for (Shape shape : shapes) {
    System.out.println(shape.area());  // calls the correct subclass method
}

// instanceof check
if (shape instanceof Circle) {
    Circle c = (Circle) shape;  // safe cast
}
// Java 16+ pattern matching:
if (shape instanceof Circle c) {
    System.out.println(c.radius);  // no explicit cast needed
}
```

---

5. Abstraction

```
// Abstract class — cannot be instantiated, may have both abstract and concrete methods
abstract class Vehicle {
    String brand;

    Vehicle(String brand) { this.brand = brand; }

    // Abstract method — must be implemented by subclass
    abstract double fuelEfficiency();

    // Concrete method — inherited as-is
    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Efficiency: " + fuelEfficiency());
    }
}

class Car extends Vehicle {
    double mpg;
    Car(String brand, double mpg) {
        super(brand);
        this.mpg = mpg;
    }

    @Override
    double fuelEfficiency() { return mpg; }
}
```

6. Interfaces

```java
// Interface — pure abstraction (all methods abstract by default in Java 7)
interface Comparable<T> {
    int compareTo(T other);
}

// Interface with default method (Java 8+)
interface Printable {
    void print();

    default void printTwice() {  // default implementation
        print();
        print();
    }

    static void info() {  // static method in interface
        System.out.println("Printable interface");
    }
}

// Implementing multiple interfaces (Java's answer to multiple inheritance)
class Document implements Printable, Serializable {
    String content;

    @Override
    public void print() { System.out.println(content); }
}

// Functional interface (1 abstract method) — used with lambdas
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b);
}

MathOperation add = (a, b) -> a + b;
MathOperation mul = (a, b) -> a * b;
System.out.println(add.operate(3, 4));  // 7
```

Abstract Class vs Interface — Interview Cheat Sheet

	Abstract Class	Interface
Instantiation	No	No
Constructor	Yes	No
Fields	Any type	`public static final` only
Methods	Abstract + concrete	Abstract + default + static
Multiple inheritance	No (single extends)	Yes (multiple implements)
When to use	Shared state + partial implementation	Contract / capability definition

---

7. Generics

```
// Generic class
class Pair<A, B> {
    A first;
    B second;

    Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }
}

Pair<String, Integer> p = new Pair<>("Alice", 25);

// Generic method
public static <T extends Comparable<T>> T findMax(T[] arr) {
    T max = arr[0];
    for (T item : arr) {
        if (item.compareTo(max) > 0) max = item;
    }
    return max;
}

// Bounded type parameters
class MinHeap<T extends Comparable<T>> { ... }

// Wildcard — use when you don't care about specific type
public static double sumList(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) sum += n.doubleValue();
    return sum;
}
sumList(List.of(1, 2, 3));       // works with Integer
sumList(List.of(1.1, 2.2, 3.3)); // works with Double

// Upper bounded: <? extends T> — read-only
// Lower bounded: <? super T>   — write-capable
```

8. OOP in DSA — Practical Examples

Custom Comparator

```
// Implementing Comparable in a custom class
class Interval implements Comparable<Interval> {
    int start, end;

    Interval(int start, int end) {
        this.start = start;
        this.end = end;
    }

    @Override
    public int compareTo(Interval other) {
        return this.start - other.start;  // sort by start time
    }
}

// Using Comparator (separate from class)
Comparator<int[]> comp = (a, b) -> a[0] - b[0];  // sort 2D array by first element
Arrays.sort(intervals, comp);
```

LRU Cache Using OOP

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, Integer> cache;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        // LinkedHashMap with access order = true implements LRU naturally
        this.cache = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
                return size() > capacity;
            }
        };
    }

    public int get(int key) {
        return cache.getOrDefault(key, -1);
    }

    public void put(int key, int value) {
        cache.put(key, value);
    }
}
```

Union-Find (Disjoint Set) — OOP Implementation

```
class UnionFind {
    private int[] parent;
    private int[] rank;
    private int components;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        components = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);  // path compression
        }
        return parent[x];
    }

    public boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;  // already connected

        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }

        components--;
        return true;
    }

    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }

    public int getComponents() { return components; }
}
```

Summary

OOP Concept	DSA Application
Class/Object	Custom node types (TreeNode, ListNode, GraphNode)
Encapsulation	LRU Cache, Trie, Segment Tree
Inheritance	Problem-specific node hierarchies
Polymorphism	Comparator patterns, generic algorithms
Abstract class	Abstract graph/tree base classes
Interface	Comparable, Iterable, custom function interfaces
Generics	Generic Pair, generic data structures

Section 1.5 — Generics, Lambda Expressions, and Streams

---

1. Lambda Expressions (Java 8+)

> Lambdas replace anonymous inner classes for functional interfaces. Essential for sorting, filtering, and custom comparators in interviews.

Syntax

```
// Old way (anonymous inner class)
Comparator<Integer> oldComp = new Comparator<Integer>() {
    @Override
    public int compare(Integer a, Integer b) {
        return a - b;
    }
};

// Lambda way
Comparator<Integer> comp = (a, b) -> a - b;

// Lambda forms
() -> expression                    // no args
(x) -> expression                   // one arg
(x, y) -> expression                // two args
(x, y) -> { statement1; statement2; return result; }  // block body
```

Lambda in Sorting (Most Common DSA Use)

```java
// Sort integers descending
Integer[] arr = {3, 1, 4, 1, 5, 9};
Arrays.sort(arr, (a, b) -> b - a);  // descending

// WARNING: (b - a) can overflow for large values. Use Integer.compare instead:
Arrays.sort(arr, (a, b) -> Integer.compare(b, a));  // safe descending

// Sort strings by length
String[] words = {"banana", "fig", "apple", "kiwi"};
Arrays.sort(words, (a, b) -> a.length() - b.length());

// Sort 2D array by first element, then second
int[][] points = {{1,2}, {1,0}, {2,1}};
Arrays.sort(points, (a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);

// Sort by multiple criteria (chain comparators)
List<String> names = Arrays.asList("Bob", "Alice", "Charlie", "Dave", "Ann");
names.sort(Comparator.comparingInt(String::length)
           .thenComparing(Comparator.naturalOrder()));

// Sort with null safety
List<Integer> list = Arrays.asList(3, null, 1, null, 2);
list.sort(Comparator.nullsLast(Integer::compareTo));
```

Method References

```
// lambda: x -> System.out.println(x)
// method ref: System.out::println

// Types of method references:
// 1. Static method
Function<String, Integer> parser = Integer::parseInt;

// 2. Instance method of a specific object
String prefix = "Hello";
Predicate<String> startsWith = prefix::startsWith;

// 3. Instance method of arbitrary instance of a type
Function<String, String> toUpper = String::toUpperCase;

// 4. Constructor
Supplier<ArrayList<Integer>> listFactory = ArrayList::new;
```

---

2. Functional Interfaces (from java.util.function)

```
// Predicate<T>: T → boolean
Predicate<Integer> isEven = n -> n % 2 == 0;
isEven.test(4);          // true
isEven.negate().test(4); // false
Predicate<Integer> isPositive = n -> n > 0;
isEven.and(isPositive).test(4);  // true (both)
isEven.or(isPositive).test(-2);  // true (one)

// Function<T, R>: T → R
Function<String, Integer> strLen = String::length;
strLen.apply("hello");  // 5
// Chaining: andThen, compose
Function<Integer, Integer> triple = x -> x * 3;
Function<Integer, Integer> addTen = x -> x + 10;
Function<Integer, Integer> tripleAndAdd = triple.andThen(addTen);
tripleAndAdd.apply(5);  // 25

// BiFunction<T, U, R>: (T, U) → R
BiFunction<String, Integer, String> repeat = (s, n) -> s.repeat(n);

// Supplier<T>: () → T
Supplier<List<Integer>> newList = ArrayList::new;

// Consumer<T>: T → void
Consumer<String> printer = System.out::println;
printer.accept("Hello");

// UnaryOperator<T>: T → T (special Function where input = output type)
UnaryOperator<String> trim = String::trim;

// BinaryOperator<T>: (T, T) → T
BinaryOperator<Integer> max = Math::max;
```

---

3. Streams (Java 8+)

> Streams allow declarative, pipeline-style data processing. Not always needed in DSA, but useful for clean code in simpler problems.

Stream Pipeline

```
// source → intermediate operations → terminal operation
List<Integer> nums = Arrays.asList(5, 3, 1, 4, 2, 6, 8, 7);

// Example pipeline:
int result = nums.stream()          // source
    .filter(n -> n > 3)             // intermediate: keep elements > 3
    .map(n -> n * 2)                // intermediate: double each
    .sorted()                        // intermediate: sort
    .reduce(0, Integer::sum);        // terminal: sum

System.out.println(result);  // (4*2 + 5*2 + 6*2 + 7*2 + 8*2) = 60
```

Creating Streams

```
// From collection
List<Integer> list = Arrays.asList(1, 2, 3);
Stream<Integer> s1 = list.stream();
Stream<Integer> s2 = list.parallelStream();  // parallel processing

// From array
int[] arr = {1, 2, 3};
IntStream is = Arrays.stream(arr);           // primitive stream (more efficient)
Stream<Integer> bs = Arrays.stream(arr).boxed(); // to object stream

// From values
Stream<String> s3 = Stream.of("a", "b", "c");

// Range streams (useful in DSA)
IntStream range = IntStream.range(0, 10);       // [0, 10)
IntStream rangeClosed = IntStream.rangeClosed(1, 10); // [1, 10]

// Generate (infinite — must use limit)
Stream<Integer> zeros = Stream.generate(() -> 0).limit(5);
Stream<Integer> seq = Stream.iterate(0, n -> n + 1).limit(10);
```

Intermediate Operations

```
List<String> words = Arrays.asList("hello", "world", "java", "stream");

// filter: keep elements matching predicate
words.stream().filter(s -> s.length() > 4)  // ["hello", "world", "stream"]

// map: transform each element
words.stream().map(String::toUpperCase)  // ["HELLO", "WORLD", "JAVA", "STREAM"]
words.stream().mapToInt(String::length)  // IntStream: [5, 5, 4, 6]

// flatMap: flatten nested streams
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1, 2), Arrays.asList(3, 4)
);
nested.stream().flatMap(Collection::stream)  // [1, 2, 3, 4]

// distinct: remove duplicates
Stream.of(1, 2, 2, 3, 3, 3).distinct()  // [1, 2, 3]

// sorted
words.stream().sorted()                          // alphabetical
words.stream().sorted(Comparator.reverseOrder()) // reverse
words.stream().sorted(Comparator.comparingInt(String::length)) // by length

// limit / skip
words.stream().limit(2)   // first 2 elements
words.stream().skip(1)    // skip first 1, return rest

// peek (for debugging — same as forEach but intermediate)
words.stream().peek(System.out::println).filter(s -> s.length() > 4)
```

Terminal Operations

```java
// collect: gather into collection
List<String> filtered = words.stream()
    .filter(s -> s.length() > 4)
    .collect(Collectors.toList());

// Collect to different collection types
Set<String> set = words.stream().collect(Collectors.toSet());
List<String> unmodifiable = words.stream().collect(Collectors.toUnmodifiableList());

// Joining strings
String joined = words.stream().collect(Collectors.joining(", "));  // "hello, world, java, stream"
String joined2 = words.stream().collect(Collectors.joining(", ", "[", "]")); // "[hello, world, ...]"

// Grouping (very useful for frequency maps)
Map<Integer, List<String>> byLength = words.stream()
    .collect(Collectors.groupingBy(String::length));
// {5=[hello, world], 4=[java], 6=[stream]}

// Counting by group
Map<Integer, Long> countByLength = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));

// Partitioning (split into two groups)
Map<Boolean, List<String>> partition = words.stream()
    .collect(Collectors.partitioningBy(s -> s.length() > 4));
// {true=[hello, world, stream], false=[java]}

// Convert to Map
Map<String, Integer> wordLengths = words.stream()
    .collect(Collectors.toMap(
        w -> w,           // key function
        String::length    // value function
    ));

// forEach
words.stream().forEach(System.out::println);

// count
long count = words.stream().filter(s -> s.length() > 4).count();

// reduce
int sum = IntStream.rangeClosed(1, 10).reduce(0, Integer::sum);
Optional<Integer> product = IntStream.rangeClosed(1, 5).boxed()
    .reduce((a, b) -> a * b);
```

```java
// min / max
Optional<String> shortest = words.stream().min(Comparator.comparingInt(String::length));
Optional<String> longest = words.stream().max(Comparator.comparingInt(String::length));

// findFirst / findAny
Optional<String> first = words.stream().filter(s -> s.startsWith("j")).findFirst();

// anyMatch / allMatch / noneMatch
boolean any = words.stream().anyMatch(s -> s.contains("ava")); // true
boolean all = words.stream().allMatch(s -> s.length() > 2);   // true
boolean none = words.stream().noneMatch(s -> s.contains("z")); // true

// toArray
Object[] arr = words.stream().toArray();
String[] strArr = words.stream().toArray(String[]::new);
```

IntStream for DSA

```java
// Sum of array
int sum = IntStream.of(arr).sum();

// Average
OptionalDouble avg = IntStream.of(arr).average();

// Statistics
IntSummaryStatistics stats = IntStream.of(arr).summaryStatistics();
stats.getMax();
stats.getMin();
stats.getSum();
stats.getAverage();
stats.getCount();

// Convert int[] to List<Integer>
List<Integer> list = IntStream.of(arr).boxed().collect(Collectors.toList());

// Convert List<Integer> to int[]
int[] arr2 = list.stream().mapToInt(Integer::intValue).toArray();

// Range sum
int rangeSum = IntStream.rangeClosed(1, 100).sum();  // 5050
```

---

4. Optional

```
// Avoid null checks with Optional
Optional<String> opt = Optional.of("hello");
Optional<String> empty = Optional.empty();
Optional<String> nullable = Optional.ofNullable(null);  // null → empty Optional

// Checking
opt.isPresent()    // true
opt.isEmpty()      // false (Java 11+)
empty.isPresent()  // false

// Getting value
opt.get()          // "hello" (throws NoSuchElementException if empty)
opt.orElse("default")           // value or default
opt.orElseGet(() -> compute())  // lazy default
opt.orElseThrow(() -> new RuntimeException("missing"))

// Transforming
opt.map(String::toUpperCase)    // Optional<String>["HELLO"]
opt.filter(s -> s.length() > 3) // Optional<String>["hello"] (length 5 > 3)
opt.flatMap(s -> Optional.of(s.toUpperCase())) // flatten nested Optional

// Usage pattern
Optional<User> user = findUser(id);
String name = user.map(User::getName).orElse("Unknown");
```

Summary

Feature	DSA Use Case
Lambda	Custom comparators, sort logic
Method reference	Cleaner function passing
Stream.filter	Data filtering problems
Stream.map	Element transformation
Collectors.groupingBy	Frequency map, grouping problems
IntStream	Numeric computations, range operations
Optional	Null-safe return types

> **Interview Tip:** Don't over-use streams in interviews — they can obscure your algorithmic thinking. Use them for simple transformations, but for complex logic (nested loops, conditions), stick to explicit code. Interviewers want to see your logic, not your lambda fluency.

Section 2.1 — List: ArrayList and LinkedList

The List Interface

```
// List is an interface — ArrayList and LinkedList are implementations
List<Integer> list = new ArrayList<>();    // most common
List<Integer> list2 = new LinkedList<>();  // use when frequent head/tail ops

// Common operations (both implementations)
list.add(10);              // append to end
list.add(0, 5);            // insert at index 0
list.get(0);               // get by index
list.set(0, 100);          // update at index
list.remove(0);            // remove by index (returns removed element)
list.remove(Integer.valueOf(10));  // remove by value (must box int → Integer)
list.size();               // number of elements
list.isEmpty();            // true if size == 0
list.contains(10);         // O(n) linear search
list.indexOf(10);          // first occurrence index (-1 if not found)
list.lastIndexOf(10);      // last occurrence index
list.clear();              // remove all elements
```

ArrayList

Internal Working

`ArrayList = dynamic array - Backed by: Object[] array - Initial capacity: 10 (default) - Growth factor: 1.5x w`

Time Complexity

Operation	Complexity	Notes
`add(e)`	O(1) amortized	O(n) on resize
`add(i, e)`	O(n)	Shifts elements right
`get(i)`	O(1)	Direct array access
`set(i, e)`	O(1)	Direct array update
`remove(i)`	O(n)	Shifts elements left
`contains(e)`	O(n)	Linear scan
`indexOf(e)`	O(n)	Linear scan
`size()`	O(1)	Cached field

Complete ArrayList API

```java
import java.util.*;

List<Integer> list = new ArrayList<>();

// Adding elements
list.add(1);               // [1]
list.add(2);               // [1, 2]
list.add(3);               // [1, 2, 3]
list.add(0, 0);            // [0, 1, 2, 3] — O(n)
list.addAll(Arrays.asList(4, 5, 6));  // [0, 1, 2, 3, 4, 5, 6]
list.addAll(2, Arrays.asList(10, 11)); // insert at index 2

// Accessing elements
list.get(0);               // 0
list.size();               // size
list.isEmpty();            // false

// Modifying elements
list.set(0, 99);           // replace index 0 with 99

// Removing elements
list.remove(0);            // remove by index, returns removed element
list.remove(Integer.valueOf(99)); // remove by value (first occurrence)
list.removeAll(Arrays.asList(1, 2)); // remove all matching
list.retainAll(Arrays.asList(3, 4)); // keep only these values

// Searching
list.contains(3);          // true
list.indexOf(3);           // first occurrence
list.lastIndexOf(3);       // last occurrence

// Sorting
Collections.sort(list);                        // ascending
Collections.sort(list, Collections.reverseOrder()); // descending
list.sort((a, b) -> b - a);                    // lambda comparator

// Sub-list (view, not copy — modifications affect original)
List<Integer> sub = list.subList(1, 4);  // [1, 4) elements

// Convert to array
Object[] arr = list.toArray();
Integer[] intArr = list.toArray(new Integer[0]);
int[] primitiveArr = list.stream().mapToInt(Integer::intValue).toArray();

// Create from array
```

```java
List<Integer> fromArr = Arrays.asList(1, 2, 3);    // Fixed size!
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3)); // Mutable

// Immutable list (Java 9+)
List<Integer> immutable = List.of(1, 2, 3);  // cannot add/remove/set

// Iterate
for (int val : list) { }
list.forEach(System.out::println);
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int val = it.next();
    if (val < 0) it.remove();  // safe removal during iteration
}

// Capacity management (optimization)
ArrayList<Integer> al = new ArrayList<>(1000);  // pre-allocate capacity
al.ensureCapacity(2000);  // grow if needed
al.trimToSize();          // release unused memory
```

DSA Patterns with ArrayList

```
// Dynamic array as stack
List<Integer> stack = new ArrayList<>();
stack.add(val);                          // push
stack.remove(stack.size() - 1);          // pop (O(1))
stack.get(stack.size() - 1);             // peek (O(1))

// Building result list during DFS/BFS
List<Integer> path = new ArrayList<>();
path.add(node.val);
// ... recurse ...
path.remove(path.size() - 1);  // backtrack

// Frequency counting with List
List<Integer>[] buckets = new ArrayList[n + 1];
for (int i = 0; i <= n; i++) buckets[i] = new ArrayList<>();
buckets[freq[i]].add(i);

// 2D result
List<List<Integer>> result = new ArrayList<>();
List<Integer> row = new ArrayList<>();
row.add(1); row.add(2);
result.add(row);
```

LinkedList

Internal Working

`LinkedList = doubly linked list - Each node has: data, prev pointer, next pointer - No array backing — pure no`

Time Complexity

Operation	Complexity	Notes
`addFirst(e)` / `addLast(e)`	O(1)	Head/tail pointers
`add(e)`	O(1)	Add to tail
`add(i, e)`	O(n)	Must traverse to index
`get(i)`	O(n)	Must traverse to index
`removeFirst()` / `removeLast()`	O(1)	Head/tail pointers
`remove(i)`	O(n)	Must traverse + O(1) removal
`contains(e)`	O(n)	Linear scan

When to Use LinkedList vs ArrayList

Use ArrayList when	Use LinkedList when
Random access needed (O(1) get)	Frequent head insertions/deletions
Memory efficiency matters	Implementing queue/deque
Most operations are at end	Frequent insertions in middle (if you have the node)
Cache-friendly iteration	Order of insertion matters, no index access

In practice: ArrayList is preferred 90% of the time. Use ArrayDeque for queue/stack operations.

```java
// LinkedList as Deque (double-ended queue)
LinkedList<Integer> deque = new LinkedList<>();
deque.addFirst(1);     // add to head
deque.addLast(2);      // add to tail
deque.removeFirst();   // remove from head
deque.removeLast();    // remove from tail
deque.peekFirst();     // head without removal
deque.peekLast();      // tail without removal

// Manual LinkedList implementation (for interviews)
class MyLinkedList {
    private static class Node {
        int val;
        Node next;
        Node(int val) { this.val = val; }
    }

    private Node head;
    private int size;

    public void addAtHead(int val) {
        Node node = new Node(val);
        node.next = head;
        head = node;
        size++;
    }

    public void addAtTail(int val) {
        if (head == null) { addAtHead(val); return; }
        Node curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = new Node(val);
        size++;
    }

    public int get(int index) {
        if (index < 0 || index >= size) return -1;
        Node curr = head;
        for (int i = 0; i < index; i++) curr = curr.next;
        return curr.val;
    }

    public void deleteAtIndex(int index) {
        if (index < 0 || index >= size) return;
        if (index == 0) { head = head.next; size--; return; }
```

```
            Node curr = head;
            for (int i = 0; i < index - 1; i++) curr = curr.next;
            curr.next = curr.next.next;
            size--;
        }
    }
```

Interview Traps and Tips

```
// Trap 1: ConcurrentModificationException
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4));
for (int val : list) {
    if (val % 2 == 0) list.remove(Integer.valueOf(val));  // THROWS CME!
}
// Fix: use Iterator.remove() or removeIf
list.removeIf(val -> val % 2 == 0);  // Java 8+ cleanest way

// Trap 2: Arrays.asList returns fixed-size List
List<Integer> fixed = Arrays.asList(1, 2, 3);
fixed.add(4);  // THROWS UnsupportedOperationException!
// Fix:
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3));

// Trap 3: remove(int) vs remove(Object)
List<Integer> list2 = new ArrayList<>(Arrays.asList(1, 2, 3));
list2.remove(1);                // removes INDEX 1 → list becomes [1, 3]
list2.remove(Integer.valueOf(1)); // removes VALUE 1 → list becomes [2, 3]

// Trap 4: subList is a view
List<Integer> original = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
List<Integer> sub = original.subList(1, 4);  // [2, 3, 4]
sub.set(0, 99);  // also modifies original! original = [1, 99, 3, 4, 5]
// To get independent copy:
List<Integer> copy = new ArrayList<>(original.subList(1, 4));
```

Complexity Summary

	ArrayList	LinkedList
Access by index	O(1)	O(n)
Insert/Delete at end	O(1) amortized	O(1)
Insert/Delete at head	O(n)	O(1)
Insert/Delete in middle	O(n)	O(n)
Memory overhead	Low (array)	High (2 pointers per node)
Cache performance	Excellent	Poor (random memory)

Section 2.2 — Map: HashMap and TreeMap

The Map Interface

```
// Map stores key-value pairs, keys are unique
Map<String, Integer> map = new HashMap<>();     // unordered, O(1) ops
Map<String, Integer> tree = new TreeMap<>();    // sorted by key, O(log n)
Map<String, Integer> linked = new LinkedHashMap<>(); // insertion order

// Common operations
map.put("a", 1);
map.get("a");           // 1
map.containsKey("a");   // true
map.containsValue(1);   // true (O(n)!)
map.remove("a");
map.size();
map.isEmpty();
```

HashMap

Internal Working (Critical for Interviews)

`HashMap = Hash Table with Separate Chaining (Java 8+: with Tree optimization) Internal structure: - Array of b`

Time Complexity

Operation	Average	Worst Case	Notes
`put(k, v)`	O(1)	O(n)	Worst: all keys hash to same bucket
`get(k)`	O(1)	O(log n)	Java 8+: tree buckets
`remove(k)`	O(1)	O(log n)	
`containsKey(k)`	O(1)	O(log n)	
`containsValue(v)`	O(n)	O(n)	Must scan all values
Iteration	O(n)	O(n)	Visits all entries

Complete HashMap API

```java
Map<String, Integer> map = new HashMap<>();

// Put / Get
map.put("apple", 3);
map.put("banana", 5);
int val = map.get("apple");         // 3
int def = map.getOrDefault("cherry", 0);  // 0 (key not present)

// Check existence
map.containsKey("apple");   // true
map.containsValue(5);       // true (O(n))

// Remove
map.remove("apple");                    // remove key, returns old value
map.remove("banana", 5);               // conditional remove (only if value matches)

// Size
map.size();
map.isEmpty();

// Iteration
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String key = entry.getKey();
    int value = entry.getValue();
}
for (String key : map.keySet()) { }
for (int value : map.values()) { }
map.forEach((k, v) -> System.out.println(k + ": " + v));

// Advanced operations (Java 8+)
// putIfAbsent — only put if key not present
map.putIfAbsent("apple", 10);  // doesn't overwrite existing

// computeIfAbsent — compute and put if absent
map.computeIfAbsent("cherry", k -> k.length());  // "cherry" → 6
// Very useful for building adjacency lists:
adjList.computeIfAbsent(node, k -> new ArrayList<>()).add(neighbor);

// computeIfPresent — update only if key exists
map.computeIfPresent("banana", (k, v) -> v + 1);  // 5 → 6

// compute — always compute new value
map.compute("apple", (k, v) -> (v == null) ? 1 : v + 1);

// merge — merge with existing value
```

```java
map.merge("apple", 1, Integer::sum);  // if absent: put 1; if present: sum

// replaceAll
map.replaceAll((k, v) -> v * 2);

// getOrDefault chaining for frequency maps
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
// Or using merge:
for (char c : s.toCharArray()) {
    freq.merge(c, 1, Integer::sum);
}
// Or using compute:
for (char c : s.toCharArray()) {
    freq.compute(c, (k, v) -> v == null ? 1 : v + 1);
}
```

HashMap DSA Patterns

```java
// Pattern 1: Frequency map (most common)
Map<Integer, Integer> count = new HashMap<>();
for (int n : nums) count.put(n, count.getOrDefault(n, 0) + 1);

// Pattern 2: Two Sum
Map<Integer, Integer> seen = new HashMap<>(); // val → index
for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (seen.containsKey(complement)) return new int[]{seen.get(complement), i};
    seen.put(nums[i], i);
}

// Pattern 3: Group anagrams
Map<String, List<String>> groups = new HashMap<>();
for (String word : words) {
    char[] chars = word.toCharArray();
    Arrays.sort(chars);
    String key = new String(chars);
    groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
}

// Pattern 4: Track first occurrence index
Map<Integer, Integer> firstSeen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    if (!firstSeen.containsKey(nums[i])) firstSeen.put(nums[i], i);
}

// Pattern 5: Adjacency list (graph)
Map<Integer, List<Integer>> adj = new HashMap<>();
for (int[] edge : edges) {
    adj.computeIfAbsent(edge[0], k -> new ArrayList<>()).add(edge[1]);
    adj.computeIfAbsent(edge[1], k -> new ArrayList<>()).add(edge[0]);
}

// Pattern 6: Count subarrays with target sum (prefix sum + hashmap)
Map<Integer, Integer> prefixCount = new HashMap<>();
prefixCount.put(0, 1);  // empty subarray has sum 0
int sum = 0, count = 0;
for (int n : nums) {
    sum += n;
    count += prefixCount.getOrDefault(sum - target, 0);
    prefixCount.put(sum, prefixCount.getOrDefault(sum, 0) + 1);
}
```

LinkedHashMap

```java
// Maintains insertion order
Map<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("c", 3);
lhm.put("a", 1);
lhm.put("b", 2);
// Iteration order: c, a, b (insertion order)

// Access order (most recently accessed last) — LRU Cache foundation
Map<Integer, Integer> lru = new LinkedHashMap<>(16, 0.75f, true);
// access-ordered: get() moves key to end

// LRU Cache with LinkedHashMap (FAANG favorite)
class LRUCache extends LinkedHashMap<Integer, Integer> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true);  // access-order
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getOrDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
        return size() > capacity;
    }
}
```

TreeMap

Internal Working

`TreeMap = Red-Black Tree (self-balancing BST) - Keys are stored in sorted order - All operations: O(log n) - A`

Time Complexity

Operation	Complexity
`put(k, v)`	O(log n)
`get(k)`	O(log n)
`remove(k)`	O(log n)
`containsKey(k)`	O(log n)
`firstKey()` / `lastKey()`	O(log n)
`floorKey(k)` / `ceilingKey(k)`	O(log n)
Iteration	O(n)

```java
TreeMap<Integer, String> tree = new TreeMap<>();
tree.put(5, "five");
tree.put(1, "one");
tree.put(3, "three");
tree.put(7, "seven");

// Sorted iteration
for (Map.Entry<Integer, String> e : tree.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
// 1:one, 3:three, 5:five, 7:seven

// NavigableMap operations (unique to TreeMap)
tree.firstKey();          // 1 (smallest)
tree.lastKey();           // 7 (largest)
tree.floorKey(4);         // 3 (largest key <= 4)
tree.ceilingKey(4);       // 5 (smallest key >= 4)
tree.lowerKey(5);         // 3 (strictly less than 5)
tree.higherKey(5);        // 7 (strictly greater than 5)
tree.pollFirstEntry();    // removes and returns entry with smallest key
tree.pollLastEntry();     // removes and returns entry with largest key

// SubMap operations (range queries)
tree.subMap(1, true, 5, true);   // keys in [1, 5] inclusive
tree.headMap(5);                 // keys strictly less than 5
tree.tailMap(3);                 // keys >= 3
tree.descendingMap();            // reverse order view
tree.descendingKeySet();         // reverse order keys

// Use case: find k-th smallest sum, sliding window maximum
// Use case: count smaller numbers — TreeMap + rank
```

TreeMap DSA Patterns

```
// Pattern 1: Range counting
TreeMap<Integer, Integer> freqMap = new TreeMap<>();
// How many keys in range [lo, hi]?
int count = freqMap.subMap(lo, true, hi, true).values().stream()
                   .mapToInt(Integer::intValue).sum();

// Pattern 2: Sliding window maximum (monotonic approach usually better, but TreeMap works)
TreeMap<Integer, Integer> window = new TreeMap<>();
for (int val : nums) {
    window.merge(val, 1, Integer::sum);
    if (window.size() > k) {
        // remove oldest element
    }
    window.lastKey();  // current maximum
}

// Pattern 3: Difference array problems
TreeMap<Integer, Integer> diff = new TreeMap<>();
// Book a room from start to end:
diff.merge(start, 1, Integer::sum);
diff.merge(end, -1, Integer::sum);
// Check if all slots free: scan prefix sum

// Pattern 4: Coordinate compression
TreeMap<Integer, Integer> compress = new TreeMap<>();
int rank = 0;
for (int val : sortedUniqueValues) compress.put(val, rank++);
```

HashMap vs TreeMap vs LinkedHashMap

	HashMap	TreeMap	LinkedHashMap
Order	None	Sorted by key	Insertion order
Get/ Put/ Remove	O(1) avg	O(log n)	O(1) avg
null keys	1 allowed	Not allowed	1 allowed
Thread safe	No	No	No
Use when	Fast lookup	Sorted order needed, range queries	Maintain insertion order

> **Interview Tip:** “Should I use HashMap or TreeMap?” — if you need sorted keys or range queries, TreeMap. Otherwise, HashMap. This distinction alone wins you points.

Section 2.3 — Set: HashSet and TreeSet

The Set Interface

```
// Set: collection with NO duplicates
Set<Integer> hashSet = new HashSet<>();      // unordered, O(1) ops
Set<Integer> treeSet = new TreeSet<>();      // sorted, O(log n)
Set<Integer> linkedSet = new LinkedHashSet<>(); // insertion order

// Core operations
set.add(10);           // adds if not present
set.remove(10);        // removes if present
set.contains(10);      // O(1) for HashSet, O(log n) for TreeSet
set.size();
set.isEmpty();
set.clear();
```

HashSet

Internal Working

`HashSet = HashMap internally (key → dummy PRESENT object) - All HashSet operations delegate to an internal Has`

Time Complexity

Operation	Complexity
`add(e)`	O(1) avg
`remove(e)`	O(1) avg
`contains(e)`	O(1) avg
`size()`	O(1)
Iteration	O(n)

Complete HashSet API

```java
Set<Integer> set = new HashSet<>();
Set<Integer> set2 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5));
Set<Integer> set3 = new HashSet<>(set2);  // copy constructor

// Add elements
set.add(1);     // true (added)
set.add(1);     // false (already present — no duplicate)
set.addAll(Arrays.asList(2, 3, 4));

// Check
set.contains(1);   // true
set.contains(99);  // false

// Remove
set.remove(1);          // true (removed)
set.remove(99);         // false (not present)
set.removeAll(Arrays.asList(2, 3));  // remove multiple

// Iterate (order is not guaranteed)
for (int val : set) {
    System.out.println(val);
}
set.forEach(System.out::println);

// Set operations
Set<Integer> a = new HashSet<>(Arrays.asList(1, 2, 3, 4));
Set<Integer> b = new HashSet<>(Arrays.asList(3, 4, 5, 6));

// Union
Set<Integer> union = new HashSet<>(a);
union.addAll(b);           // {1, 2, 3, 4, 5, 6}

// Intersection
Set<Integer> intersection = new HashSet<>(a);
intersection.retainAll(b); // {3, 4}

// Difference (A - B)
Set<Integer> diff = new HashSet<>(a);
diff.removeAll(b);         // {1, 2}

// Is subset?
a.containsAll(b);  // false (b has 5, 6 not in a)

// Convert to sorted list
List<Integer> sorted = new ArrayList<>(set);
```

```
        Collections.sort(sorted);

        // Convert to array
        Integer[] arr = set.toArray(new Integer[0]);
```

HashSet DSA Patterns

```java
// Pattern 1: Duplicate detection
public boolean hasDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums) {
        if (!seen.add(n)) return true;  // add returns false if already present
    }
    return false;
}

// Pattern 2: Lookup in O(1) — convert array to set first
Set<Integer> numSet = new HashSet<>();
for (int n : nums) numSet.add(n);
if (numSet.contains(target)) { }

// Pattern 3: Longest consecutive sequence
public int longestConsecutive(int[] nums) {
    Set<Integer> numSet = new HashSet<>();
    for (int n : nums) numSet.add(n);

    int maxLen = 0;
    for (int n : numSet) {
        if (!numSet.contains(n - 1)) {  // start of sequence
            int curr = n, len = 1;
            while (numSet.contains(curr + 1)) { curr++; len++; }
            maxLen = Math.max(maxLen, len);
        }
    }
    return maxLen;
}

// Pattern 4: Two Sum with Set
public boolean twoSum(int[] nums, int target) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums) {
        if (seen.contains(target - n)) return true;
        seen.add(n);
    }
    return false;
}

// Pattern 5: Remove duplicates while preserving order
public int[] removeDuplicates(int[] nums) {
    Set<Integer> seen = new LinkedHashSet<>();
    for (int n : nums) seen.add(n);
    return seen.stream().mapToInt(Integer::intValue).toArray();
```

```
        }

        // Pattern 6: Character set for sliding window
        Set<Character> charSet = new HashSet<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            while (charSet.contains(s.charAt(right))) {
                charSet.remove(s.charAt(left++));
            }
            charSet.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
```

TreeSet

Internal Working

`TreeSet = TreeMap internally (element → dummy PRESENT object) - Elements stored in a Red-Black Tree - Sorted o`

Time Complexity

Operation	Complexity
`add(e)`	O(log n)
`remove(e)`	O(log n)
`contains(e)`	O(log n)
`first()` / `last()`	O(log n)
`floor(e)` / `ceiling(e)`	O(log n)
Iteration	O(n)

```java
TreeSet<Integer> tset = new TreeSet<>(Arrays.asList(5, 1, 3, 7, 9, 2));
// Internally sorted: [1, 2, 3, 5, 7, 9]

// Navigation operations
tset.first();          // 1 (smallest)
tset.last();           // 9 (largest)
tset.floor(4);         // 3 (largest element <= 4)
tset.ceiling(4);       // 5 (smallest element >= 4)
tset.lower(5);         // 3 (strictly less than 5)
tset.higher(5);        // 7 (strictly greater than 5)
tset.pollFirst();      // 1 (removes and returns smallest)
tset.pollLast();       // 9 (removes and returns largest)

// Sub-set operations
tset.subSet(2, true, 7, true);  // [2, 3, 5, 7]
tset.headSet(5);                // [1, 2, 3] (strictly less than 5)
tset.tailSet(5);                // [5, 7, 9] (>= 5)
tset.descendingSet();           // reverse order view

// Custom ordering with Comparator
TreeSet<String> byLength = new TreeSet<>(
    Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())
);
byLength.add("banana");
byLength.add("fig");
byLength.add("apple");
// Iteration: fig, apple, banana (by length, then alphabetical)
```

TreeSet DSA Patterns

```java
// Pattern 1: Kth smallest/largest element dynamically
// (More commonly done with PriorityQueue, but TreeSet works too)
TreeSet<Integer> sorted = new TreeSet<>();
for (int n : stream) {
    sorted.add(n);
    if (sorted.size() > k) sorted.pollLast(); // keep k smallest
}
int kthSmallest = sorted.last();

// Pattern 2: Count of elements in range
TreeSet<Integer> set = new TreeSet<>();
// Elements in [lo, hi]:
NavigableSet<Integer> sub = set.subSet(lo, true, hi, true);
int count = sub.size();

// Pattern 3: Closest value to target
TreeSet<Integer> values = new TreeSet<>();
// ... populate ...
Integer floor = values.floor(target);    // closest ≤ target
Integer ceil  = values.ceiling(target);  // closest ≥ target
// Choose closer:
int closest;
if (floor == null) closest = ceil;
else if (ceil == null) closest = floor;
else closest = (target - floor <= ceil - target) ? floor : ceil;

// Pattern 4: Meeting rooms / room scheduling
TreeSet<Integer> rooms = new TreeSet<>(); // end times of ongoing meetings
for (int[] meeting : sortedByStart) {
    if (!rooms.isEmpty() && rooms.first() <= meeting[0]) {
        rooms.pollFirst(); // reuse a room
    }
    rooms.add(meeting[1]); // assign room, track end time
}
return rooms.size(); // minimum rooms needed
```

LinkedHashSet

```
// Maintains insertion order + no duplicates
Set<Integer> lhs = new LinkedHashSet<>(Arrays.asList(3, 1, 4, 1, 5, 9));
// Iteration order: 3, 1, 4, 5, 9 (insertion order, duplicates removed)

// Use case: unique elements preserving order (like Python's dict keys)
// Use case: LRU cache (with LinkedHashMap)
```

Set Comparison Summary

	HashSet	TreeSet	LinkedHashSet
Order	None	Sorted	Insertion order
Add/ Remove/ Contains	O(1) avg	O(log n)	O(1) avg
null allowed	Yes (one)	No	Yes (one)
Navigation ops	No	Yes	No
Use when	Fast lookup, dedup	Sorted set, range ops	Ordered dedup

---

Section 2.4 — Queue, Stack, Deque, and PriorityQueue

Stack

Using ArrayDeque as Stack (Recommended)

```
// Java's legacy Stack class is synchronized (slow) — use ArrayDeque instead
Deque<Integer> stack = new ArrayDeque<>();

// Push
stack.push(1);     // equivalent to addFirst() — adds to HEAD
stack.push(2);
stack.push(3);

// Pop
int top = stack.pop();  // removes and returns head — 3

// Peek
int peek = stack.peek();  // returns head without removing — 2

// Check empty
stack.isEmpty();

// Size
stack.size();
```

Stack DSA Patterns

```java
// Pattern 1: Balanced Brackets
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }
    return stack.isEmpty();
}

// Pattern 2: Evaluate expression / convert infix to postfix
// (uses two stacks or one stack)
public int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String token : tokens) {
        if (token.equals("+") || token.equals("-") ||
            token.equals("*") || token.equals("/")) {
            int b = stack.pop(), a = stack.pop();
            switch (token) {
                case "+": stack.push(a + b); break;
                case "-": stack.push(a - b); break;
                case "*": stack.push(a * b); break;
                case "/": stack.push(a / b); break;
            }
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}

// Pattern 3: Min stack — supporting getMin in O(1)
class MinStack {
    private Deque<Integer> stack = new ArrayDeque<>();
    private Deque<Integer> minStack = new ArrayDeque<>();

    public void push(int val) {
        stack.push(val);
```

```
            int currMin = minStack.isEmpty() ? val : Math.min(val, minStack.peek());
            minStack.push(currMin);
        }

        public void pop() {
            stack.pop();
            minStack.pop();
        }

        public int top() { return stack.peek(); }
        public int getMin() { return minStack.peek(); }
    }
```

Queue

Using ArrayDeque as Queue (Recommended)

```
    // FIFO: add to tail, remove from head
    Queue<Integer> queue = new ArrayDeque<>();

    // Enqueue (add to tail)
    queue.offer(1);   // preferred (returns false on failure)
    queue.add(2);     // throws exception on failure
    queue.offer(3);

    // Dequeue (remove from head)
    int front = queue.poll();    // removes and returns head (null if empty)
    int front2 = queue.remove(); // removes and returns head (throws if empty)

    // Peek head
    int peek = queue.peek();   // null if empty
    int peek2 = queue.element(); // throws if empty

    // Check
    queue.isEmpty();
    queue.size();
```

BFS Queue Pattern (Most Important)

```java
// BFS template — used for trees, graphs, shortest path
public int bfs(int[][] grid, int startR, int startC) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new ArrayDeque<>();
    boolean[][] visited = new boolean[rows][cols];

    queue.offer(new int[]{startR, startC});
    visited[startR][startC] = true;
    int distance = 0;

    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};

    while (!queue.isEmpty()) {
        int size = queue.size();  // process level by level
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            int r = curr[0], c = curr[1];

            if (isTarget(grid, r, c)) return distance;

            for (int[] dir : dirs) {
                int nr = r + dir[0], nc = c + dir[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                    && !visited[nr][nc] && grid[nr][nc] != 0) {
                    queue.offer(new int[]{nr, nc});
                    visited[nr][nc] = true;
                }
            }
        }
        distance++;
    }
    return -1;  // not found
}
```

Deque (Double-Ended Queue)

ArrayDeque (Best All-Around Implementation)

```
Deque<Integer> deque = new ArrayDeque<>();

// Add to head / tail
deque.addFirst(1);    // [1]
deque.addLast(2);     // [1, 2]
deque.offerFirst(0);  // [0, 1, 2]
deque.offerLast(3);   // [0, 1, 2, 3]

// Remove from head / tail
deque.removeFirst();  // 0 — [1, 2, 3]
deque.removeLast();   // 3 — [1, 2]
deque.pollFirst();    // 1 — [2]   (null if empty)
deque.pollLast();     // 2 — []    (null if empty)

// Peek head / tail
deque.peekFirst();    // null if empty
deque.peekLast();     // null if empty
deque.getFirst();     // throws if empty
deque.getLast();      // throws if empty

// Stack usage (LIFO)
deque.push(val);   // = addFirst
deque.pop();       // = removeFirst
deque.peek();      // = peekFirst

// Queue usage (FIFO)
deque.offer(val);  // = addLast
deque.poll();      // = removeFirst
deque.peek();      // = peekFirst
```

Monotonic Deque (Sliding Window Maximum)

```java
// Sliding window maximum — O(n)
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();  // stores INDICES

    for (int i = 0; i < n; i++) {
        // Remove elements out of window
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }

        // Remove elements smaller than current (maintain decreasing order)
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }

        deque.offerLast(i);

        // Record result once window is full
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
    return result;
}
```

PriorityQueue (Heap)

Internal Working

`PriorityQueue = Binary Heap - Min-heap by default (smallest element at top) - Backed by array - Parent-child r`

Time Complexity

Operation	Complexity
`offer(e)` / `add(e)`	O(log n)
`poll()`	O(log n)
`peek()`	O(1)
`contains(e)`	O(n)
`remove(e)`	O(n)
Build heap from n elements	O(n)

Complete PriorityQueue API

```
// Min-heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// Or:
PriorityQueue<Integer> maxHeap2 = new PriorityQueue<>((a, b) -> b - a);

// Custom comparator (e.g., sort by frequency)
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);  // by second element

// Operations
minHeap.offer(5);    // add
minHeap.offer(1);
minHeap.offer(3);
minHeap.peek();      // 1 (min, not removed)
minHeap.poll();      // 1 (removes and returns min)
minHeap.size();
minHeap.isEmpty();

// Build from collection — O(n) (more efficient than n insertions)
PriorityQueue<Integer> pq2 = new PriorityQueue<>(Arrays.asList(5, 3, 1, 4, 2));

// Convert to sorted array
List<Integer> sorted = new ArrayList<>();
while (!minHeap.isEmpty()) sorted.add(minHeap.poll());  // O(n log n)
```

PriorityQueue DSA Patterns

```java
// Pattern 1: Kth Largest Element — O(n log k)
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int n : nums) {
        minHeap.offer(n);
        if (minHeap.size() > k) minHeap.poll();  // keep only k largest
    }
    return minHeap.peek();  // kth largest = smallest of top-k
}

// Pattern 2: Top K Frequent Elements
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        minHeap.offer(new int[]{e.getKey(), e.getValue()});
        if (minHeap.size() > k) minHeap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = minHeap.poll()[0];
    return result;
}

// Pattern 3: Merge K Sorted Lists
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode head : lists) {
        if (head != null) heap.offer(head);
    }

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        curr.next = node;
        curr = curr.next;
        if (node.next != null) heap.offer(node.next);
    }
    return dummy.next;
}

// Pattern 4: Find Median from Data Stream
```

```java
class MedianFinder {
    private PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // lower ha
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();  // upper half

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());
        if (maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() > minHeap.size()) return maxHeap.peek();
        return (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}

// Pattern 5: Dijkstra's Shortest Path
public int[] dijkstra(int n, int[][] edges, int src) {
    Map<Integer, List<int[]>> adj = new HashMap<>();
    for (int[] e : edges) {
        adj.computeIfAbsent(e[0], k -> new ArrayList<>()).add(new int[]{e[1], e[2]});
        adj.computeIfAbsent(e[1], k -> new ArrayList<>()).add(new int[]{e[0], e[2]});
    }

    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // [node, distance]
    pq.offer(new int[]{src, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0], d = curr[1];
        if (d > dist[node]) continue;  // outdated entry

        for (int[] neighbor : adj.getOrDefault(node, new ArrayList<>())) {
            int next = neighbor[0], weight = neighbor[1];
            if (dist[node] + weight < dist[next]) {
                dist[next] = dist[node] + weight;
                pq.offer(new int[]{next, dist[next]});
            }
        }
```

```
        }
        return dist;
    }

    // Pattern 6: Task scheduling by frequency
    public String reorganizeString(String s) {
        int[] freq = new int[26];
        for (char c : s.toCharArray()) freq[c - 'a']++;

        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[1] - a[1]);
        for (int i = 0; i < 26; i++) {
            if (freq[i] > 0) maxHeap.offer(new int[]{i + 'a', freq[i]});
        }

        StringBuilder sb = new StringBuilder();
        while (maxHeap.size() >= 2) {
            int[] first = maxHeap.poll();
            int[] second = maxHeap.poll();
            sb.append((char) first[0]);
            sb.append((char) second[0]);
            if (--first[1] > 0) maxHeap.offer(first);
            if (--second[1] > 0) maxHeap.offer(second);
        }

        if (!maxHeap.isEmpty()) {
            int[] last = maxHeap.poll();
            if (last[1] > 1) return "";  // impossible
            sb.append((char) last[0]);
        }
        return sb.toString();
    }
```

Queue/Stack/Deque Summary

	Stack	Queue	Deque
Order	LIFO	FIFO	Both ends
Add	`push()`	`offer()`	`addFirst/Last()`
Remove	`pop()`	`poll()`	`pollFirst/Last()`
Peek	`peek()`	`peek()`	`peekFirst/Last()`
Best Implementation	`ArrayDeque`	`ArrayDeque`	`ArrayDeque`
Use case	DFS, brackets, undo	BFS, level order, scheduling	Sliding window

> **Interview Tip:** Always say “I’ll use ArrayDeque as my stack/queue — it’s more efficient than Java’s Stack and LinkedList.” This demonstrates production-quality Java knowledge.

Section 2.5 — Comparator, Comparable, and Collections Utilities

---

1. Comparable Interface

```java
// Comparable: "I know how to compare myself to another of my type"
// Used for natural ordering — the default sort order

class Student implements Comparable<Student> {
    String name;
    int gpa;
    int age;

    Student(String name, int gpa, int age) {
        this.name = name;
        this.gpa = gpa;
        this.age = age;
    }

    @Override
    public int compareTo(Student other) {
        // Return negative: this < other
        // Return 0: this == other
        // Return positive: this > other

        // Sort by GPA descending, then name ascending
        if (this.gpa != other.gpa) return other.gpa - this.gpa;
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return name + "(" + gpa + ")";
    }
}

// Usage
List<Student> students = new ArrayList<>();
students.add(new Student("Alice", 90, 20));
students.add(new Student("Bob", 95, 22));
students.add(new Student("Charlie", 90, 21));

Collections.sort(students);  // uses compareTo
// Result: [Bob(95), Alice(90), Charlie(90)] — by GPA desc, then name asc

// compareTo contract:
// 1. sgn(x.compareTo(y)) == -sgn(y.compareTo(x))
// 2. Transitive: if x.compareTo(y) > 0 and y.compareTo(z) > 0 → x.compareTo(z) > 0
// 3. Consistency: x.compareTo(y) == 0 → x.equals(y) (strongly recommended)
```

2. Comparator Interface

```
// Comparator: external comparison logic (doesn't touch the class)
// More flexible than Comparable — can have multiple comparators

class Student {
    String name;
    int gpa;
    int age;
    // ... constructor, getters ...
}

// Old style (anonymous inner class)
Comparator<Student> byGpa = new Comparator<Student>() {
    @Override
    public int compare(Student a, Student b) {
        return b.gpa - a.gpa;  // descending
    }
};

// Lambda style (Java 8+)
Comparator<Student> byGpa = (a, b) -> b.gpa - a.gpa;
Comparator<Student> byName = (a, b) -> a.name.compareTo(b.name);
Comparator<Student> byAge = Comparator.comparingInt(s -> s.age);

// Chained comparators
Comparator<Student> combined = Comparator
    .comparingInt((Student s) -> s.gpa).reversed()  // desc GPA
    .thenComparing(s -> s.name)                     // asc name
    .thenComparingInt(s -> s.age);                  // asc age

// Usage
students.sort(byGpa);
Arrays.sort(arr, byGpa);
PriorityQueue<Student> pq = new PriorityQueue<>(byGpa);
TreeSet<Student> tset = new TreeSet<>(byGpa);
```

Comparator Factory Methods (Java 8+)

```
// Comparator.comparing — for any key
Comparator<String> byLength = Comparator.comparingInt(String::length);
Comparator<String> alphabetical = Comparator.comparing(Function.identity());
Comparator<String> natural = Comparator.naturalOrder();
Comparator<String> reverse = Comparator.reverseOrder();
Comparator<String> nullFirst = Comparator.nullsFirst(Comparator.naturalOrder());
Comparator<String> nullLast = Comparator.nullsLast(Comparator.naturalOrder());

// Chaining with thenComparing
Comparator<int[]> byFirst = Comparator.comparingInt((int[] a) -> a[0]);
Comparator<int[]> byFirstThenSecond = byFirst.thenComparingInt(a -> a[1]);

// Reversing
Comparator<Integer> descInt = Comparator.reverseOrder();
Comparator<Student> descGpa = Comparator.comparingInt(Student::getGpa).reversed();
```

Critical Comparator Patterns for Interviews

```
// 1. Sort intervals by start time
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

// 2. Sort by length, then lexicographically
Arrays.sort(words, (a, b) -> a.length() != b.length() ?
            a.length() - b.length() : a.compareTo(b));

// 3. Custom sort for frequency problems
// Sort by frequency desc, then value asc
int[] result = freq.entrySet().stream()
    .sorted(Map.Entry.<Integer, Integer>comparingByValue().reversed()
        .thenComparingByKey())
    .limit(k)
    .mapToInt(Map.Entry::getKey)
    .toArray();

// 4. Negative sort trick — beware of integer overflow!
// BAD (can overflow):
Comparator<Integer> bad = (a, b) -> b - a;

// SAFE:
Comparator<Integer> safe = (a, b) -> Integer.compare(b, a);
Comparator<Integer> safe2 = Collections.reverseOrder();

// When is (b - a) SAFE? Only when a,b are small ints that can't overflow.
// General rule: always use Integer.compare() for safety.

// 5. Sort 2D array
int[][] matrix = {{3,2}, {1,4}, {3,0}, {1,2}};
Arrays.sort(matrix, (a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);
```

---

3. Collections Utility Class

```java
import java.util.Collections;

List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6));

// Sorting
Collections.sort(list);                          // ascending — [1,1,2,3,4,5,6,9]
Collections.sort(list, Collections.reverseOrder()); // descending — [9,6,5,4,3,2,1,1]

// Searching (binary search — list must be sorted)
int idx = Collections.binarySearch(list, 5);    // index of 5 (or negative)
// If not found: returns -(insertion point) - 1

// Min/Max
Collections.min(list);   // 1
Collections.max(list);   // 9
Collections.min(list, Comparator.reverseOrder());  // with comparator

// Frequency
Collections.frequency(list, 1);  // 2 (count of 1s)

// Reverse
Collections.reverse(list);       // reverse in-place

// Shuffle
Collections.shuffle(list);       // random order
Collections.shuffle(list, new Random(42));  // with seed

// Fill
Collections.fill(list, 0);       // fill all with 0

// Copy
List<Integer> dest = new ArrayList<>(Collections.nCopies(list.size(), 0));
Collections.copy(dest, list);    // copy list into dest (dest must be same size)

// Swap
Collections.swap(list, 0, list.size() - 1);  // swap first and last

// Rotate
Collections.rotate(list, 2);    // rotate right by 2

// Disjoint
Collections.disjoint(list1, list2);  // true if no common elements

// Unmodifiable wrappers
List<Integer> unmod = Collections.unmodifiableList(list);
```

```
Set<Integer> unmodSet = Collections.unmodifiableSet(set);
Map<K, V> unmodMap = Collections.unmodifiableMap(map);

// Synchronized wrappers (thread-safe, but usually use ConcurrentHashMap instead)
List<Integer> syncList = Collections.synchronizedList(list);

// Empty and singleton
List<Integer> empty = Collections.emptyList();   // immutable empty list
List<Integer> single = Collections.singletonList(42); // immutable single-element

// nCopies
List<Integer> zeros = Collections.nCopies(5, 0);  // [0, 0, 0, 0, 0]
```

---

4. Arrays Utility

```
import java.util.Arrays;

// Covered in Section 1 but key points:
int[] arr = {5, 2, 8, 1, 9};

Arrays.sort(arr);                    // [1, 2, 5, 8, 9] — in-place
Arrays.sort(arr, 1, 4);              // sort subarray [1,4)
Arrays.binarySearch(arr, 5);         // works only on sorted array
Arrays.fill(arr, 0);
Arrays.copyOf(arr, 3);               // [1, 2, 5]
Arrays.copyOfRange(arr, 1, 4);       // [2, 5, 8]
Arrays.equals(arr1, arr2);
Arrays.toString(arr);                // "[1, 2, 5, 8, 9]"
Arrays.deepToString(matrix);         // for 2D arrays

// Sort object array with comparator
Integer[] arr2 = {5, 2, 8, 1, 9};
Arrays.sort(arr2, Comparator.reverseOrder());

// Parallel sort (for large arrays)
Arrays.parallelSort(arr);            // uses fork/join framework

// stream
Arrays.stream(arr).sum();
Arrays.stream(arr).max().getAsInt();
Arrays.stream(arr).filter(n -> n > 3).toArray();
```

---

5. Interview-Ready Quick Reference

```java
// Most common operations cheat sheet

// Frequency map
Map<T, Integer> freq = new HashMap<>();
for (T item : collection) freq.merge(item, 1, Integer::sum);

// Sort with custom comparator
list.sort(Comparator.comparingInt(obj -> obj.field));

// Find max/min in list
int max = Collections.max(list);
int min = list.stream().mapToInt(Integer::intValue).min().getAsInt();

// Convert between types
int[] arr = list.stream().mapToInt(Integer::intValue).toArray();
List<Integer> list = new ArrayList<>(Arrays.asList(arr)); // won't work for int[]
List<Integer> list = IntStream.of(arr).boxed().collect(Collectors.toList());
Integer[] intArr = list.toArray(new Integer[0]);

// Reverse array
int[] arr = {1, 2, 3, 4, 5};
for (int i = 0, j = arr.length - 1; i < j; i++, j--) {
    int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
}

// Swap in array
int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;

// Fill 2D array
for (int[] row : dp) Arrays.fill(row, -1);
// Or:
int[][] dp = new int[m][n];
for (int[] row : dp) Arrays.fill(row, Integer.MAX_VALUE);
```

Cheat Sheet: Which Collection to Use?

Requirement	Use
Fast lookup by key	`HashMap`
Sorted keys, range ops	`TreeMap`
Preserve insertion order	`LinkedHashMap`
Fast membership check	`HashSet`
Sorted unique elements	`TreeSet`
Dynamic array, random access	`ArrayList`
Frequent head/tail ops	`ArrayDeque`
Min/Max element	`PriorityQueue`
LIFO (stack)	`ArrayDeque` (as stack)
FIFO (queue)	`ArrayDeque` (as queue)
Both ends	`ArrayDeque` (as deque)
Sliding window max/min	`ArrayDeque` (monotonic)
Top-K elements	`PriorityQueue` (size K)
Shortest path	`PriorityQueue` (Dijkstra)
Median stream	Two `PriorityQueue`s

Section 3.1 — Big O Analysis

---

1. What is Big O?

Big O notation describes the **upper bound** of an algorithm's resource usage (time or space) as input size grows. It's about the **order of magnitude**, not exact values.

`O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)`

For n = 1,000: - O(1) → 1 operation - O(log n) → ~10 operations - O(n) → 1,000 operations - O(n log n) → ~10,000 operations - O(n²) → 1,000,000 operations - O(2ⁿ) → 10^301 operations (impossible)

---

2. Common Complexities Explained

O(1) — Constant Time

```
// Time does not depend on input size
int[] arr = {1, 2, 3, 4, 5};
int first = arr[0];              // O(1)
int last = arr[arr.length - 1];  // O(1)
map.get(key);                    // O(1) average
set.contains(val);               // O(1) average
stack.push(val);                 // O(1)
```

O(log n) — Logarithmic

```
// Input is halved each step
// Classic: binary search
int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {      // ← repeats log₂(n) times
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
// n=1000: ~10 iterations. n=1000000: ~20 iterations
```

O(n) — Linear

```
// Visits each element once
int sum = 0;
for (int n : arr) sum += n;       // O(n) — one pass

// Single pass with HashMap — still O(n)
Map<Integer, Integer> freq = new HashMap<>();
for (int n : arr) freq.merge(n, 1, Integer::sum);  // O(n)
```

O(n log n) — Linearithmic

```
// Comparison-based sorting algorithms
Arrays.sort(arr);           // O(n log n) — merge sort / tim sort
Collections.sort(list);     // O(n log n)
// Building a heap: O(n), then n extractions: O(n log n)
// n insertions into TreeMap/TreeSet: O(n log n)
```

O(n²) — Quadratic

```
// Nested loops — brute force
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // O(n²) total iterations
    }
}
// Bubble sort, insertion sort, selection sort
// Two-sum naive, checking all pairs
```

O(2ⁿ) — Exponential

```
// Recursion that branches into 2 each call
// Classic: naive Fibonacci, all subsets
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);  // 2 calls each time → O(2ⁿ)
}
// All subsets of n elements: 2ⁿ subsets
// Can usually be optimized with memoization/DP
```

O(n!) — Factorial

```
// All permutations of n elements
// Backtracking problems: generate all permutations
// Traveling salesman (brute force)
// n=10: 3,628,800 — barely feasible
// n=15: 1.3 trillion — impossible
```

---

3. Analyzing Your Code

Rules

`Rule 1: Drop constants O(2n) → O(n) O(100) → O(1) O(n/2) → O(n) Rule 2: Drop lower-order terms O(n² + n) → O(n`

Practice: Identify Complexity

```java
// Example 1
void example1(int n) {
    for (int i = 0; i < n; i++) {          // O(n)
        for (int j = 0; j < n; j++) {      // O(n)
            System.out.print(i + j);        // O(1)
        }
    }
}  // Total: O(n²)

// Example 2
void example2(int n) {
    for (int i = n; i > 0; i /= 2) {      // halves each time → O(log n)
        System.out.println(i);
    }
}  // Total: O(log n)

// Example 3
void example3(int[] arr) {
    for (int i = 0; i < arr.length; i++) {  // O(n)
        for (int j = i; j < arr.length; j++) { // O(n) inner
            // sum: (n) + (n-1) + ... + 1 = n(n+1)/2
        }
    }
}  // Total: O(n²) — even though inner loop shrinks

// Example 4 — with function call
void example4(int[] arr) {
    for (int val : arr) {        // O(n)
        Arrays.sort(arr);        // O(n log n) INSIDE the loop!
    }
}  // Total: O(n² log n) — careful with function calls in loops!

// Example 5 — recursive with memo
int[] memo = new int[n + 1];
int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}  // With memoization: O(n) time, O(n) space
```

4. Space Complexity

`O(1) — Fixed amount of variables (no arrays, no recursion) O(n) — Array, list, map of size n; recursion depth`

```java
// O(1) space
int sum = 0;
for (int n : arr) sum += n;  // only one variable

// O(n) space
int[] prefix = new int[n];   // array of size n
Map<Integer, Integer> map = new HashMap<>();  // up to n entries

// O(log n) space — recursion stack
int binarySearch(int[] arr, int l, int r, int target) {
    if (l > r) return -1;
    int mid = (l + r) / 2;
    // recursive call depth: O(log n)
    return arr[mid] == target ? mid :
           arr[mid] < target ? binarySearch(arr, mid+1, r, target) :
                               binarySearch(arr, l, mid-1, target);
}

// O(n) space — recursion
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);  // n stack frames
}
```

---

5. Complexity of Java Built-in Operations

Operation	Time	Space
`Arrays.sort(int[])`	O(n log n)	O(log n)
`Arrays.sort(Integer[])` with comparator	O(n log n)	O(log n)
`String.substring(l, r)`	O(r-l)	O(r-l)
`String.contains(s)`	O(n*m)	O(1)
`StringBuilder.append()`	O(1) amortized	—
`HashMap.get/put`	O(1) avg	—
`TreeMap.get/put`	O(log n)	—
`PriorityQueue.offer/poll`	O(log n)	—
`Collections.sort(List)`	O(n log n)	O(log n)
`String.split(regex)`	O(n)	O(n)
`HashSet.contains`	O(1) avg	—

---

6. Interview Communication Template

When explaining complexity in an interview:

> “Let me analyze the time and space complexity.
> **Time:** The outer loop runs n times, and for each iteration, the inner operation takes O(log n) due to the binary search. So overall, time complexity is **O(n log n)**.
> **Space:** I’m using a HashMap that stores at most n entries, plus constant extra variables. So space complexity is **O(n)**.
> Can I do better? I think there might be an O(n) solution using [two pointers / prefix sum / sliding window]...”

---

7. Target Complexity Guide by Problem Size

n (input size)	Max Acceptable Complexity
n ≤ 10	O(n!) — backtracking OK
n ≤ 20	O(2^n) — bitmask DP
n ≤ 100	O(n^3) — 3 nested loops
n ≤ 1,000	O(n^2) — 2 nested loops
n ≤ 100,000	O(n log n) — sort, heap, BST
n ≤ 1,000,000	O(n) — single/dual pass
n ≤ 10^9	O(log n) or O(1)

> **Interview Tip:** If the constraint says n ≤ 10^5, and your solution is O(n^2), that's 10^10 operations — will TLE. Always check constraints to determine target complexity.

Section 3.2 — Recursion and Memoization

---

1. Recursion Fundamentals

`Every recursive function needs: 1. BASE CASE – when to stop 2. RECURSIVE CASE – break problem into smaller sub`

```
// Template
returnType solve(params) {
    // 1. Base case
    if (baseCondition) return baseValue;

    // 2. Recursive case (decompose)
    subResult = solve(smallerProblem);

    // 3. Combine
    return combine(subResult, currentElement);
}
```

---

2. Classic Recursion Examples

Factorial

```
// O(n) time, O(n) space (stack frames)
int factorial(int n) {
    if (n <= 1) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}

// Iteration equivalent (O(1) space)
int factorialIter(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}
```

Power Function

```
// Naive: O(n)
double power(double base, int exp) {
    if (exp == 0) return 1;
    return base * power(base, exp - 1);
}

// Fast power (Binary Exponentiation): O(log n)
double fastPower(double base, int exp) {
    if (exp == 0) return 1;
    if (exp < 0) return 1.0 / fastPower(base, -exp);

    if (exp % 2 == 0) {
        double half = fastPower(base, exp / 2);
        return half * half;  // IMPORTANT: compute once, use twice
    } else {
        return base * fastPower(base, exp - 1);
    }
}
// Used in: LeetCode 50 (Pow(x,n)), modular exponentiation
```

Binary Search (Recursive)

```
// O(log n) time, O(log n) space
int binarySearch(int[] arr, int target, int left, int right) {
    if (left > right) return -1;  // base case: not found

    int mid = left + (right - left) / 2;

    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearch(arr, target, mid + 1, right);
    return binarySearch(arr, target, left, mid - 1);
}
```

Merge Sort

```
// O(n log n) time, O(n) space
void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;  // base case: single element

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);        // sort left half
    mergeSort(arr, mid + 1, right);   // sort right half
    merge(arr, left, mid, right);     // merge sorted halves
}

void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int l = 0; l < temp.length; l++) arr[left + l] = temp[l];
}
```

---

3. Recursion Tree Thinking

`Visualize the call tree to analyze: 1. How many levels deep? → O(depth) for space 2. How many calls per level?`

4. Memoization (Top-Down DP)

```
// Pattern: cache results of sub-problems in a map/array

// Fibonacci — naive O(2^n)
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// Fibonacci — memoized O(n)
int[] memo = new int[n + 1];
Arrays.fill(memo, -1);

int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];       // cache hit
    return memo[n] = fib(n-1) + fib(n-2);   // compute & cache
}

// Using HashMap for non-integer keys
Map<String, Integer> memo = new HashMap<>();
int solve(String state) {
    if (memo.containsKey(state)) return memo.get(state);
    // ... compute result ...
    memo.put(state, result);
    return result;
}
```

Memoization Template

```
// General template for memoization
class Solution {
    private int[] memo;

    public int solve(int n) {
        memo = new int[n + 1];
        Arrays.fill(memo, -1);
        return dp(n);
    }

    private int dp(int n) {
        // Base cases
        if (n == 0) return 0;
        if (n == 1) return 1;

        // Check cache
        if (memo[n] != -1) return memo[n];

        // Compute and cache
        return memo[n] = dp(n - 1) + dp(n - 2);
    }
}
```

---

5. Classic Memoization Problems

Climbing Stairs

```
// How many ways to climb n stairs (1 or 2 at a time)?
// dp[n] = dp[n-1] + dp[n-2]

int climbStairs(int n) {
    if (n <= 2) return n;
    int[] dp = new int[n + 1];
    dp[1] = 1; dp[2] = 2;
    for (int i = 3; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Coin Change

```java
// Minimum coins to make amount (can reuse coins)
// dp[amount] = min(dp[amount - coin] + 1) for each coin

int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);  // init to "infinity"
    dp[0] = 0;

    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

House Robber

```java
// Can't rob adjacent houses
// dp[i] = max(dp[i-1], dp[i-2] + nums[i])

int rob(int[] nums) {
    int n = nums.length;
    if (n == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < n; i++) {
        int curr = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

Longest Common Subsequence

```java
// dp[i][j] = LCS of s1[0..i-1] and s2[0..j-1]
int lcs(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[m][n];
}
```

---

6. Key Recursion Patterns for DSA

Tree Recursion (Post-order)

```
// Pattern: compute something at each node using children's results
int height(TreeNode root) {
    if (root == null) return 0;                    // base case
    int leftH = height(root.left);                 // recurse left
    int rightH = height(root.right);               // recurse right
    return 1 + Math.max(leftH, rightH);            // combine
}

int diameter(TreeNode root) {
    int[] maxDiam = {0};  // use array to pass by reference

    int dfs(TreeNode node) {
        if (node == null) return 0;
        int left = dfs(node.left);
        int right = dfs(node.right);
        maxDiam[0] = Math.max(maxDiam[0], left + right);
        return 1 + Math.max(left, right);
    }

    dfs(root);
    return maxDiam[0];
}
```

Backtracking Template

```
void backtrack(result, current, choices) {
    if (isDone) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (choice : choices) {
        if (isValid(choice)) {
            current.add(choice);          // make choice
            backtrack(result, current, remainingChoices);
            current.remove(current.size() - 1);  // undo choice
        }
    }
}
```

Divide and Conquer Template

```
T solve(int[] arr, int left, int right) {
    // Base case
    if (left == right) return base;

    int mid = left + (right - left) / 2;

    // Divide
    T leftResult = solve(arr, left, mid);
    T rightResult = solve(arr, mid + 1, right);

    // Conquer (combine)
    return merge(leftResult, rightResult);
}
```

---

7. Tail Recursion (Java Doesn’t Optimize It)

```java
// Java does NOT optimize tail calls (unlike Haskell, Scala)
// Deep recursion will cause StackOverflowError for n > ~10,000

// Stack size limit demonstration
int deepRecurse(int n) {
    if (n == 0) return 0;
    return 1 + deepRecurse(n - 1);  // StackOverflow for n > ~8000
}

// Solution: convert to iteration or use explicit stack
int iterative(int n) {
    Deque<Integer> stack = new ArrayDeque<>();
    // ... simulate recursion with explicit stack
}

// Or increase stack size (not always possible in interviews):
// java -Xss64m Solution  (64MB stack)
```

---

8. Recursion vs Iteration Decision Guide

Scenario	Prefer
Tree traversal	Recursion (cleaner code)
Graph DFS	Recursion (or iterative with explicit stack)
Simple loop	Iteration
n > 10,000 and deep recursion	Iteration (avoid StackOverflow)
Memoization with many states	Top-down recursion + memo
Multiple state transitions	Bottom-up DP (iteration)
Backtracking	Recursion (natural)
Binary search	Iteration (O(1) space)

> **Interview Tip:** For tree problems, recursive solutions are usually cleaner and preferred. For long chains (n=10^6), mention you’d use iteration to avoid stack overflow — this shows production awareness.

Section 3.3 — Sorting and Searching

1. Sorting Algorithms

Quick Reference

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes
Radix Sort	O(d*n)	O(d*n)	O(d*n)	O(n+k)	Yes
Tim Sort (Java)	O(n)	O(n log n)	O(n log n)	O(n)	Yes

Java uses: Dual-Pivot Quicksort for primitive arrays, TimSort for Object arrays

Implementations You Must Know

Merge Sort (Divide and Conquer)

```java
// Stable, O(n log n) guaranteed, O(n) space
void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

void merge(int[] arr, int left, int mid, int right) {
    int[] temp = Arrays.copyOfRange(arr, left, right + 1);
    int i = 0, j = mid - left + 1, k = left;

    while (i <= mid - left && j <= right - left) {
        if (temp[i] <= temp[j]) arr[k++] = temp[i++];
        else arr[k++] = temp[j++];
    }
    while (i <= mid - left) arr[k++] = temp[i++];
    while (j <= right - left) arr[k++] = temp[j++];
}

// Use case: count inversions (modify merge step)
long countInversions;
void mergeCount(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeCount(arr, left, mid);
    mergeCount(arr, mid + 1, right);
    mergeAndCount(arr, left, mid, right);
}
```

Quick Sort

```java
// Average O(n log n), O(n²) worst (mitigated by random pivot)
void quickSort(int[] arr, int left, int right) {
    if (left >= right) return;

    int pivotIdx = partition(arr, left, right);
    quickSort(arr, left, pivotIdx - 1);
    quickSort(arr, pivotIdx + 1, right);
}

int partition(int[] arr, int left, int right) {
    // Randomize pivot to avoid O(n²) worst case
    int randIdx = left + (int)(Math.random() * (right - left + 1));
    swap(arr, randIdx, right);

    int pivot = arr[right];
    int i = left - 1;

    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr, i, j);
        }
    }
    swap(arr, i + 1, right);
    return i + 1;
}

void swap(int[] arr, int i, int j) {
    int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
}
```

Counting Sort (When range is known)

```java
// O(n + k) where k = range of values
void countingSort(int[] arr, int maxVal) {
    int[] count = new int[maxVal + 1];
    for (int n : arr) count[n]++;

    int idx = 0;
    for (int i = 0; i <= maxVal; i++) {
        while (count[i]-- > 0) arr[idx++] = i;
    }
}

// Character frequency sort
void sortChars(char[] arr) {
    int[] freq = new int[26];
    for (char c : arr) freq[c - 'a']++;
    int idx = 0;
    for (int i = 0; i < 26; i++) {
        while (freq[i]-- > 0) arr[idx++] = (char)('a' + i);
    }
}
```

Insertion Sort (Best for small/nearly-sorted arrays)

```java
void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

---

2. Binary Search — Core Variations

> “Binary search is easy to understand, but hard to implement correctly.” — Donald Knuth

Template 1: Find Exact Target

```java
// Returns index of target, or -1 if not found
int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Template 2: Find Leftmost (First Occurrence)

```java
// Returns index of first occurrence of target, or -1
int findFirst(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            right = mid - 1;  // keep searching LEFT
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
// Or using the "lower bound" approach:
int lowerBound(int[] arr, int target) {
    int left = 0, right = arr.length;
    while (left < right) {           // note: left < right (not <=)
        int mid = left + (right - left) / 2;
        if (arr[mid] < target) left = mid + 1;
        else right = mid;            // include mid in search space
    }
    return left;  // index of first element >= target
}
```

Template 3: Find Rightmost (Last Occurrence)

```
int findLast(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            left = mid + 1;   // keep searching RIGHT
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
```

Template 4: Rotated Sorted Array

```java
// {4,5,6,7,0,1,2} — find target
int searchRotated(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;

        // Left half is sorted
        if (arr[left] <= arr[mid]) {
            if (arr[left] <= target && target < arr[mid]) {
                right = mid - 1;  // target in sorted left half
            } else {
                left = mid + 1;   // target in right half
            }
        }
        // Right half is sorted
        else {
            if (arr[mid] < target && target <= arr[right]) {
                left = mid + 1;   // target in sorted right half
            } else {
                right = mid - 1;  // target in left half
            }
        }
    }
    return -1;
}
```

Template 5: Find Peak Element

```java
// Peak: arr[i] > arr[i-1] and arr[i] > arr[i+1]
int findPeak(int[] arr) {
    int left = 0, right = arr.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] > arr[mid + 1]) {
            right = mid;        // peak is at mid or left of mid
        } else {
            left = mid + 1;     // peak is right of mid
        }
    }
    return left;  // left == right == peak index
}
```

Template 6: Binary Search on Answer Space

```java
// "Minimize the maximum" or "find the minimum that satisfies condition"
// Key insight: search on the answer itself, not the array

// Example: Koko Eating Bananas
// Can Koko eat all bananas at speed k in h hours?
boolean canFinish(int[] piles, int h, int k) {
    long hours = 0;
    for (int pile : piles) {
        hours += (pile + k - 1) / k;  // ceil division
    }
    return hours <= h;
}

int minEatingSpeed(int[] piles, int h) {
    int left = 1, right = Arrays.stream(piles).max().getAsInt();

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (canFinish(piles, h, mid)) {
            right = mid;     // try smaller speed
        } else {
            left = mid + 1;  // need larger speed
        }
    }
    return left;
}

// Pattern for "minimize X such that condition(X) is satisfied":
// left = minimum possible answer
// right = maximum possible answer
// Condition must be monotonic: if condition(X) is true, condition(X+1) is also true
```

---

3. Search Patterns Summary

```
// Binary search decision tree:
// 1. Is array sorted? → Direct binary search
// 2. Is array sorted and rotated? → Template 4
// 3. Find peak? → Template 5
// 4. Find first/last occurrence? → Templates 2/3
// 5. Minimize/maximize answer? → Binary search on answer space

// Signal keywords for binary search on answer:
// "minimum speed", "maximum pages", "minimize distance",
// "smallest possible", "largest valid k"
```

---

4. Common Sorting Interview Problems

```java
// 1. Sort Colors (Dutch National Flag)
void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) swap(nums, low++, mid++);
        else if (nums[mid] == 1) mid++;
        else swap(nums, mid, high--);
    }
}

// 2. Kth Largest Element (Quick Select — O(n) average)
int findKthLargest(int[] nums, int k) {
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}

int quickSelect(int[] nums, int left, int right, int k) {
    if (left == right) return nums[left];

    int pivotIdx = partition(nums, left, right);
    if (pivotIdx == k) return nums[pivotIdx];
    else if (pivotIdx < k) return quickSelect(nums, pivotIdx + 1, right, k);
    else return quickSelect(nums, left, pivotIdx - 1, k);
}

// 3. Meeting Rooms II (sort + min-heap)
int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    PriorityQueue<Integer> endTimes = new PriorityQueue<>();

    for (int[] interval : intervals) {
        if (!endTimes.isEmpty() && endTimes.peek() <= interval[0]) {
            endTimes.poll();  // reuse room
        }
        endTimes.offer(interval[1]);
    }
    return endTimes.size();
}
```

Interview Tip: When asked “can you sort this differently?”, think about: 1. Custom comparator (sort by a specific field) 2. Partial sort (QuickSelect for Kth element) 3. Non-comparison sort (counting sort, bucket sort when values have bounded range)

Pattern 1 — Sliding Window

Intuition

Imagine a window (a contiguous sub-array or substring) that “slides” over the input. Instead of recomputing from scratch for each position, you add the new element entering the window and remove the element leaving it.

Key insight: Avoid O(n²) by reusing computation from the previous window.

Pattern Recognition Signals

Look for these keywords in the problem: - “subarray” / “substring” / “contiguous” - “maximum/minimum of length k” - “longest/shortest with constraint” - “at most k distinct characters” - “sum equals k”

Types of Sliding Window

Type 1: Fixed Window Size

```
Window size is exactly k. Slide one step at a time. - Remove element leaving the left - Add element entering t
```

Type 2: Variable Window Size

```
Window grows on the right, shrinks from the left. - Expand right pointer always - Shrink left pointer when win
```

Templates

Fixed Window Template

```java
// Maximum sum of subarray of size k
int maxSumFixed(int[] arr, int k) {
    int n = arr.length;
    if (n < k) return -1;

    // Build initial window [0, k-1]
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];

    int maxSum = windowSum;

    // Slide window: add arr[right], remove arr[right - k]
    for (int right = k; right < n; right++) {
        windowSum += arr[right] - arr[right - k];
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

Variable Window Template

```java
// Longest window satisfying some condition
int longestVariable(int[] arr, int constraint) {
    int left = 0, maxLen = 0;
    // Some state to track window validity (map, count, sum, etc.)

    for (int right = 0; right < arr.length; right++) {
        // 1. Add arr[right] to window (expand)
        addToWindow(arr[right]);

        // 2. Shrink window from left while invalid
        while (windowIsInvalid()) {
            removeFromWindow(arr[left]);
            left++;
        }

        // 3. Update answer (window is now valid)
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Problem 1: Longest Substring Without Repeating Characters (LC 3)

Brute force: O(n²) — check every substring

Sliding window: O(n)

```java
// Signal keywords: "longest substring", "no repeating characters"
int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);

        // If c was seen and is inside current window
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
            left = lastSeen.get(c) + 1;  // shrink window past duplicate
        }

        lastSeen.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}

// Dry run: "abcabcbb"
// r=0: c='a', left=0, window="a", len=1
// r=1: c='b', left=0, window="ab", len=2
// r=2: c='c', left=0, window="abc", len=3
// r=3: c='a', seen at 0 >= left=0, left=1, window="bca", len=3
// r=4: c='b', seen at 1 >= left=1, left=2, window="cab", len=3
// Result: 3
```

Problem 2: Longest Substring with At Most K Distinct Characters (LC 340)

```
int lengthOfLongestSubstringKDistinct(String s, int k) {
    Map<Character, Integer> freq = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        freq.put(c, freq.getOrDefault(c, 0) + 1);  // expand

        // Shrink until at most k distinct
        while (freq.size() > k) {
            char leftChar = s.charAt(left);
            freq.put(leftChar, freq.get(leftChar) - 1);
            if (freq.get(leftChar) == 0) freq.remove(leftChar);
            left++;
        }

        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Problem 3: Minimum Window Substring (LC 76) — Hard

```java
// Find smallest window in s containing all chars of t
String minWindow(String s, String t) {
    if (s.length() < t.length()) return "";

    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    int left = 0, formed = 0, required = need.size();
    Map<Character, Integer> window = new HashMap<>();
    int[] ans = {-1, 0, 0};  // {length, left, right}

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.merge(c, 1, Integer::sum);

        if (need.containsKey(c) && window.get(c).equals(need.get(c))) {
            formed++;
        }

        // Shrink window while valid
        while (left <= right && formed == required) {
            if (ans[0] == -1 || right - left + 1 < ans[0]) {
                ans[0] = right - left + 1;
                ans[1] = left;
                ans[2] = right;
            }

            char leftChar = s.charAt(left);
            window.put(leftChar, window.get(leftChar) - 1);
            if (need.containsKey(leftChar) && window.get(leftChar) < need.get(leftChar)) {
                formed--;
            }
            left++;
        }
    }
    return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);
}
```

Problem 4: Maximum Sum Subarray of Size K (Fixed)

```
double findMaxAverage(int[] nums, int k) {
    long sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];

    long maxSum = sum;
    for (int i = k; i < nums.length; i++) {
        sum += nums[i] - nums[i - k];  // slide: add right, remove left
        maxSum = Math.max(maxSum, sum);
    }
    return (double) maxSum / k;
}
```

Problem 5: Permutation in String (LC 567)

```
// Is any permutation of p a substring of s?
boolean checkInclusion(String p, String s) {
    if (p.length() > s.length()) return false;

    int[] pCount = new int[26];
    int[] wCount = new int[26];

    for (char c : p.toCharArray()) pCount[c - 'a']++;

    int k = p.length();
    for (int i = 0; i < s.length(); i++) {
        wCount[s.charAt(i) - 'a']++;

        if (i >= k) wCount[s.charAt(i - k) - 'a']--;  // slide

        if (Arrays.equals(pCount, wCount)) return true;
    }
    return false;
}
```

Edge Cases

1. k > array length (fixed window) 2. All same characters 3. Empty string 4. Single character string 5. Window

---

Complexity Analysis

Problem	Time	Space
Fixed window	O(n)	O(1)
Variable window (charset)	O(n)	O(alphabet size)
Minimum window substring	O(n + m)	O(n + m)

Pattern Summary Table

Problem Type	Expand	Shrink Condition	Track
No repeating chars	Always right++	duplicate in window	lastSeen map
K distinct chars	Always right++	distinct > k	freq map
Min window with all chars	Always right++	all chars covered	freq map + formed count
Max sum of k	Fixed slide	—	running sum
All anagrams	Fixed slide	—	char count arrays

Pattern 2 — Two Pointers

Intuition

Use two indices (pointers) to scan the array from different positions simultaneously, eliminating the need for a nested loop.

Key insight: If data is sorted, two pointers let you make informed decisions about which pointer to move, achieving O(n) instead of O(n^2).

Pattern Recognition Signals

- “Pair with target sum in sorted array”
- “Three sum”, “Four sum”
- “Palindrome check”
- “Remove duplicates in-place”

- “Merge two sorted arrays”
- “Reverse array/string”
- “Container with most water”
- “Linked list cycle” (fast/slow)

Types of Two Pointers

Type 1: Opposite Direction (converge toward center)

`left → ← right Used for: sum problems on sorted arrays, palindromes`

Type 2: Same Direction (sliding window variant)

`left → → right Used for: remove duplicates, fast/slow cycle detection`

Type 3: Two Arrays

`i → (array 1) j → (array 2) Used for: merge sorted arrays, compare sequences`

Template 1: Two Sum (Sorted Array)

```
int[] twoSum(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left < right) {
        int sum = arr[left] + arr[right];

        if (sum == target) return new int[]{arr[left], arr[right]};
        else if (sum < target) left++;   // need larger sum
        else right--;                    // need smaller sum
    }
    return new int[]{};  // no pair found
}
```

Template 2: Three Sum (LC 15)

```java
// All unique triplets that sum to zero
List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);  // MUST sort first
    List<List<Integer>> result = new ArrayList<>();

    for (int i = 0; i < nums.length - 2; i++) {
        // Skip duplicates for first element
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = nums.length - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];

            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                // Skip duplicates for second and third elements
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++; right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}

// Dry run: [-1, 0, 1, 2, -1, -4] → sorted: [-4, -1, -1, 0, 1, 2]
// i=0: nums[i]=-4, l=1, r=5
//   sum=-4+(-1)+2=-3 < 0, l++
//   sum=-4+(-1)+2=-3 < 0, l++
//   sum=-4+0+2=-2 < 0, l++
//   sum=-4+1+2=-1 < 0, l++
//   l >= r, stop
// i=1: nums[i]=-1, l=2, r=5
//   sum=-1+(-1)+2=0 → add [-1,-1,2]
//   skip dups, l=3, r=4
//   sum=-1+0+1=0 → add [-1,0,1]
// i=2: nums[i]=-1 == nums[1], skip
// Result: [[-1,-1,2],[-1,0,1]]
```

Template 3: Container With Most Water (LC 11)

```java
// Two pointers converging, greedy choice
int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxWater = 0;

    while (left < right) {
        int h = Math.min(height[left], height[right]);
        int w = right - left;
        maxWater = Math.max(maxWater, h * w);

        // Move the pointer at the SHORTER wall
        // (moving the taller one can only decrease height, not increase)
        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

Template 4: Remove Duplicates from Sorted Array (LC 26)

```java
// In-place, O(1) extra space
// slow pointer marks position for next unique element
// fast pointer scans ahead
int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;

    int slow = 0;  // last position of unique element

    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;  // count of unique elements
}
```

Template 5: Fast/Slow Pointers (Floyd’s Cycle Detection)

```java
// Detect cycle in linked list
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;          // moves 1 step
        fast = fast.next.next;     // moves 2 steps

        if (slow == fast) return true;  // cycle detected
    }
    return false;
}

// Find start of cycle
ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }

    if (fast == null || fast.next == null) return null;

    // Reset slow to head; fast stays at meeting point
    // Both move at speed 1; they meet at cycle start
    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }
    return slow;
}

// Find middle of linked list
ListNode findMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;  // for odd: exact middle; for even: upper middle
}
```

Template 6: Trapping Rain Water (LC 42) — Two Pointers

```java
int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) leftMax = height[left];
            else water += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax) rightMax = height[right];
            else water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
// Key insight: water at position i = min(maxLeft[i], maxRight[i]) - height[i]
// Two pointers avoid needing O(n) precomputed arrays
```

Template 7: Palindrome Verification

```
boolean isPalindrome(String s) {
    // Clean: keep only alphanumeric, lowercase
    int left = 0, right = s.length() - 1;

    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;

        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) return false;
        left++;
        right--;
    }
    return true;
}
```

Merge Two Sorted Arrays

```
void mergeSorted(int[] arr1, int m, int[] arr2, int n) {
    // Merge arr2 into arr1 (arr1 has extra space at end)
    int i = m - 1, j = n - 1, k = m + n - 1;  // start from the END

    while (i >= 0 && j >= 0) {
        if (arr1[i] > arr2[j]) arr1[k--] = arr1[i--];
        else arr1[k--] = arr2[j--];
    }
    while (j >= 0) arr1[k--] = arr2[j--];
}
```

Edge Cases

1. Array with < 2 elements 2. All elements equal 3. Already sorted in required order 4. Target sum not achieva

---

Complexity Summary

Problem	Brute Force	Two Pointers
Two Sum (sorted)	$O(n^2)$	$O(n)$
Three Sum	$O(n^3)$	$O(n^2)$
Remove Duplicates	$O(n^2)$	$O(n)$
Container With Most Water	$O(n^2)$	$O(n)$
Trapping Rain Water	$O(n^2)$	$O(n)$
Cycle Detection	O(n) space	O(1) space

Pattern 3 — Binary Search

Intuition

Binary search eliminates half the search space with each comparison. Works on **monotonic** data — where there's a clear "left half" and "right half" separated by the answer.

Key insight: You don't need a sorted array. You need a **condition** that's monotonically true/false, letting you eliminate half the space.

Pattern Recognition Signals

- "Sorted array", "rotated sorted array"
- "Find minimum/maximum satisfying condition"
- "Kth smallest/largest"
- "Minimize the maximum", "Maximize the minimum"
- "Feasibility check" with a clear threshold

The Universal Binary Search Template

```
// Find the LEFTMOST position where condition(x) is TRUE
// condition must be: FFFFFFF TTTTTTT (monotonic)
int binarySearch(int lo, int hi) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (condition(mid)) {
            hi = mid;        // true: answer is mid or to the left
        } else {
            lo = mid + 1;    // false: answer is to the right
        }
    }
    return lo;  // lo == hi == first true position
}
```

---

5 Standard Binary Search Problems

1. Classic Search

```
int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

2. First Bad Version (LC 278)

```java
// isBadVersion(n) returns whether n is bad
// Find the first bad version (all versions after it are also bad)
int firstBadVersion(int n) {
    int lo = 1, hi = n;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (isBadVersion(mid)) hi = mid;    // bad: could be the first
        else lo = mid + 1;                   // good: first bad is to the right
    }
    return lo;
}
```

3. Search Insert Position (LC 35)

```java
// Find index where target would be inserted to keep sorted order
int searchInsert(int[] nums, int target) {
    int lo = 0, hi = nums.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

4. Rotated Sorted Array (LC 33)

```
int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;

    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;

        // Left portion is sorted
        if (nums[lo] <= nums[mid]) {
            if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        }
        // Right portion is sorted
        else {
            if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

5. Find Minimum in Rotated Sorted Array (LC 153)

```
int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) lo = mid + 1;  // min is in right half
        else hi = mid;                           // min is in left half (or mid)
    }
    return nums[lo];
}
```

Binary Search on Answer Space

Koko Eating Bananas (LC 875)

```
// Can finish at speed k within h hours?
boolean canFinish(int[] piles, int h, int k) {
    long hours = 0;
    for (int p : piles) hours += (p + k - 1) / k;  // ceil(p/k)
    return hours <= h;
}

int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 1;
    for (int p : piles) hi = Math.max(hi, p);  // max pile = upper bound

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canFinish(piles, h, mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Capacity to Ship Packages (LC 1011)

```
boolean canShip(int[] weights, int days, int cap) {
    int daysNeeded = 1, currLoad = 0;
    for (int w : weights) {
        if (currLoad + w > cap) { daysNeeded++; currLoad = 0; }
        currLoad += w;
    }
    return daysNeeded <= days;
}

int shipWithinDays(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) { lo = Math.max(lo, w); hi += w; }
    // lo = max single weight (minimum possible capacity)
    // hi = sum of all (1 day capacity)

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canShip(weights, days, mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Split Array Largest Sum (LC 410)

```
boolean canSplit(int[] nums, int m, int maxSum) {
    int pieces = 1, curr = 0;
    for (int n : nums) {
        if (curr + n > maxSum) { pieces++; curr = 0; }
        curr += n;
    }
    return pieces <= m;
}

int splitArray(int[] nums, int m) {
    int lo = 0, hi = 0;
    for (int n : nums) { lo = Math.max(lo, n); hi += n; }

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canSplit(nums, m, mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Aggressive Cows / Maximize Minimum Distance (Atcoder/Codeforces classic)

```
boolean canPlace(int[] positions, int n, int minDist) {
    int count = 1, last = positions[0];
    for (int i = 1; i < positions.length; i++) {
        if (positions[i] - last >= minDist) {
            count++;
            last = positions[i];
            if (count >= n) return true;
        }
    }
    return false;
}

int maxMinDist(int[] positions, int n) {
    Arrays.sort(positions);
    int lo = 1, hi = positions[positions.length - 1] - positions[0];

    while (lo < hi) {
        int mid = lo + (hi - lo + 1) / 2;  // +1 for "maximize" problems
        if (canPlace(positions, n, mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
// Note: for "minimize" use hi=mid; for "maximize" use lo=mid (with +1 in mid calc)
```

Special: 2D Binary Search (LC 74)

```java
// Search in m×n matrix where rows/cols are sorted
boolean searchMatrix(int[][] matrix, int target) {
    int m = matrix.length, n = matrix[0].length;
    int lo = 0, hi = m * n - 1;

    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int val = matrix[mid / n][mid % n];  // convert 1D index to 2D
        if (val == target) return true;
        if (val < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return false;
}
```

Off-by-One Rules (The Hard Part)

```java
// RULE 1: When to use left < right vs left <= right
// - left <= right: when searching for exact target in [left, right]
// - left < right: when searching for boundary (template-based)

// RULE 2: When to use hi = mid vs hi = mid - 1
// - hi = mid: when condition is true at mid, but mid could BE the answer
// - hi = mid - 1: when you've confirmed mid is NOT the answer

// RULE 3: Mid formula for "maximize" problems
int midForMax = lo + (hi - lo + 1) / 2;  // rounds up to avoid infinite loop

// Example: lo=3, hi=4
// Standard mid = 3 + (4-3)/2 = 3 → if condition(3)=false: lo=mid=3 (infinite loop!)
// Ceiling mid = 3 + (4-3+1)/2 = 4 → lo=mid=4, breaks loop

// RULE 4: Post-condition check
// After binary search, always validate:
// - Is lo within bounds?
// - Does nums[lo] actually equal target?
```

Complexity

Type	Time	Space
Basic binary search	O(log n)	O(1)
Binary search on answer	O(n log(range))	O(1)
Recursive binary search	O(log n)	O(log n)

> **Interview Tip:** When stuck on a problem with “minimum/maximum satisfying a constraint”, ask yourself: “Is this monotonic? Can I binary search the answer?” This unlocks a whole class of hard problems.

Pattern 4 — Prefix Sum

Intuition

Pre-compute cumulative sums to answer range sum queries in O(1) instead of O(n).

Key insight: Sum of any subarray [l, r] = prefix[r+1] - prefix[l]

Pattern Recognition Signals

- “Sum of subarray/range”
- “Number of subarrays with sum = k”
- “Query: sum from index l to r”
- “Count subarrays with specific sum”
- “Balanced parentheses count”
- “Range update queries”

Template 1: 1D Prefix Sum

```
// Build prefix sum array
int[] buildPrefix(int[] arr) {
    int n = arr.length;
    int[] prefix = new int[n + 1];  // prefix[0] = 0 (empty prefix)
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    return prefix;
}

// Range sum query in O(1)
int rangeSum(int[] prefix, int l, int r) {
    return prefix[r + 1] - prefix[l];  // sum of arr[l..r] inclusive
}

// Example:
// arr:    [1, 2, 3, 4, 5]
// prefix: [0, 1, 3, 6, 10, 15]
// sum(1, 3) = prefix[4] - prefix[1] = 10 - 1 = 9    (2+3+4)
```

Template 2: 2D Prefix Sum

```
// Matrix range sum query
int[][] build2DPrefix(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    int[][] prefix = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            prefix[i][j] = matrix[i-1][j-1]
                         + prefix[i-1][j]
                         + prefix[i][j-1]
                         - prefix[i-1][j-1];  // subtract double-counted corner
        }
    }
    return prefix;
}

// Query: sum of rectangle (r1,c1) to (r2,c2) in original matrix (0-indexed)
int query(int[][] prefix, int r1, int c1, int r2, int c2) {
    return prefix[r2+1][c2+1]
         - prefix[r1][c2+1]
         - prefix[r2+1][c1]
         + prefix[r1][c1];  // add back double-subtracted corner
}
```

Problem 1: Subarray Sum Equals K (LC 560)

The most important prefix sum pattern

```java
// Count subarrays with sum exactly k
// Key insight: if prefix[j] - prefix[i] = k, then prefix[i] = prefix[j] - k
int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);  // empty prefix (sum = 0) exists once

    int sum = 0, count = 0;

    for (int n : nums) {
        sum += n;
        count += prefixCount.getOrDefault(sum - k, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return count;
}

// Dry run: nums = [1, 1, 1], k = 2
// sum=0: prefixCount={0:1}
// n=1: sum=1, need sum-k=1-2=-1, count+=0, prefixCount={0:1, 1:1}
// n=1: sum=2, need sum-k=2-2=0, count+=1, prefixCount={0:1, 1:1, 2:1}
// n=1: sum=3, need sum-k=3-2=1, count+=1, prefixCount={0:1, 1:1, 2:1, 3:1}
// Result: count=2    ([1,1] starting at 0 and [1,1] starting at 1)
```

Problem 2: Continuous Subarray Sum (LC 523)

```java
// Check if any subarray of length >= 2 has sum that's multiple of k
boolean checkSubarraySum(int[] nums, int k) {
    Map<Integer, Integer> modSeen = new HashMap<>();
    modSeen.put(0, -1);  // empty prefix, seen at "index -1"

    int sum = 0;
    for (int i = 0; i < nums.length; i++) {
        sum = (sum + nums[i]) % k;

        if (modSeen.containsKey(sum)) {
            if (i - modSeen.get(sum) >= 2) return true;  // length >= 2
        } else {
            modSeen.put(sum, i);  // only store FIRST occurrence
        }
    }
    return false;
}
// Key insight: if prefix[j] % k == prefix[i] % k, then sum(i..j) % k == 0
```

Problem 3: Range Sum Query — Immutable (LC 303)

```java
class NumArray {
    private int[] prefix;

    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }

    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

Problem 4: Maximum Size Subarray Sum Equals k (LC 325)

```
int maxSubArrayLen(int[] nums, int k) {
    Map<Integer, Integer> firstSeen = new HashMap<>();
    firstSeen.put(0, -1);  // empty prefix starts at -1

    int sum = 0, maxLen = 0;

    for (int i = 0; i < nums.length; i++) {
        sum += nums[i];

        if (firstSeen.containsKey(sum - k)) {
            maxLen = Math.max(maxLen, i - firstSeen.get(sum - k));
        }

        if (!firstSeen.containsKey(sum)) {
            firstSeen.put(sum, i);  // only store FIRST occurrence for max length
        }
    }
    return maxLen;
}
```

Problem 5: Difference Array (Range Update)

```
// Update range [l, r] by adding val — O(1) per update, O(n) to reconstruct
int[] differenceArray(int n) {
    return new int[n + 1];
}

void rangeAdd(int[] diff, int l, int r, int val) {
    diff[l] += val;
    diff[r + 1] -= val;
}

int[] reconstruct(int[] diff, int n) {
    int[] result = new int[n];
    result[0] = diff[0];
    for (int i = 1; i < n; i++) {
        result[i] = result[i - 1] + diff[i];
    }
    return result;
}

// Example: n=5, add 3 to [1,3], add -1 to [2,4]
// diff: [0, 3, -1, 0, -3, 1]  (and extra -1 at index 4+1=5: diff[5]-=1)
// Wait, recompute:
// rangeAdd(diff, 1, 3, 3): diff[1]+=3, diff[4]-=3  → diff=[0,3,0,0,-3,0]
// rangeAdd(diff, 2, 4, -1): diff[2]+=-1, diff[5]-= -1  → diff=[0,3,-1,0,-3,1]
// reconstruct: [0, 3, 2, 2, -1]  → result[1]=3, result[2]=2, result[3]=2, result[4]=-1
```

Problem 6: Count of Subarrays with Balance 0 (0/1 problems)

```java
// Count subarrays with equal 0s and 1s
int countEqualZeroOne(int[] nums) {
    // Convert 0 → -1, so equal count means sum = 0
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);

    int sum = 0, count = 0;
    for (int n : nums) {
        sum += (n == 0) ? -1 : 1;
        count += prefixCount.getOrDefault(sum, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Complexity Summary

Pattern	Preprocessing	Query
1D prefix sum	O(n)	O(1)
2D prefix sum	O(m*n)	O(1)
Prefix sum + HashMap	O(n)	O(n) total
Difference array	O(1) per update	O(n) to read

> **Interview Tip:** Whenever you see “sum of subarray” or “how many subarrays with sum = k”, immediately think: prefix sum. Then ask: do I need exact count (HashMap) or just a check (check prefix values)?

Pattern 5 — HashMap / HashSet Patterns

Core Insight

Hash tables provide O(1) average lookup, enabling many $O(n^2)$ problems to be solved in O(n).

Key applications: frequency counting, grouping, seen-before checks, complement lookups.

Pattern 1: Frequency Counting

```java
// Count occurrences of each element
Map<Integer, Integer> freq = new HashMap<>();
for (int n : nums) freq.merge(n, 1, Integer::sum);

// OR with getOrDefault:
for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

// Character frequency
int[] charFreq = new int[26];
for (char c : s.toCharArray()) charFreq[c - 'a']++;

// Top K frequent elements
List<Map.Entry<Integer, Integer>> entries = new ArrayList<>(freq.entrySet());
entries.sort((a, b) -> b.getValue() - a.getValue());
```

Pattern 2: Two Sum Variants

```java
// Two Sum (LC 1) — exact indices
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();  // value → index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{};
}

// Two Sum II (sorted) — use two pointers instead
// Two Sum IV (BST) — use HashSet + traversal
boolean findTarget(TreeNode root, int k) {
    Set<Integer> seen = new HashSet<>();
    return dfs(root, k, seen);
}
boolean dfs(TreeNode node, int k, Set<Integer> seen) {
    if (node == null) return false;
    if (seen.contains(k - node.val)) return true;
    seen.add(node.val);
    return dfs(node.left, k, seen) || dfs(node.right, k, seen);
}
```

Pattern 3: Grouping / Bucketing

```java
// Group Anagrams (LC 49)
List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String s : strs) {
        char[] chars = s.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(groups.values());
}

// Alternative key without sorting (faster):
String getKey(String s) {
    int[] freq = new int[26];
    for (char c : s.toCharArray()) freq[c - 'a']++;
    return Arrays.toString(freq);  // "#2#0#0#..." style key
}
```

Pattern 4: Seen-Before / Duplicate Detection

```java
// Contains Duplicate (LC 217)
boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums) {
        if (!seen.add(n)) return true;  // add returns false if already present
    }
    return false;
}

// Contains Duplicate II (LC 219) — within k distance
boolean containsNearbyDuplicate(int[] nums, int k) {
    Map<Integer, Integer> indexMap = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        if (indexMap.containsKey(nums[i]) && i - indexMap.get(nums[i]) <= k) {
            return true;
        }
        indexMap.put(nums[i], i);
    }
    return false;
}
```

Pattern 5: Complement Pattern

```java
// Find all pairs with difference = k
List<int[]> findPairsWithDiff(int[] nums, int k) {
    Set<Integer> numSet = new HashSet<>();
    Set<String> seen = new HashSet<>();
    List<int[]> result = new ArrayList<>();

    for (int n : nums) numSet.add(n);

    for (int n : nums) {
        int complement = n - k;
        if (numSet.contains(complement) && complement != n) {
            String key = Math.min(n, complement) + "," + Math.max(n, complement);
            if (seen.add(key)) result.add(new int[]{complement, n});
        }
    }
    return result;
}
```

Pattern 6: LRU Cache (Design Problem)

```java
// LRU Cache (LC 146) — O(1) get and put
class LRUCache {
    private final int capacity;
    private final Map<Integer, Node> map;
    private final Node head, tail;  // dummy head and tail

    static class Node {
        int key, val;
        Node prev, next;
        Node(int k, int v) { key = k; val = v; }
    }

    public LRUCache(int capacity) {
        this.capacity = capacity;
        map = new HashMap<>();
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node);
        insertToFront(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            remove(map.get(key));
        }
        if (map.size() == capacity) {
            Node lru = tail.prev;  // least recently used
            remove(lru);
            map.remove(lru.key);
        }
        Node node = new Node(key, value);
        insertToFront(node);
        map.put(key, node);
    }

    private void remove(Node node) {
        node.prev.next = node.next;
```

```
            node.next.prev = node.prev;
        }

        private void insertToFront(Node node) {
            node.next = head.next;
            node.prev = head;
            head.next.prev = node;
            head.next = node;
        }
    }

    // Simpler implementation using LinkedHashMap:
    class LRUCacheSimple extends LinkedHashMap<Integer, Integer> {
        private final int capacity;

        public LRUCacheSimple(int capacity) {
            super(capacity, 0.75f, true);  // true = access order
            this.capacity = capacity;
        }

        public int get(int key) { return super.getOrDefault(key, -1); }
        public void put(int key, int value) { super.put(key, value); }

        @Override
        protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
            return size() > capacity;
        }
    }
```

Pattern 7: Isomorphic / Pattern Matching

```java
// Isomorphic Strings (LC 205)
boolean isIsomorphic(String s, String t) {
    Map<Character, Character> sToT = new HashMap<>();
    Map<Character, Character> tToS = new HashMap<>();

    for (int i = 0; i < s.length(); i++) {
        char sc = s.charAt(i), tc = t.charAt(i);

        if (sToT.containsKey(sc) && sToT.get(sc) != tc) return false;
        if (tToS.containsKey(tc) && tToS.get(tc) != sc) return false;

        sToT.put(sc, tc);
        tToS.put(tc, sc);
    }
    return true;
}

// Word Pattern (LC 290)
boolean wordPattern(String pattern, String s) {
    String[] words = s.split(" ");
    if (pattern.length() != words.length) return false;

    Map<Character, String> charToWord = new HashMap<>();
    Map<String, Character> wordToChar = new HashMap<>();

    for (int i = 0; i < pattern.length(); i++) {
        char p = pattern.charAt(i);
        String w = words[i];

        if (charToWord.containsKey(p) && !charToWord.get(p).equals(w)) return false;
        if (wordToChar.containsKey(w) && wordToChar.get(w) != p) return false;

        charToWord.put(p, w);
        wordToChar.put(w, p);
    }
    return true;
}
```

Complexity Summary

Operation	HashMap	HashSet
Insert	O(1) avg	O(1) avg
Lookup	O(1) avg	O(1) avg
Delete	O(1) avg	O(1) avg
Iterate	O(n)	O(n)
Space	O(n)	O(n)

> **Interview Tip:** “I’ll use a HashMap to trade space for time” — this is one of the most frequent optimizations in interviews. Master the frequency map pattern and its variants.

Pattern 6 — Stack Patterns

Core Insight

Stacks excel at problems where you need to **process elements in reverse order** or where the answer to the current element depends on **previous unseen elements**.

Pattern 1: Monotonic Stack (Most Important)

A monotonic stack maintains elements in sorted order (increasing or decreasing), popping elements when the order is violated.

Use for: Next Greater/Smaller Element, Histogram, Stock Span

Next Greater Element Template

```java
// For each element, find the next greater element to its right
int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);  // default: no greater element
    Deque<Integer> stack = new ArrayDeque<>();  // stores INDICES

    for (int i = 0; i < n; i++) {
        // Pop all elements smaller than current → current is their "next greater"
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}

// Dry run: nums = [2, 1, 2, 4, 3]
// i=0: stack=[0(val=2)]
// i=1: 1 < 2, push → stack=[0,1]
// i=2: 2 >= 1, pop 1 → result[1]=2; 2 == 2, no pop → stack=[0,2]
// i=3: 4 > 2, pop 2 → result[2]=4; 4 > 2, pop 0 → result[0]=4 → stack=[3]
// i=4: 3 < 4, push → stack=[3,4]
// result: [4, 2, 4, -1, -1]
```

Next Greater Element — Circular Array (LC 503)

```
int[] nextGreaterElementsCircular(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    // Loop twice to simulate circular behavior
    for (int i = 0; i < 2 * n; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i % n]) {
            result[stack.pop()] = nums[i % n];
        }
        if (i < n) stack.push(i);
    }
    return result;
}
```

Pattern 2: Largest Rectangle in Histogram (LC 84)

```java
// Classic monotonic stack problem
int largestRectangleArea(int[] heights) {
    int n = heights.length;
    Deque<Integer> stack = new ArrayDeque<>();
    int maxArea = 0;

    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];  // sentinel 0 at end to flush stack

        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}

// Maximal Rectangle in Binary Matrix (LC 85) — builds on histogram
int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int n = matrix[0].length;
    int[] heights = new int[n];
    int maxArea = 0;

    for (char[] row : matrix) {
        for (int j = 0; j < n; j++) {
            heights[j] = row[j] == '0' ? 0 : heights[j] + 1;
        }
        maxArea = Math.max(maxArea, largestRectangleArea(heights));
    }
    return maxArea;
}
```

Pattern 3: Stock Span / Daily Temperatures (LC 739)

```java
// For each day, how many days until a warmer temperature?
int[] dailyTemperatures(int[] temps) {
    int n = temps.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();  // indices of unresolved days

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temps[stack.peek()] < temps[i]) {
            int prevDay = stack.pop();
            result[prevDay] = i - prevDay;  // days to wait
        }
        stack.push(i);
    }
    return result;  // remaining elements = 0 (no warmer day found)
}
```

Pattern 4: Bracket Matching

```java
// Valid Parentheses (LC 20)
boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false;
        }
    }
    return stack.isEmpty();
}

// Minimum Remove to Make Valid Parentheses (LC 1249)
String minRemoveToMakeValid(String s) {
    Deque<Integer> stack = new ArrayDeque<>();  // unmatched '(' indices
    Set<Integer> remove = new HashSet<>();

    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == '(') {
            stack.push(i);
        } else if (c == ')') {
            if (stack.isEmpty()) remove.add(i);  // unmatched ')'
            else stack.pop();
        }
    }

    while (!stack.isEmpty()) remove.add(stack.pop());  // unmatched '('

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < s.length(); i++) {
        if (!remove.contains(i)) sb.append(s.charAt(i));
    }
    return sb.toString();
}
```

Pattern 5: Calculator Problems

```java
// Basic Calculator II (LC 227) — +, -, *, /
int calculate(String s) {
    Deque<Integer> stack = new ArrayDeque<>();
    int curr = 0;
    char op = '+';

    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);

        if (Character.isDigit(c)) {
            curr = curr * 10 + (c - '0');
        }

        if ((!Character.isDigit(c) && c != ' ') || i == s.length() - 1) {
            switch (op) {
                case '+': stack.push(curr); break;
                case '-': stack.push(-curr); break;
                case '*': stack.push(stack.pop() * curr); break;
                case '/': stack.push(stack.pop() / curr); break;
            }
            op = c;
            curr = 0;
        }
    }

    int result = 0;
    while (!stack.isEmpty()) result += stack.pop();
    return result;
}
```

Pattern 6: Decode String (LC 394)

```java
// "3[a]2[bc]" → "aaabcbc"
// "3[a2[c]]" → "accaccacc"
String decodeString(String s) {
    Deque<Integer> countStack = new ArrayDeque<>();
    Deque<StringBuilder> strStack = new ArrayDeque<>();
    StringBuilder curr = new StringBuilder();
    int k = 0;

    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            k = k * 10 + (c - '0');
        } else if (c == '[') {
            countStack.push(k);
            strStack.push(curr);
            curr = new StringBuilder();
            k = 0;
        } else if (c == ']') {
            int count = countStack.pop();
            StringBuilder prev = strStack.pop();
            for (int i = 0; i < count; i++) prev.append(curr);
            curr = prev;
        } else {
            curr.append(c);
        }
    }
    return curr.toString();
}
```

Monotonic Stack — Choosing Direction

Problem	Stack Type	Direction
Next greater to right	Decreasing	Left to right
Next greater to left	Decreasing	Right to left
Next smaller to right	Increasing	Left to right
Next smaller to left	Increasing	Right to left

```java
// Template for next smaller to the LEFT
int[] prevSmaller(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] >= nums[i]) stack.pop();
        result[i] = stack.isEmpty() ? -1 : nums[stack.peek()];
        stack.push(i);
    }
    return result;
}
```

Complexity

Pattern	Time	Space
Monotonic stack	O(n)	O(n)
Histogram area	O(n)	O(n)
Bracket matching	O(n)	O(n)
Calculator	O(n)	O(n)

> **Interview Tip:** When you see “next greater/smaller element”, immediately think monotonic stack. It’s the key pattern for histogram problems, stock prices, and temperature-change problems.

Pattern 7 — Tree Patterns

Tree Node Definition

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

---

1. Tree Traversals

DFS — Recursive (Cleaner)

```
// Inorder: Left → Root → Right (gives sorted order for BST)
void inorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    inorder(root.left, result);
    result.add(root.val);
    inorder(root.right, result);
}

// Preorder: Root → Left → Right (good for serialization/copying)
void preorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    result.add(root.val);
    preorder(root.left, result);
    preorder(root.right, result);
}

// Postorder: Left → Right → Root (good for deletion, size calculation)
void postorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    postorder(root.left, result);
    postorder(root.right, result);
    result.add(root.val);
}
```

DFS — Iterative with Stack

```java
// Iterative inorder
List<Integer> inorderIterative(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        result.add(curr.val);
        curr = curr.right;
    }
    return result;
}

// Iterative preorder
List<Integer> preorderIterative(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val);
        if (node.right != null) stack.push(node.right);  // right first (LIFO)
        if (node.left != null) stack.push(node.left);
    }
    return result;
}
```

BFS — Level Order

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

2. Height and Diameter

```
// Height (max depth)
int height(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(height(root.left), height(root.right));
}

// Diameter of Binary Tree (LC 543)
// Diameter = longest path (may not pass through root)
int diameterOfBinaryTree(TreeNode root) {
    int[] maxDiam = {0};

    // Returns height, updates maxDiam as side effect
    java.util.function.Function<TreeNode, Integer> dfs = null;
    dfs = node -> {
        if (node == null) return 0;
        // Can't use lambda recursion directly — use helper method
        return 0;
    };

    // Proper implementation with helper
    maxDiamHelper(root, maxDiam);
    return maxDiam[0];
}

int maxDiamHelper(TreeNode node, int[] maxDiam) {
    if (node == null) return 0;
    int left = maxDiamHelper(node.left, maxDiam);
    int right = maxDiamHelper(node.right, maxDiam);
    maxDiam[0] = Math.max(maxDiam[0], left + right);
    return 1 + Math.max(left, right);
}
```

3. Path Sum Problems

```java
// Path Sum I — does root-to-leaf path with targetSum exist?
boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;
    if (root.left == null && root.right == null) return root.val == targetSum;
    return hasPathSum(root.left, targetSum - root.val) ||
           hasPathSum(root.right, targetSum - root.val);
}

// Path Sum II — all root-to-leaf paths with targetSum
List<List<Integer>> pathSum(TreeNode root, int targetSum) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(root, targetSum, new ArrayList<>(), result);
    return result;
}

void backtrack(TreeNode node, int remaining, List<Integer> path, List<List<Integer>> result) {
    if (node == null) return;
    path.add(node.val);

    if (node.left == null && node.right == null && remaining == node.val) {
        result.add(new ArrayList<>(path));
    } else {
        backtrack(node.left, remaining - node.val, path, result);
        backtrack(node.right, remaining - node.val, path, result);
    }
    path.remove(path.size() - 1);  // backtrack
}

// Path Sum III — any path (not just root-to-leaf), sum = target
// Approach: prefix sum + HashMap
int pathSumIII(TreeNode root, int targetSum) {
    Map<Long, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0L, 1);
    return dfsPathSum(root, 0, targetSum, prefixCount);
}

int dfsPathSum(TreeNode node, long curr, int target, Map<Long, Integer> prefixCount) {
    if (node == null) return 0;
    curr += node.val;
    int count = prefixCount.getOrDefault(curr - target, 0);
    prefixCount.merge(curr, 1, Integer::sum);

    count += dfsPathSum(node.left, curr, target, prefixCount);
    count += dfsPathSum(node.right, curr, target, prefixCount);
```

```
        prefixCount.merge(curr, -1, Integer::sum);  // backtrack
        return count;
    }
```

---

4. Lowest Common Ancestor (LCA)

```
    // LCA of Binary Tree (LC 236)
    TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        // If both found in different subtrees → current is LCA
        if (left != null && right != null) return root;
        // Otherwise, return the non-null one
        return left != null ? left : right;
    }

    // LCA of BST (LC 235) — simpler, uses BST property
    TreeNode lowestCommonAncestorBST(TreeNode root, TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) root = root.left;
            else if (p.val > root.val && q.val > root.val) root = root.right;
            else return root;  // root is between p and q → LCA
        }
        return null;
    }
```

---

5. BST Problems

```java
// Validate BST (LC 98)
boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return validate(node.left, min, node.val) &&
           validate(node.right, node.val, max);
}

// Kth Smallest in BST (LC 230)
int kthSmallest(TreeNode root, int k) {
    int[] count = {0, 0};  // [counter, result]
    inorderKth(root, k, count);
    return count[1];
}

void inorderKth(TreeNode node, int k, int[] count) {
    if (node == null || count[0] >= k) return;
    inorderKth(node.left, k, count);
    count[0]++;
    if (count[0] == k) { count[1] = node.val; return; }
    inorderKth(node.right, k, count);
}

// Insert in BST
TreeNode insertBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val) root.left = insertBST(root.left, val);
    else root.right = insertBST(root.right, val);
    return root;
}

// Delete in BST
TreeNode deleteBST(TreeNode root, int key) {
    if (root == null) return null;
    if (key < root.val) root.left = deleteBST(root.left, key);
    else if (key > root.val) root.right = deleteBST(root.right, key);
    else {
        // Node found
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;
        // Has two children: replace with inorder successor
```

```
            TreeNode successor = root.right;
            while (successor.left != null) successor = successor.left;
            root.val = successor.val;
            root.right = deleteBST(root.right, successor.val);
        }
        return root;
    }
```

---

6. Tree Serialization (LC 297)

```
    // Serialize: preorder traversal with null markers
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serializeHelper(root, sb);
        return sb.toString();
    }

    void serializeHelper(TreeNode node, StringBuilder sb) {
        if (node == null) { sb.append("N,"); return; }
        sb.append(node.val).append(',');
        serializeHelper(node.left, sb);
        serializeHelper(node.right, sb);
    }

    // Deserialize: rebuild from preorder string
    public TreeNode deserialize(String data) {
        Deque<String> queue = new ArrayDeque<>(Arrays.asList(data.split(",")));
        return deserializeHelper(queue);
    }

    TreeNode deserializeHelper(Deque<String> queue) {
        String val = queue.poll();
        if ("N".equals(val)) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = deserializeHelper(queue);
        node.right = deserializeHelper(queue);
        return node;
    }
```

---

7. Right Side View (LC 199)

```
List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) result.add(node.val);  // rightmost of level
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return result;
}
```

Complexity Summary

Operation	BST (balanced)	BST (unbalanced)	General Tree
Search	O(log n)	O(n)	O(n)
Insert	O(log n)	O(n)	O(n)
Delete	O(log n)	O(n)	O(n)
Traversal	O(n)	O(n)	O(n)
Height	O(log n)	O(n)	O(n)

> **Interview Tip:** For any tree problem, first identify: Is it a BST? If yes, use BST properties (O(log n) operations). If it’s a general binary tree, you’ll need O(n) traversal. Mention this distinction early.

Pattern 8 — Graph Patterns

Graph Representations

```
// Adjacency List (most common)
Map<Integer, List<Integer>> adj = new HashMap<>();
// Add undirected edge
adj.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
adj.computeIfAbsent(v, k -> new ArrayList<>()).add(u);

// Adjacency Matrix (dense graphs, O(1) edge check)
int[][] matrix = new int[n][n];
matrix[u][v] = 1;

// Edge List (for Kruskal's MST)
int[][] edges = {{0,1,4}, {1,2,1}, ...};  // [u, v, weight]

// Grid as Graph (4-directional)
int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
int[][] grid = new int[rows][cols];
```

1. BFS — Shortest Path in Unweighted Graph

```java
int[] bfsShortestPath(Map<Integer, List<Integer>> adj, int src, int n) {
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    dist[src] = 0;

    Queue<Integer> queue = new ArrayDeque<>();
    queue.offer(src);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : adj.getOrDefault(node, new ArrayList<>())) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[node] + 1;
                queue.offer(neighbor);
            }
        }
    }
    return dist;
}

// BFS on Grid (Number of Islands BFS approach)
int numIslands(char[][] grid) {
    int islands = 0;
    int rows = grid.length, cols = grid[0].length;
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                islands++;
                Queue<int[]> queue = new ArrayDeque<>();
                queue.offer(new int[]{r, c});
                grid[r][c] = '0';  // mark visited

                while (!queue.isEmpty()) {
                    int[] curr = queue.poll();
                    for (int[] dir : dirs) {
                        int nr = curr[0] + dir[0], nc = curr[1] + dir[1];
                        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                            && grid[nr][nc] == '1') {
                            grid[nr][nc] = '0';
                            queue.offer(new int[]{nr, nc});
                        }
                    }
                }
```

```
                }
            }
        }
        return islands;
    }
```

---

2. DFS — Connected Components, Path Finding

```
    // DFS template (iterative)
    void dfs(Map<Integer, List<Integer>> adj, int start, boolean[] visited) {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(start);
        visited[start] = true;

        while (!stack.isEmpty()) {
            int node = stack.pop();
            // process node

            for (int neighbor : adj.getOrDefault(node, new ArrayList<>())) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    stack.push(neighbor);
                }
            }
        }
    }

    // DFS recursive
    void dfsRecursive(int node, boolean[] visited, Map<Integer, List<Integer>> adj) {
        visited[node] = true;
        // process node
        for (int neighbor : adj.getOrDefault(node, new ArrayList<>())) {
            if (!visited[neighbor]) {
                dfsRecursive(neighbor, visited, adj);
            }
        }
    }
```

---

3. Cycle Detection

Undirected Graph

```
boolean hasCycleUndirected(int n, int[][] edges) {
    UnionFind uf = new UnionFind(n);
    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) return true;  // already connected
    }
    return false;
}

// Or DFS approach
boolean hasCycleDFS(Map<Integer, List<Integer>> adj, int n) {
    boolean[] visited = new boolean[n];
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dfsHasCycle(adj, i, -1, visited)) return true;
    }
    return false;
}

boolean dfsHasCycle(Map<Integer, List<Integer>> adj, int node, int parent, boolean[] visited) {
    visited[node] = true;
    for (int neighbor : adj.getOrDefault(node, new ArrayList<>())) {
        if (!visited[neighbor]) {
            if (dfsHasCycle(adj, neighbor, node, visited)) return true;
        } else if (neighbor != parent) {
            return true;  // back edge = cycle
        }
    }
    return false;
}
```

Directed Graph

```java
// Use 3-color DFS: 0=white(unvisited), 1=gray(in stack), 2=black(done)
boolean hasCycleDirected(int n, Map<Integer, List<Integer>> adj) {
    int[] color = new int[n];  // 0=unvisited, 1=in stack, 2=done
    for (int i = 0; i < n; i++) {
        if (color[i] == 0 && dfsCycle(adj, i, color)) return true;
    }
    return false;
}

boolean dfsCycle(Map<Integer, List<Integer>> adj, int node, int[] color) {
    color[node] = 1;  // gray (in recursion stack)
    for (int neighbor : adj.getOrDefault(node, new ArrayList<>())) {
        if (color[neighbor] == 1) return true;  // back edge = cycle
        if (color[neighbor] == 0 && dfsCycle(adj, neighbor, color)) return true;
    }
    color[node] = 2;  // black (done)
    return false;
}
```

---

4. Topological Sort (Directed Acyclic Graph)

```
// Kahn's Algorithm (BFS-based) — also detects cycles
List<Integer> topologicalSort(int n, int[][] prerequisites) {
    int[] indegree = new int[n];
    Map<Integer, List<Integer>> adj = new HashMap<>();

    for (int[] pre : prerequisites) {
        adj.computeIfAbsent(pre[1], k -> new ArrayList<>()).add(pre[0]);
        indegree[pre[0]]++;
    }

    Queue<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) queue.offer(i);  // nodes with no dependencies
    }

    List<Integer> order = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int next : adj.getOrDefault(node, new ArrayList<>())) {
            if (--indegree[next] == 0) queue.offer(next);
        }
    }

    return order.size() == n ? order : new ArrayList<>();  // empty if cycle
}

// Course Schedule (LC 207) — can you finish all courses?
boolean canFinish(int numCourses, int[][] prerequisites) {
    return topologicalSort(numCourses, prerequisites).size() == numCourses;
}
```

---

5. Union Find (Disjoint Set Union)

```
class UnionFind {
    private int[] parent, rank;
    private int components;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        components = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);  // path compression
        return parent[x];
    }

    public boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;

        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }

        components--;
        return true;
    }

    public boolean connected(int x, int y) { return find(x) == find(y); }
    public int getComponents() { return components; }
}

// Applications
// - Number of Connected Components
// - Redundant Connection (find cycle)
// - Accounts Merge
// - Making a Large Island
```

6. Dijkstra’s Algorithm (Weighted Shortest Path)

```java
int[] dijkstra(int n, Map<Integer, List<int[]>> adj, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // [node, distance]
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{src, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0], d = curr[1];

        if (d > dist[node]) continue;  // skip outdated entry

        for (int[] edge : adj.getOrDefault(node, new ArrayList<>())) {
            int next = edge[0], weight = edge[1];
            if (dist[node] + weight < dist[next]) {
                dist[next] = dist[node] + weight;
                pq.offer(new int[]{next, dist[next]});
            }
        }
    }
    return dist;
}
// Time: O((V + E) log V)  Space: O(V + E)
```

---

7. Bellman-Ford (Handles Negative Weights)

```java
int[] bellmanFord(int n, int[][] edges, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    // Relax all edges n-1 times
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    // Check for negative cycles
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
            return null;  // negative cycle detected
        }
    }
    return dist;
}
```

---

8. Graph Coloring / Bipartite Check

```
boolean isBipartite(int[][] graph) {
    int n = graph.length;
    int[] color = new int[n];  // 0=uncolored, 1=red, -1=blue

    for (int start = 0; start < n; start++) {
        if (color[start] != 0) continue;

        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(start);
        color[start] = 1;

        while (!queue.isEmpty()) {
            int node = queue.poll();
            for (int neighbor : graph[node]) {
                if (color[neighbor] == 0) {
                    color[neighbor] = -color[node];  // opposite color
                    queue.offer(neighbor);
                } else if (color[neighbor] == color[node]) {
                    return false;  // same color = not bipartite
                }
            }
        }
    }
    return true;
}
```

Algorithm Selection Guide

Scenario	Algorithm	Time
Unweighted shortest path	BFS	O(V+E)
Weighted shortest path (no neg)	Dijkstra	O((V+E)logV)
Weighted shortest path (with neg)	Bellman-Ford	O(VE)
All-pairs shortest path	Floyd-Warshall	$O(V^3)$
Connected components	DFS/BFS/UnionFind	O(V+E)
Topological order	Kahn's/DFS	O(V+E)
Minimum spanning tree	Kruskal/Prim	O(E log E)
Cycle detection (undirected)	UnionFind/DFS	O(V+E)
Cycle detection (directed)	DFS (3-color)	O(V+E)

Pattern 9 — Dynamic Programming

Core Insight

DP solves optimization problems by breaking them into **overlapping sub-problems** and storing results to avoid recomputation.

Two approaches: - **Top-Down (Memoization):** Recursion + cache - **Bottom-Up (Tabulation):** Iterative, fill table from base case

---

DP Recognition Signals

- "Maximum/minimum"
- "Number of ways"
- "Is it possible"
- Problem can be broken into smaller overlapping sub-problems
- Optimal substructure: optimal solution contains optimal sub-solutions

---

1. Knapsack Patterns

0/1 Knapsack

```java
// Given weights and values, maximize value within capacity
// Each item: use 0 or 1 time
int knapsack01(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i][w] = dp[i-1][w];  // don't take item i
            if (weights[i-1] <= w) {
                dp[i][w] = Math.max(dp[i][w],
                           dp[i-1][w - weights[i-1]] + values[i-1]);  // take item i
            }
        }
    }
    return dp[n][capacity];
}

// Space-optimized: O(capacity) space (traverse backwards!)
int knapsack01Optimized(int[] weights, int[] values, int capacity) {
    int[] dp = new int[capacity + 1];

    for (int i = 0; i < weights.length; i++) {
        for (int w = capacity; w >= weights[i]; w--) {  // BACKWARDS to avoid using item twice
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[capacity];
}
```

Unbounded Knapsack (Coin Change Type)

```java
// Each item can be used unlimited times
int unboundedKnapsack(int[] weights, int[] values, int capacity) {
    int[] dp = new int[capacity + 1];

    for (int w = 0; w <= capacity; w++) {
        for (int i = 0; i < weights.length; i++) {  // FORWARD (can reuse)
            if (weights[i] <= w) {
                dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
            }
        }
    }
    return dp[capacity];
}

// Coin Change (LC 322) — minimum coins
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);  // "infinity"
    dp[0] = 0;

    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

// Coin Change II (LC 518) — count ways
int coinChangeWays(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    dp[0] = 1;

    for (int coin : coins) {
        for (int a = coin; a <= amount; a++) {
            dp[a] += dp[a - coin];  // add ways using this coin
        }
    }
    return dp[amount];
}
```

2. Longest Increasing Subsequence (LIS)

```java
// O(n²) DP approach
int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);  // each element is LIS of length 1

    int maxLen = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        maxLen = Math.max(maxLen, dp[i]);
    }
    return maxLen;
}

// O(n log n) patience sorting approach
int lengthOfLIS_NLogN(int[] nums) {
    List<Integer> tails = new ArrayList<>();  // tails[i] = smallest tail of LIS length i+1

    for (int n : nums) {
        int pos = Collections.binarySearch(tails, n);
        if (pos < 0) pos = -(pos + 1);  // insertion point

        if (pos == tails.size()) tails.add(n);  // extend LIS
        else tails.set(pos, n);                 // replace to minimize tail
    }
    return tails.size();
}
```

3. Longest Common Subsequence (LCS)

```java
int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i-1) == text2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[m][n];
}

// Longest Common Substring (contiguous)
int longestCommonSubstring(String s1, String s2) {
    int m = s1.length(), n = s2.length(), maxLen = 0;
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1] + 1;
                maxLen = Math.max(maxLen, dp[i][j]);
            }
        }
    }
    return maxLen;
}

// Edit Distance (LC 72)
int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i-1) == word2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1];
            } else {
```

```
                    dp[i][j] = 1 + Math.min(dp[i-1][j-1],   // replace
                                   Math.min(dp[i-1][j],       // delete
                                            dp[i][j-1]));      // insert
                }
            }
        }
        return dp[m][n];
    }
```

---

4. Partition DP

```java
// Partition Equal Subset Sum (LC 416)
// Can we partition into two subsets with equal sum?
boolean canPartition(int[] nums) {
    int total = Arrays.stream(nums).sum();
    if (total % 2 != 0) return false;

    int target = total / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;

    for (int n : nums) {
        for (int j = target; j >= n; j--) {  // backwards (0/1 knapsack)
            dp[j] = dp[j] || dp[j - n];
        }
    }
    return dp[target];
}

// Word Break (LC 139)
boolean wordBreak(String s, List<String> wordDict) {
    Set<String> dict = new HashSet<>(wordDict);
    int n = s.length();
    boolean[] dp = new boolean[n + 1];
    dp[0] = true;

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[n];
}
```

5. Matrix / Grid DP

```java
// Unique Paths (LC 62)
int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}

// Minimum Path Sum (LC 64)
int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dp = new int[m][n];
    dp[0][0] = grid[0][0];

    for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
    for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m-1][n-1];
}
```

---

6. State Machine DP (Stock Problems)

```java
// Best Time to Buy and Sell Stock with Cooldown (LC 309)
// States: HOLD, SOLD (cooldown), REST
int maxProfitCooldown(int[] prices) {
    int hold = Integer.MIN_VALUE, sold = 0, rest = 0;

    for (int price : prices) {
        int prevHold = hold, prevSold = sold, prevRest = rest;
        hold = Math.max(prevHold, prevRest - price);  // buy (only from rest)
        sold = prevHold + price;                      // sell
        rest = Math.max(prevRest, prevSold);          // wait
    }
    return Math.max(sold, rest);
}
```

DP Template Decision Guide

`1. Is there a clear "last decision" at each step? → State transition DP 2. Does it involve sequences? (arrays,`

> **Interview Tip:** When you identify DP, first write the recurrence relation verbally: “dp[i] represents the [maximum/minimum/count] of [subproblem] using elements 0..i”. Getting the definition right is 80% of the solution.

Pattern 10 — Backtracking

Core Insight

Backtracking = DFS with pruning. Explore all possibilities, but abandon a branch as soon as you know it can’t lead to a solution.

Think of it as: Making choices → Exploring → Undoing choices if they don’t work

---

Universal Backtracking Template

```
void backtrack(result, current, choices, start) {
    // Base case: solution complete
    if (isSolution()) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (choice : choices[start..end]) {
        if (!isValid(choice)) continue;  // pruning

        current.add(choice);              // make choice
        backtrack(result, current, choices, nextStart);
        current.remove(current.size() - 1);  // undo choice (backtrack)
    }
}
```

Problem 1: Subsets (LC 78)

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrackSubsets(nums, 0, new ArrayList<>(), result);
    return result;
}

void backtrackSubsets(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current));  // add at each step (every subset is valid)

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrackSubsets(nums, i + 1, current, result);
        current.remove(current.size() - 1);
    }
}

// Dry run: nums = [1, 2, 3]
// start=0: add [] → explore i=0,1,2
//   current=[1], start=1: add [1] → explore i=1,2
//     current=[1,2], start=2: add [1,2] → explore i=2
//       current=[1,2,3], start=3: add [1,2,3], return
//     current=[1,3], start=3: add [1,3], return
//   current=[2], start=2: add [2] → explore i=2
//     current=[2,3], start=3: add [2,3], return
//   current=[3], start=3: add [3], return
// Result: [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]] = 8 subsets = 2^3
```

Problem 2: Subsets II (with duplicates, LC 90)

```
List<List<Integer>> subsetsWithDup(int[] nums) {
    Arrays.sort(nums);  // MUST sort to group duplicates
    List<List<Integer>> result = new ArrayList<>();
    backtrackSubsetsII(nums, 0, new ArrayList<>(), result);
    return result;
}

void backtrackSubsetsII(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current));

    for (int i = start; i < nums.length; i++) {
        // Skip duplicates at same level
        if (i > start && nums[i] == nums[i - 1]) continue;

        current.add(nums[i]);
        backtrackSubsetsII(nums, i + 1, current, result);
        current.remove(current.size() - 1);
    }
}
```

Problem 3: Permutations (LC 46)

```java
List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrackPermute(nums, new boolean[nums.length], new ArrayList<>(), result);
    return result;
}

void backtrackPermute(int[] nums, boolean[] used, List<Integer> current, List<List<Integer>> result) {
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;  // skip already chosen

        used[i] = true;
        current.add(nums[i]);
        backtrackPermute(nums, used, current, result);
        current.remove(current.size() - 1);
        used[i] = false;
    }
}
// Total permutations: n! (n=3 → 6, n=4 → 24)
```

Problem 4: Combination Sum (LC 39)

```java
// Same number can be used multiple times
List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);  // optional but enables early pruning
    backtrackCombSum(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

void backtrackCombSum(int[] candidates, int remaining, int start,
                      List<Integer> current, List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remaining) break;  // pruning: sorted, no need to continue

        current.add(candidates[i]);
        backtrackCombSum(candidates, remaining - candidates[i], i, current, result);  // i (not i+1) for
        current.remove(current.size() - 1);
    }
}
```

Problem 5: Combination Sum II (with duplicates, LC 40)

```java
// Each number used once, no duplicate combinations
List<List<Integer>> combinationSum2(int[] candidates, int target) {
    Arrays.sort(candidates);
    List<List<Integer>> result = new ArrayList<>();
    backtrackCombSum2(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

void backtrackCombSum2(int[] candidates, int remaining, int start,
                       List<Integer> current, List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remaining) break;
        if (i > start && candidates[i] == candidates[i - 1]) continue;  // skip dups

        current.add(candidates[i]);
        backtrackCombSum2(candidates, remaining - candidates[i], i + 1, current, result);
        current.remove(current.size() - 1);
    }
}
```

Problem 6: N-Queens (LC 51)

```java
List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    int[] queens = new int[n];  // queens[row] = column of queen in that row
    Arrays.fill(queens, -1);

    Set<Integer> cols = new HashSet<>();
    Set<Integer> diag1 = new HashSet<>();  // row - col
    Set<Integer> diag2 = new HashSet<>();  // row + col

    backtrackQueens(n, 0, queens, cols, diag1, diag2, result);
    return result;
}

void backtrackQueens(int n, int row, int[] queens,
                     Set<Integer> cols, Set<Integer> diag1, Set<Integer> diag2,
                     List<List<String>> result) {
    if (row == n) {
        result.add(buildBoard(queens, n));
        return;
    }

    for (int col = 0; col < n; col++) {
        if (cols.contains(col)) continue;
        if (diag1.contains(row - col)) continue;
        if (diag2.contains(row + col)) continue;

        queens[row] = col;
        cols.add(col);
        diag1.add(row - col);
        diag2.add(row + col);

        backtrackQueens(n, row + 1, queens, cols, diag1, diag2, result);

        queens[row] = -1;
        cols.remove(col);
        diag1.remove(row - col);
        diag2.remove(row + col);
    }
}

List<String> buildBoard(int[] queens, int n) {
    List<String> board = new ArrayList<>();
    for (int row = 0; row < n; row++) {
        char[] line = new char[n];
        Arrays.fill(line, '.');
```

```
            line[queens[row]] = 'Q';
            board.add(new String(line));
        }
        return board;
    }
```

Problem 7: Word Search (LC 79)

```
    boolean exist(char[][] board, String word) {
        int rows = board.length, cols = board[0].length;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (dfsWordSearch(board, word, r, c, 0)) return true;
            }
        }
        return false;
    }

    boolean dfsWordSearch(char[][] board, String word, int r, int c, int idx) {
        if (idx == word.length()) return true;
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
        if (board[r][c] != word.charAt(idx)) return false;

        char temp = board[r][c];
        board[r][c] = '#';  // mark visited

        boolean found = dfsWordSearch(board, word, r+1, c, idx+1) ||
                        dfsWordSearch(board, word, r-1, c, idx+1) ||
                        dfsWordSearch(board, word, r, c+1, idx+1) ||
                        dfsWordSearch(board, word, r, c-1, idx+1);

        board[r][c] = temp;  // restore (backtrack)
        return found;
    }
```

Key Differences Between Problems

Problem	Duplicates	Reuse	Order Matters
Subsets	No	No	No
Subsets II	Yes	No	No
Permutations	No	No	Yes
Combination Sum	No	Yes	No
Combination Sum II	Yes	No	No

Pruning Techniques

```
// 1. Early termination: sort + break
if (candidates[i] > remaining) break;

// 2. Skip duplicates at same level
if (i > start && nums[i] == nums[i-1]) continue;

// 3. Mark visited (for grids, permutations)
visited[i] = true; // ... recurse ... visited[i] = false;

// 4. Constraint check before recursion
if (!isValid(choice)) continue;
```

> **Interview Tip:** Backtracking has exponential time complexity (O(2^n) or O(n!)). Always mention this, then say: “Let me see if DP can solve this instead.” Sometimes backtracking IS the intended solution (N-Queens, word search).

Pattern 11 — Heap / Priority Queue

Core Insight

A heap gives you O(1) access to the min/max and O(log n) insert/delete. Perfect for “top K”, “kth element”, “streaming median”, and “greedy with priority”.

Pattern 1: Top K Elements

```java
// Kth Largest Element (LC 215)
// Approach: min-heap of size k — maintains top k largest
int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();  // min at top

    for (int n : nums) {
        minHeap.offer(n);
        if (minHeap.size() > k) minHeap.poll();  // remove smallest
    }
    return minHeap.peek();  // smallest of top-k = kth largest
}
// Time: O(n log k)  Space: O(k)

// Top K Frequent Elements (LC 347)
int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    // Min-heap by frequency (keeps top k most frequent)
    PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        heap.offer(new int[]{e.getKey(), e.getValue()});
        if (heap.size() > k) heap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = heap.poll()[0];
    return result;
}
```

Pattern 2: Merge K Sorted Lists / Arrays

```java
// Merge K Sorted Lists (LC 23)
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> a.val - b.val);

    for (ListNode head : lists) {
        if (head != null) heap.offer(head);
    }

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        curr.next = node;
        curr = curr.next;
        if (node.next != null) heap.offer(node.next);
    }
    return dummy.next;
}
// Time: O(n log k) where n = total nodes, k = number of lists

// Merge K Sorted Arrays
int[] mergeKArrays(int[][] arrays) {
    PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    // [value, arrayIndex, elementIndex]

    for (int i = 0; i < arrays.length; i++) {
        if (arrays[i].length > 0) {
            heap.offer(new int[]{arrays[i][0], i, 0});
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!heap.isEmpty()) {
        int[] curr = heap.poll();
        result.add(curr[0]);
        int arrIdx = curr[1], elemIdx = curr[2];
        if (elemIdx + 1 < arrays[arrIdx].length) {
            heap.offer(new int[]{arrays[arrIdx][elemIdx + 1], arrIdx, elemIdx + 1});
        }
    }
    return result.stream().mapToInt(Integer::intValue).toArray();
}
```

Pattern 3: Find Median from Data Stream (LC 295)

```
// Two heaps: maxHeap for lower half, minHeap for upper half
class MedianFinder {
    private PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    public void addNum(int num) {
        // Step 1: Push to maxHeap
        maxHeap.offer(num);

        // Step 2: Balance: maxHeap top should be <= minHeap top
        if (!minHeap.isEmpty() && maxHeap.peek() > minHeap.peek()) {
            minHeap.offer(maxHeap.poll());
        }

        // Step 3: Rebalance sizes (maxHeap can have at most 1 more)
        if (maxHeap.size() > minHeap.size() + 1) {
            minHeap.offer(maxHeap.poll());
        } else if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() > minHeap.size()) return maxHeap.peek();
        return (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}
// Time: O(log n) per addNum, O(1) for findMedian
```

Pattern 4: Task Scheduler (LC 621)

```
int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char t : tasks) freq[t - 'A']++;

    // Max-heap by frequency
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    for (int f : freq) if (f > 0) maxHeap.offer(f);

    int time = 0;
    Queue<int[]> cooldown = new ArrayDeque<>();  // [freq, available_at]

    while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
        time++;

        if (!maxHeap.isEmpty()) {
            int f = maxHeap.poll() - 1;
            if (f > 0) cooldown.offer(new int[]{f, time + n});
        }

        if (!cooldown.isEmpty() && cooldown.peek()[1] == time) {
            maxHeap.offer(cooldown.poll()[0]);
        }
    }
    return time;
}
```

Pattern 5: Sliding Window with Heap (Kth Largest in Stream)

```java
// Kth Largest Element in a Stream (LC 703)
class KthLargest {
    private final int k;
    private final PriorityQueue<Integer> heap;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        heap = new PriorityQueue<>();  // min-heap
        for (int n : nums) add(n);
    }

    public int add(int val) {
        heap.offer(val);
        if (heap.size() > k) heap.poll();  // remove smallest
        return heap.peek();  // kth largest
    }
}
```

Pattern 6: Dijkstra (Revisited as Heap Pattern)

```java
// The key insight: Dijkstra = BFS with a priority queue
// Always processes the closest unvisited node first
int networkDelayTime(int[][] times, int n, int k) {
    Map<Integer, List<int[]>> adj = new HashMap<>();
    for (int[] t : times) {
        adj.computeIfAbsent(t[0], x -> new ArrayList<>()).add(new int[]{t[1], t[2]});
    }

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{k, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0], d = curr[1];
        if (d > dist[node]) continue;

        for (int[] edge : adj.getOrDefault(node, new ArrayList<>())) {
            int next = edge[0], w = edge[1];
            if (dist[node] + w < dist[next]) {
                dist[next] = dist[node] + w;
                pq.offer(new int[]{next, dist[next]});
            }
        }
    }

    int max = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        max = Math.max(max, dist[i]);
    }
    return max;
}
```

Complexity Summary

Pattern	Time	Space
Top K (n elements, k result)	O(n log k)	O(k)
Merge K lists (n total, k lists)	O(n log k)	O(k)
Add to stream + find median	O(log n) per add	O(n)
Dijkstra (V vertices, E edges)	O((V+E) log V)	O(V+E)

> **Key insight:** min-heap of size k = efficient way to track top-k largest. This pattern appears everywhere: top K frequent, K closest points, K-way merge.

Pattern 12 — Intervals

Core Insight

Interval problems usually require sorting by start time, then deciding whether consecutive intervals overlap.

Overlap condition: `a.end >= b.start` (when a starts before b)

Pattern 1: Merge Intervals (LC 56)

```
int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);  // sort by start time

    List<int[]> merged = new ArrayList<>();
    int[] current = intervals[0];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= current[1]) {
            // Overlap: extend current interval
            current[1] = Math.max(current[1], intervals[i][1]);
        } else {
            // No overlap: save current, start new
            merged.add(current);
            current = intervals[i];
        }
    }
    merged.add(current);

    return merged.toArray(new int[0][]);
}

// Dry run: [[1,3],[2,6],[8,10],[15,18]]
// After sort: same
// current=[1,3], i=1: [2,6] overlaps (2<=3), current=[1,6]
// i=2: [8,10] no overlap (8>6), add [1,6], current=[8,10]
// i=3: [15,18] no overlap, add [8,10], current=[15,18]
// Add [15,18]
// Result: [[1,6],[8,10],[15,18]]
```

Pattern 2: Insert Interval (LC 57)

```
int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0, n = intervals.length;

    // Phase 1: Add all intervals that end before newInterval starts
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i++]);
    }

    // Phase 2: Merge all overlapping intervals
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);

    // Phase 3: Add remaining intervals
    while (i < n) result.add(intervals[i++]);

    return result.toArray(new int[0][]);
}
```

Pattern 3: Meeting Rooms I (LC 252)

```
// Can attend all meetings? (no overlaps allowed)
boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < intervals[i-1][1]) return false;  // overlap
    }
    return true;
}
```

Pattern 4: Meeting Rooms II (LC 253)

```java
// Minimum meeting rooms required
int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    // Min-heap of end times (when will rooms be free)
    PriorityQueue<Integer> endTimes = new PriorityQueue<>();

    for (int[] interval : intervals) {
        if (!endTimes.isEmpty() && endTimes.peek() <= interval[0]) {
            endTimes.poll();  // room is free, reuse it
        }
        endTimes.offer(interval[1]);  // assign/reuse room, note end time
    }
    return endTimes.size();  // rooms in use = answer
}

// Alternative: two sorted arrays (start times, end times)
int minMeetingRoomsAlt(int[][] intervals) {
    int n = intervals.length;
    int[] starts = new int[n], ends = new int[n];

    for (int i = 0; i < n; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, endPtr = 0;
    for (int startPtr = 0; startPtr < n; startPtr++) {
        if (starts[startPtr] < ends[endPtr]) rooms++;  // need new room
        else endPtr++;                                 // room freed
    }
    return rooms;
}
```

Pattern 5: Non-Overlapping Intervals (LC 435) — Greedy

```java
// Minimum intervals to remove so none overlap
int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);  // sort by END time (greedy!)

    int count = 0;
    int prevEnd = Integer.MIN_VALUE;

    for (int[] interval : intervals) {
        if (interval[0] >= prevEnd) {
            prevEnd = interval[1];  // no overlap, keep this interval
        } else {
            count++;  // overlap, remove this interval
        }
    }
    return count;
}
// Key insight: sorting by end time maximizes intervals we can keep (greedy)
```

Pattern 6: Interval List Intersections (LC 986)

```java
int[][] intervalIntersection(int[][] first, int[][] second) {
    List<int[]> result = new ArrayList<>();
    int i = 0, j = 0;

    while (i < first.length && j < second.length) {
        int lo = Math.max(first[i][0], second[j][0]);
        int hi = Math.min(first[i][1], second[j][1]);

        if (lo <= hi) result.add(new int[]{lo, hi});  // intersection exists

        // Move pointer for interval that ends earlier
        if (first[i][1] < second[j][1]) i++;
        else j++;
    }
    return result.toArray(new int[0][]);
}
```

Summary

Problem	Approach	Sort by
Merge Intervals	Greedy merge	Start time
Insert Interval	3-phase scan	Pre-sorted
Meetings I	Check overlap	Start time
Meetings II	Min-heap of ends	Start time
Min Removal	Greedy keep	End time
Intersection	Two pointers	Pre-sorted

Pattern 13 — Greedy

Core Insight

Make the locally optimal choice at each step, trusting it leads to a globally optimal solution.

When greedy works: The problem has optimal substructure AND the greedy choice property (local optimum → global optimum).

Prove it works: Usually by contradiction — show that any deviation from greedy can't improve the result.

Pattern 1: Jump Game (LC 55)

```java
// Can you reach the last index?
boolean canJump(int[] nums) {
    int maxReach = 0;

    for (int i = 0; i <= maxReach && i < nums.length; i++) {
        maxReach = Math.max(maxReach, i + nums[i]);
        if (maxReach >= nums.length - 1) return true;
    }
    return false;
}

// Jump Game II (LC 45) — minimum jumps to reach end
int jump(int[] nums) {
    int jumps = 0, curEnd = 0, farthest = 0;

    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);

        if (i == curEnd) {  // must jump here
            jumps++;
            curEnd = farthest;
        }
    }
    return jumps;
}
```

Pattern 2: Gas Station (LC 134)

```java
// Find starting station for circular tour
int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, currentGas = 0, start = 0;

    for (int i = 0; i < gas.length; i++) {
        int net = gas[i] - cost[i];
        totalGas += net;
        currentGas += net;

        if (currentGas < 0) {
            start = i + 1;  // can't start from anywhere before i+1
            currentGas = 0;
        }
    }
    return totalGas >= 0 ? start : -1;
}
```

Pattern 3: Activity Selection / Maximize Non-Overlapping Intervals

```java
// Maximum number of non-overlapping intervals (same as greedy for meeting rooms)
int maxNonOverlapping(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);  // sort by end time

    int count = 1, prevEnd = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= prevEnd) {
            count++;
            prevEnd = intervals[i][1];
        }
    }
    return count;
}
```

Pattern 4: Greedy String Problems

```java
// Assign Cookies (LC 455)
int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);  // greed factors
    Arrays.sort(s);  // cookie sizes

    int child = 0, cookie = 0;
    while (child < g.length && cookie < s.length) {
        if (s[cookie] >= g[child]) child++;  // satisfied
        cookie++;
    }
    return child;
}

// Lemonade Change (LC 860)
boolean lemonadeChange(int[] bills) {
    int five = 0, ten = 0;
    for (int bill : bills) {
        if (bill == 5) {
            five++;
        } else if (bill == 10) {
            if (five == 0) return false;
            five--; ten++;
        } else {  // bill == 20
            // Prefer to give 10+5 (saves 5s for more utility)
            if (ten > 0 && five > 0) { ten--; five--; }
            else if (five >= 3) { five -= 3; }
            else return false;
        }
    }
    return true;
}
```

Pattern 5: Partition Labels (LC 763)

```java
// Partition string so each character appears in at most one part
List<Integer> partitionLabels(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;

    List<Integer> result = new ArrayList<>();
    int start = 0, end = 0;

    for (int i = 0; i < s.length(); i++) {
        end = Math.max(end, last[s.charAt(i) - 'a']);  // extend partition
        if (i == end) {
            result.add(end - start + 1);
            start = i + 1;
        }
    }
    return result;
}
```

Summary: When to Use Greedy

Pattern	Greedy Choice
Jump game	Always extend max reach
Activity selection	Pick earliest ending
Gas station	Skip failing starts
Partition labels	Extend to last occurrence
Assign cookies	Match smallest cookie that satisfies

> **Interview Tip:** Greedy is tricky to prove correct. When using it, say: “I’ll use a greedy approach — at each step I’ll [describe choice] because [brief justification].” Even partial proof earns credit.

Pattern 14 — Trie

Core Insight

A Trie (prefix tree) stores strings where common prefixes share nodes. Enables O(m) prefix search where m = word length, regardless of dictionary size.

Trie Implementation

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd = false;
    int count = 0;  // optional: words passing through this node
}

class Trie {
    private TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
            curr.count++;
        }
        curr.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = findNode(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return findNode(prefix) != null;
    }

    private TrieNode findNode(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) return null;
            curr = curr.children[idx];
        }
        return curr;
    }
}
```

Autocomplete / Word Suggestions

```
// Find all words with given prefix
List<String> autocomplete(String prefix) {
    TrieNode node = findNode(prefix);
    List<String> results = new ArrayList<>();
    if (node != null) dfsCollect(node, new StringBuilder(prefix), results);
    return results;
}

void dfsCollect(TrieNode node, StringBuilder curr, List<String> results) {
    if (node.isEnd) results.add(curr.toString());
    for (int i = 0; i < 26; i++) {
        if (node.children[i] != null) {
            curr.append((char)('a' + i));
            dfsCollect(node.children[i], curr, results);
            curr.deleteCharAt(curr.length() - 1);
        }
    }
}
```

Word Search II (LC 212) — Trie + Backtracking

```
List<String> findWords(char[][] board, String[] words) {
    Trie trie = new Trie();
    for (String w : words) trie.insert(w);

    Set<String> result = new HashSet<>();
    int rows = board.length, cols = board[0].length;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            dfsWordSearch(board, r, c, trie.root, new StringBuilder(), result);
        }
    }
    return new ArrayList<>(result);
}

void dfsWordSearch(char[][] board, int r, int c, TrieNode node,
                   StringBuilder curr, Set<String> result) {
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return;
    char ch = board[r][c];
    if (ch == '#' || node.children[ch - 'a'] == null) return;

    curr.append(ch);
    node = node.children[ch - 'a'];
    if (node.isEnd) result.add(curr.toString());

    board[r][c] = '#';
    dfsWordSearch(board, r+1, c, node, curr, result);
    dfsWordSearch(board, r-1, c, node, curr, result);
    dfsWordSearch(board, r, c+1, node, curr, result);
    dfsWordSearch(board, r, c-1, node, curr, result);
    board[r][c] = ch;
    curr.deleteCharAt(curr.length() - 1);
}
```

XOR Trie (Maximum XOR)

```
// Maximum XOR of two numbers in array (LC 421)
// Store numbers as 32-bit binary in a Trie
class XORTrie {
    int[][] children = new int[32 * 100000][2];
    int idx = 1;

    void insert(int num) {
        int curr = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (children[curr][bit] == 0) children[curr][bit] = idx++;
            curr = children[curr][bit];
        }
    }

    int maxXOR(int num) {
        int curr = 0, xor = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            int want = 1 - bit;  // prefer opposite bit for max XOR
            if (children[curr][want] != 0) {
                xor |= (1 << i);
                curr = children[curr][want];
            } else {
                curr = children[curr][bit];
            }
        }
        return xor;
    }
}

int findMaximumXOR(int[] nums) {
    XORTrie trie = new XORTrie();
    for (int n : nums) trie.insert(n);
    int max = 0;
    for (int n : nums) max = Math.max(max, trie.maxXOR(n));
    return max;
}
```

Complexity

Operation	Time	Space
Insert word of length m	O(m)	O(m)
Search word of length m	O(m)	O(1)
Prefix check of length m	O(m)	O(1)
Total space for n words	—	O(n * m)

Pattern 15 — Bit Manipulation

Core Tricks

```
// Bit operations reference
n & 1        // last bit (0=even, 1=odd)
n >> 1       // divide by 2
n << 1       // multiply by 2
n & (n-1)    // clear lowest set bit
n | (1<<k)   // set kth bit
n & ~(1<<k)  // clear kth bit
n ^ (1<<k)   // toggle kth bit
(n >> k) & 1 // get kth bit value
Integer.bitCount(n)  // count set bits
```

XOR Tricks

```java
// Single Number (LC 136) — find the one non-duplicate
// XOR: same ^ same = 0, any ^ 0 = any
int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}

// Single Number II (appears once, rest 3 times) — bit counting
int singleNumberII(int[] nums) {
    int result = 0;
    for (int i = 0; i < 32; i++) {
        int sum = 0;
        for (int n : nums) sum += (n >> i) & 1;
        result |= (sum % 3) << i;
    }
    return result;
}

// Find two single numbers (rest appear twice) — LC 260
int[] singleNumberIII(int[] nums) {
    int xor = 0;
    for (int n : nums) xor ^= n;

    // Find rightmost bit that differs between the two numbers
    int diffBit = xor & (-xor);  // lowest set bit

    int a = 0, b = 0;
    for (int n : nums) {
        if ((n & diffBit) != 0) a ^= n;
        else b ^= n;
    }
    return new int[]{a, b};
}
```

Missing and Duplicate Numbers

```java
// Missing Number (LC 268) — [0, n] with one missing
int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) xor ^= i ^ nums[i];
    return xor;
}

// Missing Number (sum approach)
int missingNumberSum(int[] nums) {
    int n = nums.length;
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int n2 : nums) actual += n2;
    return expected - actual;
}
```

Power of 2 Checks

```java
boolean isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
boolean isPowerOfFour(int n) {
    // Power of 4: power of 2 AND in odd bit positions (0x55555555)
    return n > 0 && (n & (n - 1)) == 0 && (n & 0x55555555) != 0;
}
```

Counting Bits (LC 338)

```java
int[] countBits(int n) {
    int[] dp = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i >> 1] + (i & 1);
        // i >> 1 = i/2 (same bits except last), i & 1 = last bit
    }
    return dp;
}
```

Bitmask DP

```java
// Traveling Salesman Problem (small n)
// State: visited cities as bitmask
int tsp(int[][] dist, int n) {
    int states = 1 << n;
    int[][] dp = new int[states][n];
    for (int[] row : dp) Arrays.fill(row, Integer.MAX_VALUE / 2);
    dp[1][0] = 0;  // start at city 0

    for (int mask = 1; mask < states; mask++) {
        for (int last = 0; last < n; last++) {
            if ((mask & (1 << last)) == 0) continue;
            for (int next = 0; next < n; next++) {
                if ((mask & (1 << next)) != 0) continue;
                int newMask = mask | (1 << next);
                dp[newMask][next] = Math.min(dp[newMask][next],
                                             dp[mask][last] + dist[last][next]);
            }
        }
    }

    int result = Integer.MAX_VALUE;
    for (int last = 1; last < n; last++) {
        result = Math.min(result, dp[states - 1][last] + dist[last][0]);
    }
    return result;
}
```

Complexity Summary

Bit Operation	Time
Single number XOR	O(n)
Count set bits	O(log n) or O(1) with `Integer.bitCount`
Power of 2 check	O(1)
Bitmask DP (2^n states)	O(2^n * n)

Interview Tip: Bit manipulation problems often have elegant O(n) solutions. When you see “duplicate/missing in array” or “appear once vs twice/thrice”, think XOR first.

Section 5.1 — FAANG Interview Mindset and Communication

The FAANG Interview Reality

FAANG interviews are NOT about finding perfect developers. They're about finding people who: 1. **Think clearly under pressure** 2. **Communicate their reasoning well** 3. **Handle ambiguity professionally** 4. **Show problem-solving growth in real-time**

> You can solve 7/10 LeetCode problems and still fail if you don't communicate. You can solve 5/10 and pass if you communicate extremely well.

The Interviewer's Scorecard

Interviewers typically evaluate:

Dimension	What They Watch
Problem Solving	Do you find the right approach? Can you optimize?
Coding Quality	Clean, readable, bug-free code?
Communication	Do you explain your thinking?
Edge Case Handling	Do you consider boundaries?
Testing	Do you verify your solution?
Collaboration	Do you incorporate hints? Do you ask good questions?

The 5-Minute Framework (Every Problem)

```
1. UNDERSTAND (2 min) - Restate the problem in your own words - Ask clarifying questions - Discuss examples +
```

Clarifying Questions to Always Ask

```
Before coding anything, ask: CONSTRAINTS: - "What's the size of the input? (n = ?)" - "Are the numbers always
```

Communication Scripts

Starting a Problem

> “Let me make sure I understand the problem. You’re asking me to [restate]. The input is [describe] and I should return [describe]. Let me think about edge cases first — what if the array is empty? What if all elements are the same?”

Presenting Brute Force First

> “My first instinct is a brute force approach: [explain]. This would be $O(n^2)$ time and $O(1)$ space. Before I code this, let me think if there’s a better way…”

Transitioning to Optimized

> “I think I can improve this. If I [use a HashMap / sort the array / use two pointers], I can reduce this to $O(n)$ time. The key insight is [explain the insight]. Does this approach make sense?”

When Stuck

> “I’m working through a few approaches. I know the brute force is $O(n^2)$. I’m trying to figure out how to get this to $O(n)$. Can I think out loud for a moment?”

When Given a Hint

> “Ah, that’s a great point. So if I [incorporate their hint], then [continue reasoning]. That would change the complexity to… let me trace through this…”

Before Coding

> “I think I have a solid approach. Let me describe it one more time before I code: [describe algorithm]. Time complexity would be $O([?])$, space $O([?])$. Does this look good to you?”

During Coding

> “I’m initializing the HashMap to store [what]…”
> “This loop iterates through [what] and for each element…”
> “I’m handling the edge case here where [what]…”

After Coding

> “Let me trace through the example. With input [example]: [walk through step by step]. The output is [expected]. Now let me check edge cases: what if the array is empty? [handle] What if all elements are negative? [handle]...”

Common Mistakes and How to Avoid Them

Mistake 1: Coding Without Planning

`BAD: [Interviewer gives problem] → [You immediately start coding] GOOD: [Interviewer gives problem] → [2 min u`

Mistake 2: Silent Coding

`BAD: [Silent for 15 minutes while coding] GOOD: "I'm creating a HashMap here to store frequencies..." "This co`

Mistake 3: Ignoring Edge Cases

`BAD: "Here's my solution." [done] GOOD: "My solution works for the main case. Let me now handle: - Empty array`

Mistake 4: Wrong Complexity Analysis

`BAD: "This is O(n)." [when it's O(n log n) due to sorting] GOOD: "Time: O(n log n) for the sort, plus O(n) for`

Mistake 5: Abandoning Your Approach When Stuck

`BAD: "That approach isn't working. Let me try something completely different." GOOD: "I'm stuck on this part.`

Whiteboard / Online Editor Tips

Variable Naming

```
// BAD
int x = 0, y = 0, z = -1;
Map<Integer, Integer> m = new HashMap<>();

// GOOD
int left = 0, right = n - 1, maxArea = -1;
Map<Integer, Integer> numToIndex = new HashMap<>();
```

Write helper comments BEFORE writing code

```
// Two pointer approach
// left: start of window
// right: end of window
// maxLen: answer
int left = 0, maxLen = 0;
```

Handle null first

```
if (root == null) return 0;
if (nums == null || nums.length == 0) return -1;
```

The “No Optimal Solution Found” Strategy

If you can’t find the optimal solution, do this:

1. **Implement brute force** — show you can code correctly
2. **Analyze brute force** — explain why it’s slow
3. **Identify the bottleneck** — “the slow part is this nested loop”
4. **Suggest improvements** — “if I used a HashMap here, I could get O(1) lookup”
5. **Partially optimize** — even partial optimization shows thinking

A working brute force + intelligent discussion > stuck on optimization + nothing coded.

Handling Behavioral/System Questions After DSA

Many Big Tech rounds combine DSA + behavioral questions. Bridge your backend experience:

> “In my 5 years of backend engineering, I’ve used similar optimization thinking. For example, when we were experiencing Redis cache misses at scale, I used the same frequency-based analysis we see in this top-K problem — we identified the top 5% of keys that accounted for 80% of cache misses and implemented tiered caching...”

This demonstrates **engineering judgment**, not just algorithmic knowledge.

Section 5.2 — Problem-Solving Strategy

Pattern Recognition Framework

When you see a new problem, scan for these signals in order:

```
STEP 1: Read the problem statement carefully STEP 2: Check the constraints (n = ?) STEP 3: Look for pattern ke
```

Pattern Signal Keywords

Keywords	Pattern
“subarray/substring with constraint”, “window of size k”	**Sliding Window**
“sorted array”, “palindrome”, “two numbers that sum to”	**Two Pointers**
“sorted/rotated”, “find minimum”, “search in X”, “binary answer”	**Binary Search**
“range sum”, “sum of subarray = k”, “number of subarrays”	**Prefix Sum**
“two numbers sum to target”, “duplicates”, “frequency”, “group by”	**HashMap/HashSet**
“next greater”, “histogram”, “brackets”, “expression evaluation”	**Stack**
“binary tree”, “path sum”, “level order”, “LCA”	**Tree DFS/BFS**
“graph”, “connected components”, “shortest path”, “detect cycle”	**Graph**
“max/min with choices”, “ways to do X”, “optimal subsequence”	**Dynamic Programming**
“all subsets”, “all permutations”, “generate all X”	**Backtracking**
“top K”, “kth largest/smallest”, “merge K sorted”	**Heap**
“overlapping intervals”, “meeting rooms”, “merge ranges”	**Intervals**
“minimum jumps”, “make locally best choice”	**Greedy**
“prefix search”, “autocomplete”, “word dictionary”	**Trie**
“XOR of duplicates”, “missing number”, “power of 2”	**Bit Manipulation**

Decision Tree for Optimization

```
Input is sorted array? ├── YES: Binary Search or Two Pointers └── NO: Looking for subarray? ├── YES: Sliding W
```

Complexity-Driven Pattern Selection

When you know the required complexity (from constraints):

Required Time	Try These Patterns
O(log n)	Binary Search, Balanced BST
O(n)	Sliding Window, Two Pointers, Prefix Sum, HashMap
O(n log n)	Sorting + scan, Heap, Tree
O(n²)	DP (2D), Nested loops (if acceptable)
O(2^n)	Backtracking, Bitmask DP (small n)

Common Optimizations to Mention

```
From O(n²) to O(n): - Nested loops → Sliding window or two pointers - Linear search per element → Precompute w
```

Time Management in Interviews

```
45-minute interview structure: - 0-2 min: Clarify the problem - 2-5 min: Discuss approach, verify with intervi
```

Testing Strategy

```
// Always test in this order:
// 1. The provided example
// 2. Edge case: empty input
// 3. Edge case: single element
// 4. Edge case: all same elements
// 5. Edge case: negative numbers (if applicable)
// 6. Edge case: maximum size (does it overflow?)

// Example test walk-through:
// Problem: Two Sum, nums=[2,7,11,15], target=9
// Expected: [0,1] (nums[0]+nums[1]=9)

// Walk through:
// i=0: complement=9-2=7, seen={}, not found, add {2:0}
// i=1: complement=9-7=2, seen={2:0}, FOUND at index 0
// Return [0, 1]

// Edge cases:
// nums=[], target=0 → Should return [] (empty)
// nums=[3], target=6 → Should return [] (can't use same element twice)
// nums=[3,3], target=6 → Should return [0,1]
```

Company-Specific Behavioral Anchors

Google / Alphabet

- They value: **Clarity of thought**, can you arrive at optimal solution cleanly?
- Key soft skill: Googleyness — collaborative, intellectually humble
- Often ask: System design + algorithm in same round

Amazon

- They value: **Leadership Principles** — Bias for Action, Dive Deep, Customer Obsession
- DSA focus: Medium/Hard graphs, DP, OOP design
- Key: Connect your solution to real-world implications

Microsoft

- They value: **Collaboration and growth mindset**
- DSA focus: Trees, strings, OOP, and some system design
- Key: Clean, well-structured code. Follow-up questions on tradeoffs.

Meta (Facebook)

- They value: **Speed and correctness**
- DSA focus: Graph problems, dynamic programming, trees
- Key: Finish coding quickly, handle edge cases, optimize

Goldman Sachs / JP Morgan / Morgan Stanley

- They value: **Problem-solving under constraints** + financial reasoning
- Technical interviews similar to Big Tech but may include:
  - Order book simulation (Priority Queue)
  - Rate limiting algorithms
  - Cache invalidation
  - Stream processing (similar to Kafka patterns)
- Key: Show you understand system constraints and tradeoffs

Atlassian / Uber / Airbnb

- They value: **Working code + good engineering judgment**
- DSA focus: Similar to Big Tech but sometimes more practical
- Key: Code reviews, explaining decisions, test coverage thinking

---

Section 6.1 — 1 Month Daily Roadmap

> **Context:** You have 5 years of backend experience. You don't need to learn fundamentals from scratch. This plan maximizes ROI for DSA interview preparation.

Overview

Week	Focus	Goal
Week 1	Java + Collections + Foundations	Get Java fluent, understand Big O
Week 2	Patterns 1-7 (Arrays, Trees)	Master core patterns
Week 3	Patterns 8-15 (Graphs, DP)	Master advanced patterns
Week 4	Mock interviews + Revision	Solidify and perform under pressure

---

Week 1 — Java Fluency + DSA Foundations

Day 1 — Java Syntax Review

```
Morning (1h): Read Section 1.1-1.2 (syntax, operators, loops) Practice (1h): Write 10 Java programs: factorial
```

Day 2 — Java OOP + Generics

```
Morning (1h): Read Section 1.4-1.5 (OOP, Generics, Lambdas) Practice (1h): Implement: LinkedList, Stack, Queue
```

Day 3 — Collections Deep Dive

```
Morning (1h): Read Section 2.1-2.3 (ArrayList, HashMap, HashSet) Practice (1h): Implement frequency counter, t
```

Day 4 — Queue, Stack, PriorityQueue

```
Morning (1h): Read Section 2.4-2.5 (Queue, Stack, Deque, PQ) Practice (1h): Implement LRU Cache, Min Stack Lee
```

Day 5 — Big O + Recursion

```
Morning (1h): Read Section 3.1-3.2 (Big O, Recursion) Practice (1h): Analyze complexity of 10 code snippets Le
```

Day 6 — Sorting + Binary Search Basics

```
Morning (1h): Read Section 3.3 (Sorting, Searching) Practice (1h): Implement merge sort, quick sort, binary se
```

Day 7 — Week 1 Review

```
Morning (2h): Redo problems you got wrong this week Afternoon (2h): Solve 4 Easy problems timed (30 min each)
```

Week 2 — Core Patterns (Array, String, Tree)

Day 8 — Sliding Window

```
Study (1h): Read Pattern 1 (Sliding Window) LeetCode (3h): - 643: Maximum Average Subarray I (Easy) - 3: Longe
```

Day 9 — Two Pointers

```
Study (1h): Read Pattern 2 (Two Pointers) LeetCode (3h): - 125: Valid Palindrome (Easy) - 167: Two Sum II (Med
```

Day 10 — Binary Search Advanced

```
Study (1h): Read Pattern 3 (Binary Search) LeetCode (3h): - 33: Search in Rotated Sorted Array (Medium) - 153:
```

Day 11 — Prefix Sum + HashMap

```
Study (1h): Read Patterns 4 & 5 LeetCode (3h): - 303: Range Sum Query - Immutable (Easy) - 560: Subarray Sum E
```

Day 12 — Stack Patterns

```
Study (1h): Read Pattern 6 (Stack) LeetCode (3h): - 739: Daily Temperatures (Medium) - 496: Next Greater Eleme
```

Day 13 — Tree Fundamentals

```
Study (1h): Read Pattern 7 (Trees) — traversals, height, LCA LeetCode (3h): - 104: Maximum Depth of Binary Tre
```

Day 14 — BST + Tree Advanced

```
Study (1h): BST problems, serialization LeetCode (3h): - 98: Validate Binary Search Tree (Medium) - 230: Kth S
```

Week 3 — Advanced Patterns (Graph, DP)

Day 15 — Graph BFS/DFS

```
Study (1h): Read Pattern 8 (Graphs) — BFS, DFS, components LeetCode (3h): - 200: Number of Islands (Medium) -
```

Day 16 — Graph: Cycle + Topological Sort

```
Study (1h): Cycle detection, Topological sort LeetCode (3h): - 207: Course Schedule (Medium) - 210: Course Sch
```

Day 17 — Union Find + Shortest Path

```
Study (1h): Union Find, Dijkstra LeetCode (3h): - 547: Number of Provinces (Medium) — Union Find - 743: Networ
```

Day 18 — DP Fundamentals

```
Study (1h): Read Pattern 9 — 1D DP, memoization LeetCode (3h): - 198: House Robber (Medium) - 213: House Robbe
```

Day 19 — DP: 2D and LCS

```
Study (1h): 2D DP, LCS, Edit Distance LeetCode (3h): - 1143: Longest Common Subsequence (Medium) - 72: Edit Di
```

Day 20 — Backtracking

```
Study (1h): Read Pattern 10 (Backtracking) LeetCode (3h): - 78: Subsets (Medium) - 46: Permutations (Medium) -
```

Day 21 — Heap + Intervals

```
Study (1h): Read Patterns 11 & 12 LeetCode (3h): - 215: Kth Largest Element in Array (Medium) - 347: Top K Fre
```

Day 22-23 — Greedy + Trie + Bit Manipulation

```
Study (1h/day): Read Patterns 13, 14, 15 LeetCode (3h/day): Day 22: - 55: Jump Game (Medium) - 45: Jump Game I
```

Week 4 — Mock Interviews + Revision

Days 24-28 — Mock Interview Practice

```
DAILY STRUCTURE: Morning (1h): Revise 5-10 problems you struggled with Mock (2h): 2 timed problems (45 min eac
```

Days 29-30 — Final Revision Sprint

```
Day 29: Morning: Review all 15 pattern cheat sheets Afternoon: Solve 5 random medium problems Evening: Review
```

Daily Metrics to Track

Metric	Target
Problems solved per day	3-5
Problems solved in first attempt	>50% by week 4
Pattern recognition time	< 2 min by week 4
Time to working solution	< 30 min for medium by week 4
Complexity explanation accuracy	100%

Priority Problems by Company

Google

1. 56 Merge Intervals, 127 Word Ladder, 297 Serialize Tree
2. 72 Edit Distance, 84 Histogram, 42 Trapping Rain Water
3. 23 Merge K Lists, 239 Sliding Window Maximum

Amazon

1. 1 Two Sum, 200 Number of Islands, 347 Top K Frequent
2. 206 Reverse List, 49 Group Anagrams, 238 Product Except Self
3. 253 Meeting Rooms II, 146 LRU Cache, 297 Serialize Tree

Meta

1. 1570 Dot Product, 273 Integer to English Words, 415 Add Strings
2. 721 Accounts Merge, 314 Binary Tree Vertical Order, 560 Subarray Sum
3. 124 Binary Tree Max Path Sum, 23 Merge K Lists

Morgan Stanley / Goldman Sachs / JP Morgan

1. 146 LRU Cache, 295 Find Median Stream, 347 Top K Frequent

2. Priority Queue simulation, 871 Minimum Refueling Stops
3. 239 Sliding Window Maximum, 768 Max Chunks to Make Sorted

Section 6.2 — High-ROI LeetCode Problem List

> Curated for 5-year experienced engineers targeting FAANG/Big Tech/Banks.
> These 80 problems cover 90% of interview patterns.

Tier 1 — Must Solve (40 problems)

These appear most frequently across Google, Amazon, Meta, Microsoft, Goldman Sachs.

#	Title	Pattern	Difficulty
1	Two Sum	HashMap	Easy
2	Add Two Numbers	Linked List	Medium
3	Longest Substring Without Repeating	Sliding Window	Medium
15	3Sum	Two Pointers	Medium
20	Valid Parentheses	Stack	Easy
21	Merge Two Sorted Lists	Linked List	Easy
33	Search in Rotated Sorted Array	Binary Search	Medium
42	Trapping Rain Water	Two Pointers/Stack	Hard
46	Permutations	Backtracking	Medium
49	Group Anagrams	HashMap	Medium
56	Merge Intervals	Intervals	Medium
70	Climbing Stairs	DP	Easy
72	Edit Distance	DP (LCS)	Hard
76	Minimum Window Substring	Sliding Window	Hard
84	Largest Rectangle in Histogram	Monotonic Stack	Hard
98	Validate Binary Search Tree	Tree/BST	Medium
102	Binary Tree Level Order Traversal	Tree BFS	Medium
104	Maximum Depth of Binary Tree	Tree DFS	Easy
121	Best Time to Buy/Sell Stock	Greedy/DP	Easy
124	Binary Tree Maximum Path Sum	Tree DFS	Hard
128	Longest Consecutive Sequence	HashSet	Medium
141	Linked List Cycle	Fast/Slow Pointers	Easy
146	LRU Cache	HashMap + DLL	Medium
153	Find Min in Rotated Array	Binary Search	Medium
198	House Robber	DP	Medium
200	Number of Islands	Graph BFS/DFS	Medium
206	Reverse Linked List	Linked List	Easy
207	Course Schedule	Graph/Topological Sort	Medium
215	Kth Largest Element	Heap	Medium

#	Title	Pattern	Difficulty
226	Invert Binary Tree	Tree	Easy
230	Kth Smallest in BST	BST	Medium
236	LCA of Binary Tree	Tree	Medium
238	Product of Array Except Self	Prefix Sum	Medium
253	Meeting Rooms II	Intervals + Heap	Medium
295	Find Median from Data Stream	Two Heaps	Hard
297	Serialize/Deserialize Binary Tree	Tree	Hard
300	Longest Increasing Subsequence	DP	Medium
322	Coin Change	DP (Knapsack)	Medium
347	Top K Frequent Elements	Heap + HashMap	Medium
560	Subarray Sum Equals K	Prefix Sum + HashMap	Medium

Tier 2 — High Value (25 problems)

#	Title	Pattern	Difficulty
11	Container With Most Water	Two Pointers	Medium
23	Merge K Sorted Lists	Heap	Hard
39	Combination Sum	Backtracking	Medium
51	N-Queens	Backtracking	Hard
54	Spiral Matrix	Array Simulation	Medium
78	Subsets	Backtracking	Medium
127	Word Ladder	BFS	Hard
139	Word Break	DP	Medium
142	Linked List Cycle II	Fast/Slow Pointers	Medium
152	Maximum Product Subarray	DP	Medium
208	Implement Trie	Trie	Medium
212	Word Search II	Trie + Backtracking	Hard
239	Sliding Window Maximum	Monotonic Deque	Hard
287	Find the Duplicate Number	Fast/Slow Pointers	Medium
310	Minimum Height Trees	Graph BFS	Medium
337	House Robber III	Tree DP	Medium
416	Partition Equal Subset Sum	DP (Knapsack)	Medium
435	Non-Overlapping Intervals	Greedy + Intervals	Medium
518	Coin Change II	DP (Unbounded Knapsack)	Medium
543	Diameter of Binary Tree	Tree	Easy
547	Number of Provinces	Union Find	Medium
621	Task Scheduler	Heap + Greedy	Medium
739	Daily Temperatures	Monotonic Stack	Medium
875	Koko Eating Bananas	Binary Search on Answer	Medium
994	Rotting Oranges	Multi-source BFS	Medium

Tier 3 — Pattern Completers (15 problems)

#	Title	Pattern	Difficulty
45	Jump Game II	Greedy	Medium
55	Jump Game	Greedy	Medium
136	Single Number	Bit Manipulation	Easy
162	Find Peak Element	Binary Search	Medium
210	Course Schedule II	Topological Sort	Medium
309	Best Time to Buy Stock with Cooldown	DP State Machine	Medium
338	Counting Bits	Bit Manipulation/DP	Easy
371	Sum of Two Integers (no +)	Bit Manipulation	Medium
421	Maximum XOR	Trie (XOR)	Medium
503	Next Greater Element II	Monotonic Stack	Medium
684	Redundant Connection	Union Find	Medium
743	Network Delay Time	Dijkstra	Medium
785	Is Graph Bipartite	Graph Coloring	Medium
1011	Capacity to Ship Packages	Binary Search on Answer	Medium
1143	Longest Common Subsequence	DP (LCS)	Medium

Study Approach Per Problem

```
For each problem, practice in this order: 1. READ (5 min) - Understand constraints - Identify pattern - Write
```

Spaced Repetition Schedule

```
Day 1: Solve problem Day 2: Review approach (don't re-code) Day 7: Re-solve without hints Day 21: Re-solve und
```

Problem Difficulty Progression

```
Week 1: 80% Easy, 20% Medium Week 2: 20% Easy, 60% Medium, 20% Hard Week 3: 0% Easy, 50% Medium, 50% Hard Week
```

Complexity Cheat Sheet

Time Complexity Quick Reference

Complexity	Name	Example
O(1)	Constant	HashMap get/put, array index access
O(log n)	Logarithmic	Binary search, TreeMap operations
O(n)	Linear	Single loop, HashMap build
O(n log n)	Linearithmic	Sorting, heap build+extractions
O(n²)	Quadratic	Nested loops, bubble sort
O(n³)	Cubic	3 nested loops, naive matrix multiply
O(2^n)	Exponential	Subset generation, naive recursion
O(n!)	Factorial	Permutation generation

Java Collections Complexity

Collection	Add	Remove	Get/Contains	Iterate
ArrayList	O(1)*	O(n)	O(1)	O(n)
LinkedList	O(1)	O(1)**	O(n)	O(n)
HashMap	O(1)*	O(1)*	O(1)*	O(n)
TreeMap	O(log n)	O(log n)	O(log n)	O(n)
HashSet	O(1)*	O(1)*	O(1)*	O(n)
TreeSet	O(log n)	O(log n)	O(log n)	O(n)
PriorityQueue	O(log n)	O(log n)***	O(1) peek	O(n)
ArrayDeque	O(1)	O(1)	O(1)	O(n)

Amortized average case **At head/tail** O(n) for arbitrary element

Algorithm Complexity

Algorithm	Best	Average	Worst	Space
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
BFS/DFS	O(V+E)	O(V+E)	O(V+E)	O(V)
Dijkstra	—	O((V+E)logV)	—	O(V+E)
Bellman-Ford	O(VE)	O(VE)	O(VE)	O(V)

Space Complexity Patterns

Pattern	Space
Iterative with variables	O(1)
Recursion depth d	O(d)
Recursion depth log n	O(log n)
Array of size n	O(n)
2D array n×n	O(n²)
HashMap/HashSet of n items	O(n)

Java Collections Quick Reference

Initialization One-Liners

```java
// Lists
List<Integer> list = new ArrayList<>();
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
List<Integer> list = List.of(1, 2, 3);  // immutable
List<Integer> list = Collections.nCopies(n, 0);

// Maps
Map<Integer, Integer> map = new HashMap<>();
Map<Integer, List<Integer>> adj = new HashMap<>();
Map<Character, Integer> freq = new HashMap<>();

// Sets
Set<Integer> set = new HashSet<>();
Set<Integer> set = new HashSet<>(Arrays.asList(1, 2, 3));

// Stack / Queue
Deque<Integer> stack = new ArrayDeque<>();
Queue<Integer> queue = new ArrayDeque<>();
Deque<Integer> deque = new ArrayDeque<>();

// Priority Queues
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> pqBySecond = new PriorityQueue<>((a,b) -> a[1]-b[1]);
```

Common 1-Liners for DSA

```java
// Frequency map
Map<T, Integer> freq = new HashMap<>();
for (T item : arr) freq.merge(item, 1, Integer::sum);

// Sort descending
Arrays.sort(arr, Collections.reverseOrder()); // Integer[] only
list.sort(Comparator.reverseOrder());

// Max/min in array
int max = Arrays.stream(arr).max().getAsInt();
int min = IntStream.of(arr).min().getAsInt();

// Sum of array
int sum = IntStream.of(arr).sum();

// int[] to List<Integer>
List<Integer> list = IntStream.of(arr).boxed().collect(Collectors.toList());

// List<Integer> to int[]
int[] arr = list.stream().mapToInt(Integer::intValue).toArray();

// Prefix sum
int[] prefix = new int[n+1];
for(int i=0;i<n;i++) prefix[i+1]=prefix[i]+arr[i];

// Fill 2D array
int[][] dp = new int[m][n];
for(int[] row : dp) Arrays.fill(row, -1);
```

Stack / Queue API

```
// STACK (ArrayDeque)
stack.push(val)             // push to top
stack.pop()                 // remove top
stack.peek()                // view top (null if empty)
stack.isEmpty()
stack.size()

// QUEUE (ArrayDeque)
queue.offer(val)            // enqueue
queue.poll()                // dequeue (null if empty)
queue.peek()                // view front
queue.isEmpty()
queue.size()

// DEQUE (ArrayDeque)
deque.addFirst(val) / deque.offerFirst(val)
deque.addLast(val)  / deque.offerLast(val)
deque.pollFirst()   // remove from head
deque.pollLast()    // remove from tail
deque.peekFirst()
deque.peekLast()
```

Critical Java Gotchas

```java
// 1. Integer comparison: use equals() or Integer.compare()
Integer a = 128, b = 128;
a == b      // FALSE (beyond cache range)
a.equals(b) // TRUE

// 2. int[] sort vs Integer[] sort
int[] arr = {3,1,2};
Arrays.sort(arr);                          // ascending only
Integer[] arr2 = {3,1,2};
Arrays.sort(arr2, (a,b) -> b-a);          // can use comparator

// 3. Remove by index vs value in List
list.remove(0);                    // removes element at INDEX 0
list.remove(Integer.valueOf(0));   // removes element with VALUE 0

// 4. HashMap default behavior
map.getOrDefault(key, 0);          // safe default
map.computeIfAbsent(key, k -> new ArrayList<>()).add(val); // safe add to list

// 5. String immutability
String s = "hello";
s.concat(" world");  // s is UNCHANGED — must reassign
s = s + " world";    // creates new String
StringBuilder sb = new StringBuilder(s); // for mutation

// 6. Arrays.asList() returns fixed-size
List<Integer> fixed = Arrays.asList(1, 2, 3);
fixed.add(4); // THROWS UnsupportedOperationException!
new ArrayList<>(Arrays.asList(1,2,3)); // Mutable copy

// 7. Modulo negative numbers
int result = -7 % 3;  // = -1 in Java (NOT 2!)
int positive = ((n % m) + m) % m; // always positive
```

Pattern Recognition Quick Guide

Signal Words → Pattern Mapping

Signal	Pattern	Template Key
“Subarray/substring with constraint”	Sliding Window	expand right, shrink left
“Max/min window of size k”	Fixed Sliding Window	add right element, remove k-ago element
“Sorted array + two elements”	Two Pointers	left/right converge
“Fast/slow to detect cycle”	Floyd’s Cycle	fast=2x, slow=1x
“Sorted + find target”	Binary Search	left+right, mid = left+(right-left)/2
“Minimize max / maximize min”	Binary Search on Answer	condition() + binary search
“Sum of subarray from l to r”	Prefix Sum	prefix[r+1]-prefix[l]
“Count subarrays with sum k”	Prefix Sum + HashMap	prefixCount.get(sum-k)
“Two elements sum to target”	HashMap Complement	seen.contains(target-n)
“Frequency/group by”	HashMap	freq.getOrDefault(k,0)+1
“Next greater element”	Monotonic Stack	decreasing stack, pop when violated
“Histogram area”	Monotonic Stack	index stack, compute width on pop
“Tree traversal / path”	DFS recursive	null check, recurse left/right, combine
“Level by level”	BFS with queue	level-size loop pattern
“LCA”	Tree DFS	return non-null child
“Connected components”	BFS/DFS/UnionFind	visited[] + explore
“Cycle in directed graph”	3-color DFS	white/gray/black
“Topological order”	Kahn’s Algorithm	indegree[] + queue
“Shortest path, weighted”	Dijkstra	min-heap + dist[]
“Max/min with choices”	DP	dp[i] = max(options)
“Count ways to X”	DP	dp[i] += dp[i-choice]
“All subsets/combinations”	Backtracking	add, recurse, remove
“All permutations”	Backtracking	used[], pick any unused
“Top K elements”	Min-Heap size K	poll when size > k
“Merge K sorted”	Min-Heap	heap of (val, list, idx)

Signal	Pattern	Template Key
“Stream median”	Two Heaps	maxHeap lower, minHeap upper
“Overlapping intervals”	Sort by start + merge	curr.end = max(curr.end, next.end)
“Minimum rooms/ resources”	Heap end times	poll if freed, offer new end
“Greedy jump/reach”	Greedy	track maxReach, jump when forced
“Prefix search / autocomplete”	Trie	insert + DFS collect
“XOR duplicates/single”	Bit XOR	a^a=0, a^0=a
“Power of 2”	Bit Trick	n>0 && (n&(n-1))==0

The 3-Question Framework

For every problem: 1. **What’s the input type?** (sorted array / graph / string / tree) 2. **What’s the output?** (index / count / boolean / list / minimum) 3. **What constraint makes it hard?** (k distinct / sum = target / no overlap)

Time Budget per Pattern

Pattern	Expected Solve Time (Medium)
Sliding Window	15-20 min
Two Pointers	15-20 min
Binary Search	20-25 min
Prefix Sum	15-20 min
HashMap	10-15 min
Stack	20-25 min
Tree DFS/BFS	20-30 min
Graph	25-35 min
Dynamic Programming	30-40 min
Backtracking	25-35 min
Heap	20-25 min
Intervals	20-25 min